华沿机器人
HUAYAN ROBOTICS

Huayan Robot Library

C#



revision history

file status	[ ] Draft [√] Official release [ ] Under revision		
current version	V1.0.17.0		
imitate system	Huayan Robot System Software Group	date	May 12 , 2022
trial nuclear	Huayan robot system test group	date	May 12 , 2022
batch allow	Huayan Robot	date	May 12 , 2022
release date	May 12 , 2022		
effective date	May 12 , 2022		

A - Add M - Amend D - Delete

version number	version date	Supported Controller Versions	Release Notes
V1.0.0.0 _	2 022.05 . 12	20220314.180	A create file
V1.0.0.1 _	2 022.07 . 07	20220624.203	A New interface
V1.0.0.2 _	2 022.08 . 05	20220720.207	A New interface
V1.0.0.3 _	2 022.08.28	2 0220825.214	A Add HRIF_Set ToolMotion interface
V1.0.0.4	2023.02.08	HR5.50.3	A Added ReadPointByName interface
V1.0.0.5 _	2 023.02.16	2 0220825.214	A Add HRIF_ReadOverride interface
V1.0.1.0 _	2 023.02.17	HR 5.50.3	A Add HRIF _GetErrorCodeStr interface
V 1.0.2.0	2 023.03.03	HR 5.50.3	A Added 5 position following instructions: HRIF_SetPoseTrackingMaxMotionLimit HRIF_SetPoseTrackingPIDParams HRIF_SetPoseTrackingTargetPos HRIF_SetPoseTrackingState HRIF_SetUpdateTrackingPose
V 1.0.2.1	2 023.03.21	HR 5.50.7	M Modify the interface command: HRIF_GrpEnable _ HRIF_ Disable HRIF_GetForWordKin HRIF_GetInverseKin
V1.0.3.0	2023.08.15	HR5.51.9	A Add HRIF_EnableEndBTN
V1.0.3.1	2023.08.22	HR5.50.3	The valid range for the parameter nSlaveID in the functions ReadHoldingRegisters and WriteHoldingRegisters has been updated to 1~255.
V1.0.3.2	2023.11.09	HR6.2.17	A Added Force Control Command Interface: HRIF_SetFreeDriveMotionFreedom、 HRIF_SetFTFreeFactor、 HRIF_SetTangentForceBounds
V1.0.3.3	2023.11.28	HR6.2.17	M Modify the interface command: HRIF_MoveS、 HRIF_ReadCurWayPointID.
V1.0.4.0	2023.12.08	HR6.3.1	AAdded Virtual Wall Commands and Force Control Parameter Compensation Interface: HRIF_SetDepthThresholdForDampingArea HRIF_SetFreeDriveCompensateForce HRIF_AddSafePlane HRIF_ReadSafePlane HRIF_ReadSafePlaneList HRIF_DelSafePlane
V1.0.5.0	2023.12.12	HR6.3.1	A Added Update Virtual Wall Interface: HRIF_ UpdateSafePlane
V1.0.5.1	2024.1.11	HR6.3.3	M Modify the interface command: HRIF_ReadOveerride、

			HRIF_SetPoseTrackingState、 HRIF_ReadCmdJointVel、 HRIF_SetForceControlCoal
V1.0.5.2	2024.2.23	HR6.3.5	A Added Interface for Free-drive Activation Threshold, Linear Velocity, and Angular Velocity: HRIF_SetFTWrenchThresholds、 HRIF_SetMaxFreeDriveVel
V1.0.6.0	2024.3.26	HR6.3.6	A Added Online Joint Trajectory Planning Interface: HRIF_MovePathJOL、HRIF_GetMovePathJOLIndex
V1.0.6.1	2024.4.19	HR6.3.6	M Modified Speed Limit Range for General Motion Commands
V1.0.7.0	2024.7.17	HR7.1.5	A Added 10-Joint Related Interfaces： HRIF_SetJointMaxVel、HRIF_SetJointMaxAcc HRIF_SetMaxAcsRange、HRIF_ReadCmdJointPos HRIF_ReadCmdJointCur、HRIF_ReadActJointCur HRIF_ReadActJointPos、HRIF_ReadCmdJointVel HRIF_ReadActJointVel、HRIF_GetInverseKin HRIF_GetForwardKin、HRIF_WayPointRel、 HRIF_MoveJ、HRIF_MoveL HRIF_MoveC、HRIF_PushMovePathJ HRIF_ReadJointMaxVel、HRIF_ReadJointMaxJerk HRIF_ReadJointMaxAcc、HRIF_ReadPointByName HRIF_ReadActPos、HRIF_WayPointEx HRIF_WayPoint2、HRIF_WayPoint HRIF_ReadAxisErrorCode
V1.0.8.0	2024.8.10	HR6.4.6	A Added Free-drive Speed Mode Reading and APPData Interface： HRIF_SetFTFreeDriveSpeedMode;HRIF_ReadFTFreeDriveSpeedMode;HRIF_HRAppData;
V1.0.9.0	2024.9.10	HR6.4.8	A Added Interface： HRIF_ConfigTCP、HRIF_ConfigUCS、 HRIF_OpenBrake、HRIF_CloseBrake HRIF_MovePathJ、HRIF_MoveToSS、 HRIF_SetSteadyContactDeviationRange HRIF_CalTcp3P、HRIF_CalTcp4P HRIF_CalUcsLine、HRIF_DelPath HRIF_EndPushPathPoints、HRIF_InitPath HRIF_PushPathPoints、HRIF_EnterSafeGuard HRIF_cdsSetIO、HRIF_MoveCircularWeave HRIF_MoveLinearWeave、HRIF_ReadBrakeStatus HRIF_ReadMaxPayload、HRIF_ReadPathState HRIF_ReadPayload、HRIF_SetCollideLevel HRIF_SetMoveParamsAO、HRIF_SpeedJ HRIF_SpeedL、HRIF_UpdatePathName HRIF_ReadPathInfo、HRIF_SetSimulation HRIF_ReadPointList、HRIF_ReadTCPList HRIF_ReadUCSList、HRIF_ReadPathList HRIF_SetTriStageSwitch、 HRIF_ReadFTMotionFreedom HRIF_SetMaxSearchDistance

			HRIF_ReadTriStageSwitch
V1.0.9.1	2024.10.11	HR6.4.10	M Modify： Modify several typos Added "_nJ" suffix to the 10Joints-related interfaces.: HRIF_SetJointMaxVel、HRIF_SetJointMaxAcc HRIF_SetMaxAcsRange、HRIF_ReadCmdJointPos HRIF_ReadCmdJointCur、HRIF_ReadActJointCur HRIF_ReadActJointPos、HRIF_ReadCmdJointVel HRIF_ReadActJointVel、HRIF_GetInverseKin HRIF_GetForwardKin、HRIF_WayPointRel、 HRIF_MoveJ、HRIF_MoveL、HRIF_MoveC HRIF_ReadJointMaxVel、HRIF_ReadJointMaxJerk HRIF_ReadJointMaxAcc、HRIF_ReadPointByName HRIF_ReadActPos、HRIF_WayPointEx HRIF_WayPoint2、HRIF_WayPoint HRIF_ReadAxisErrorCode
V1.0.10.0	2024.10.28	HR6.4.12	M Modify several typos A Added Interface： HRIF_ReadDefaultScript，HRIF_SwitchScript
V1.0.11.0	2025.01.16	HR6.4.17	A Added Interface： HRIF_MoveAlignToZ HRIF_SetBaseInstallingAngle HRIF_GetBaseInstallingAngle
V1.0.12.0	2025.02.18	HR6.4.17_ZT	A Added Interface： HRIF_LoadIdentify HRIF_GetLoadIdentifyResult HRIF_SetFTCalibration HRIF_ReadForceData
V1.0.13.0	2025.03.04	HR6.5.10	M Modify： Modified Force Control Command Interfaces According to Newly Established Force Control Standard： SetFTWrenchThresholds 、 SetMaxFreeDriveVel 、 SetMaxSearchDistance、 SetSteadyContactDeviationRange、 SetTangentForceBounds、SetForceDataLimit A Added Interface： HRIF_SetLastCalibParams、 HRIF_GetLastCalibParams、 HRIF_ShortJogJ HRIF_ShortJogL HRIF_LongJogJ HRIF_LongJogL HRIF_LongMoveEvent
V1.0.14.0	2025.06.24	HR6.2.18	A Added Interface： HRIF_ReadAllBoxCI HRIF_ReadAllBoxCO HRIF_ReadAllBoxDI HRIF_ReadAllBoxDO
V1.0.14.1	2025.07.22	HR6.5.14	M Modify： HRIF_ReadDefaultScript、HRIF_SwitchScript

V1.0.15.0	2025.08.05	HR6.5.14	A Added Interface： HRIF_SetPathRefJoints
V1.0.16.0	2025.08.11	HR6.5.14	A Added Interface： HRIF_GetInverseDynamics HRIF_CalTCPOrt HRIF_SetDftTCP HRIF_ReadActCoord HRIF_ReadActCoord_nJ HRIF_ReadMaxRange HRIF_ReadMaxRange_nJ HRIF_SetFTUCS HRIF_ReadFTUCS
V1.0.16.1	2025.11.18	HR5.50.3	M Modify： HRIF_StartServo HRIF_SetBoxAOMode HRIF_MoveE
		HR6.5.14	HRIF_SetDftTCP
V1.0.17.0	2025-12-11	HR5.50.3	A Added： SDK Dynamic Library Namespace

Table of contents

CHAPTER1 OVERVIEW 1

1.1. purpose of writing 1
1.2. Terms and Conditions 1
1.3. noun analysis 1
1.4. type of data 2

CHAPTER2 ENVIRONMENT BUILD 3

2.1. create project 3
2.2. Add library files 3
2.3. initialization 4
2.4. Application Instructions for Extended Axis Interface 5

CHAPTER3 AVAILABLE INTERFACE 6

3.1. initialization 6
3.1.1. HRIF_Connect 6
3.1.2. HRIF_DisConnect 7
3.1.3. HRIF_IsConnected 8
3.1.4. HRIF_ShutdownRobot 9
3.1.5. HRIF_Connect2Box 10
3.1.6. HRIF_Electrify 11
3.1.7. HRIF_Blackout 12
3.1.8. HRIF_Connect2Controller 13
3.1.9. HRIF_IsSimulateRobot 14
3.1.10. HRIF_SetSimulation 14
3.1.11. HRIF_IsControllerStarted 16
3.1.12. HR IF_GetErrorCodeStr 17
3.1.13. HRIF_ReadVersion 18
3.1.14. HRIF_ReadRobotModel 20
3.2. Axes group control command 21
3.2.1. HRIF_GrpEnable 21
3.2.2. HRIF_GrpDisable 22
3.2.3. HRIF_GrpReset 23
3.2.4. HRIF_GrpStop 24
3.2.5. HRIF_Grpinterrupt 25
3.2.6. HRIF_GrpContinue 26
3.2.7. HRIF_GrpCloseFreeDriver 27
3.2.8. HRIF_GrpOpenFreeDriver 28
3.2.9. HRIF_OpenBrake 28
3.2.10. HRIF_CloseBrake 29
3.2.11. HRIF_ReadBrakeStatus 29

3.2.12. HRIF_SetTriStageSwitch 30
3.2.13. HRIF_ReadTriStageSwitch 30
3.2.14. HRIF_MoveToSS 31
3.2.15. HRIF_MoveAlignToZ 32
3.2.16. HRIF_LoadIdentify 33
3.2.17. HRIF_GetLoadIdentifyResult 33
3.3. script control command 35
3.3.1. HRIF_RunFunc 35
3.3.2. HRIF_StartScript 36
3.3.3. HRIF_StopScript 37
3.3.4. HRIF_PauseScript 38
3.3.5. HRIF_ContinueScript 39
3.3.6. HRIF_SwitchScript 39
3.3.7. HRIF_ReadDefaultScript 40
3.4. Electric box control command 41
3.4.1. HRIF_ReadBoxInfo 41
3.4.2. HRIF_ReadBoxCI 43
3.4.3. HRIF_ReadBoxDI 44
3.4.4. HRIF_ReadBoxCO 45
3.4.5. HRIF_ReadBoxDO 46
3.4.6. HRIF_ReadAllBoxCI 46
3.4.7. HRIF_ReadAllBoxDI 47
3.4.8. HRIF_ReadAllBoxCO 48
3.4.9. HRIF_ReadAllBoxDO 48
3.4.10. HRIF_ReadBoxAI 50
3.4.11. HRIF_ReadBoxAO 51
3.4.12. HRIF_SetBoxCO 52
3.4.13. HRIF_SetBoxDO 53
3.4.14. HRIF_SetBoxAOMode 54
3.4.15. HRIF_SetBoxAOVal 55
3.4.16. HRIF_SetEndDO 56
3.4.17. HRIF_ReadEndDI 57
3.4.18. HRIF_ReadEndDO 58
3.4.19. HRIF_ReadEndAI 59
3.4.20. HRIF_ReadEndBTN 60
3.4.21. HRIF_EnableEndBTN 61
3.4.22. HRIF_cdsSetIO 61
3.4.23. HRIF_SetMoveParamsAO 63
3.5. Status read and set instructions 65
3.5.1. HRIF_SetOverride 65
3.5.2. HRIF_SetToolMotion 66

3.5.3. HRIF_SetPayload 67
3.5.4. HRIF_SetJointMaxVel_nJ 67
3.5.5. HRIF_SetJointMaxVel 69
3.5.6. HRIF_SetJointMaxAcc 70
3.5.7. HRIF_SetJointMaxAcc_nJ 70
3.5.8. HRIF_SetLinearMaxVel 72
3.5.9. HRIF_SetLinearMaxAcc 73
3.5.10. HRIF_SetMaxAcsRange 74
3.5.11. HRIF_SetMaxAcsRange_nJ 75
3.5.12. HRIF_SetMaxPcsRange 77
3.5.13. HRIF_ReadMaxRange 79
3.5.14. HRIF_ReadMaxRange_nJ 81
3.5.15. HRIF_ReadOverride 83
3.5.16. HRIF_ReadJointMaxVel 84
3.5.17. HRIF_ReadJointMaxVel_nJ 85
3.5.18. HRIF_ReadJointMaxAcc 86
3.5.19. HRIF_ReadJointMaxAcc_nJ 87
3.5.20. HRIF_ReadJointMaxJerk 88
3.5.21. HRIF_ReadJointMaxJerk_nJ 89
3.5.22. HRIF_ReadLinearMaxSpeed 90
3.5.23. HRIF_ReadEmergencyInfo 91
3.5.24. HRIF_ReadRobotState 92
3.5.25. HRIF_ReadRobotFlags 94
3.5.26. HRIF_ReadCurWaypoint ID 95
3.5.27. HRIF_ReadPointList 95
3.5.28. HRIF_ReadAxisErrorCode 97
3.5.29. HRIF_ReadAxisErrorCode_nJ 98
3.5.30. HRIF_ReadCurFSM 99
3.5.31. HRIF_ReadCurFSMFromCPS 100
3.5.32. HRIF_ReadPointByName 101
3.5.33. HRIF_ReadPointByName_nJ 102
3.5.34. HRIF_EnterSafeGuard 104
3.5.35. HRIF_ReadMaxPayload 104
3.5.36. HRIF_ReadPayload 105
3.5.37. HRIF_SetCollideLevel 106
3.6. Position, speed, current reading instruction 107
3.6.1. HRIF_ReadActPos_nJ 107
3.6.2. HRIF_ReadActPos 108
3.6.3. HRIF_ReadCmdJointPos 110
3.6.4. HRIF_ReadCmdJointPos_nJ 110
3.6.5. HRIF_ReadActJointPos_nJ 111

3.6.6. HRIF_ReadActJointPos 113
3.6.7. HRIF_ReadCmdTcpPos 114
3.6.8. HRIF_ReadActTcpPos 115
3.6.9. HRIF_ReadCmdJointVel 116
3.6.10. HRIF_ReadCmdJointVel_nJ 116
3.6.11. HRIF_ReadActJointVel 118
3.6.12. HRIF_ReadActJointVel_nJ 118
3.6.13. HRIF_ReadCmdTcpVel 120
3.6.14. HRIF_ReadActTcpVel 121
3.6.15. HRIF_ReadCmdJointCur 122
3.6.16. HRIF_ReadCmdJointCur_nJ 122
3.6.17. HRIF_ReadActJointCur 124
3.6.18. HRIF_ReadActJointCur_nJ 124
3.6.19. HRIF_ReadTcpVelocity 126
3.7. Coordinate Transformation Calculation Instructions 127
3.7.1. HRIF_Quaternion2RPY 127
3.7.2. HR IF _RPY2Quaternion 128
3.7.3. HRIF_GetInverseKin 129
3.7.4. HRIF_GetInverseKin_nJ 131
3.7.5. HRIF_GetForwardKin 134
3.7.6. HRIF_GetForwardKin_nJ 135
3.7.7. HRIF_Base2UcsTcp 138
3.7.8. HRIF_UcsTcp2Base 140
3.7.9. HRIF_PoseAdd 142
3.7.10. HRIF_PoseSub 144
3.7.11. HRIF_PoseTrans 146
3.7.12. HRIF_PoseInverse 148
3.7.13. HRIF_PoseDist 150
3.7.14. HRIF_Poseinterpolate 152
3.7.15. HRIF_PoseDefdFrame 154
3.8. Tool coordinates and user coordinates read and write commands 156
3.8.1. HRIF_SetTCP 156
3.8.2. HRIF_SetUCS 157
3.8.3. HRIF_ReadCurTCP 158
3.8.4. HRIF_ReadCurUCS 159
3.8.5. HRIF_SetTCPByName 160
3.8.6. HRIF_SetUCSByName 161
3.8.7. HRIF_ReadTCPByName 162
3.8.8. HRIF_ReadUCSByName 163
3.8.9. HRIF_ReadTCPList 164
3.8.10. HRIF_ReadUCSList 164

3.8.11. HRIF_ConfigTCP 165
3.8.12. HRIF_CalUcsLine 166
3.8.13. HRIF_ConfigUCS 167
3.8.14. HRIF_CalTcp3P 168
3.8.15. HRIF_CalTcp4P 170
3.8.16. HRIF_CalTCPOrt 172
3.8.17. HRIF_GetInverseDynamics 173
3.8.18. HRIF_SetBaseInstallingAngle 175
3.8.19. HRIF_GetBaseInstallingAngle 175
3.8.20. HRIF_SetDftTCP 176
3.8.21. HRIF_ReadActCoord 177
3.8.22. HRIF_HRIF_ReadActCoord_nJ 178
3.9. Force control command 180
3.9.1. HRIF_SetForceControlState 180
3.9.2. HRIF_ReadForceControlState 181
3.9.3. HRIF_SetForceToolCoordinateMotion 182
3.9.4. HRIF_ForceControlInterrupt 183
3.9.5. HRIF_ForceControlContinue 184
3.9.6. HRIF_SetForceZero 185
3.9.7. HRIF_SetMaxSearchVelocities 186
3.9.8. HRIF_SetMaxSearchDistance 186
3.9.9. HRIF_ReadFTMotionFreedom 187
3.9.10. HRIF_SetControlFreedom 189
3.9.11. HRIF_SetForceControlStrategy 190
3.9.12. HRIF_SetFreeDrivePositionAndOrientation 191
3.9.13. HRIF_SetPIDControlParams 192
3.9.14. HRIF_SetMassParams 193
3.9.15. HRIF_SetDampParams 194
3.9.16. HRIF_SetStiffParams 195
3.9.17. HRIF_SetForceControlGoal 196
3.9.18. HRIF_SetControlGoal 197
3.9.19. HRIF_SetForceDataLimit 199
3.9.20. HRIF_SetForceDistanceLimit 200
3.9.21. HRIF_SetForceFreeDriveMode 201
3.9.22. HRIF_ReadFTFreeDriveSpeedMode 202
3.9.23. HRIF_SetFTFreeDriveSpeedMode 202
3.9.24. HRIF_ReadFTCabData 204
3.9.25. HRIF_SetFreeDriveMotionFreedom 204
3.9.26. HRIF_SetFTFreeFactor 205
3.9.27. HRIF_SetTangentForceBounds 206
3.9.28. HRIF_SetFreeDriveCompensateForce 206

3.9.29. HRIF_SetDepthThresholdForDampingArea 207
3.9.30. HRIF_AddSafePlane 208
3.9.31. HRIF_UpdateSafePlane 209
3.9.32. HRIF_DelSafePlane 210
3.9.33. HRIF_ReadSafePlaneList 211
3.9.34. HRIF_ReadSafePlane 212
3.9.35. HRIF_SetFTWrenchThresholds 213
3.9.36. HRIF_SetMaxFreeDriveVel 214
3.9.37. HRIF_SetSteadyContactDeviationRange 215
3.9.38. HRIF_SetFTCalibration 216
3.9.39. HRIF_ReadForceData 218
3.9.40. HRIF_SetInitializeForceSensor 219
3.9.41. HRIF_GetLastCalibParams 221
3.9.42. HRIF_SetFTUCS 223
3.9.43. HRIF_ReadFTUCS 224
3.10. General Motion Control Instructions 225
3.10.1. HRIF_MoveRelJ 225
3.10.2. HRIF_MoveRelL 226
3.10.3. HRIF_WayPointRel 227
3.10.4. HRIF_WayPointRel_nJ 231
3.10.5. HRIF_IsMotionDone 234
3.10.6. HRIF_IsBlendingDone 235
3.10.7. HRIF_WayPointEx 236
3.10.8. HRIF_WayPointEx_nJ 240
3.10.9. HRIF_WayPoint 244
3.10.10. HRIF_WayPoint_nJ 247
3.10.11. HRIF_WayPoint2 250
3.10.12. HRIF_WayPoint2_nJ 254
3.10.13. HRIF_MoveJ 258
3.10.14. HRIF_MoveJ_nJ 261
3.10.15. HRIF_MoveL 264
3.10.16. HRIF_MoveL_nJ 267
3.10.17. HRIF_MoveC 270
3.10.18. HRIF_MoveC_nJ 273
3.10.19. HRIF_MoveZ 276
3.10.20. HRIF_MoveE 279
3.10.21. HRIF_MoveS 282
3.10.22. HRIF_MoveLinearWeave 284
3.10.23. MoveCircularWeave 287
3.10.24. HRIF_ShortJogJ 291
3.10.25. HRIF_ShortJogL 292

3.10.26. HRIF_LongJogJ 293
3.10.27. HRIF_LongJogL 294
3.10.28. HRIF_LongMoveEvent 295
3.11. Continuous trajectory motion control instructions 296
3.11.1. HRIF_InitPath 296
3.11.2. HRIF_PushMovePathL 299
3.11.3. HRIF_DelPathJ 300
3.11.4. HRIF_StartPushMovePathJ 301
3.11.5. HRIF_PushMovePathJ 302
3.11.6. HRIF_EndPushMovePathJ 303
3.11.7. HRIF_MovePathJ 304
3.11.8. HRIF_ReadMovePathJState 305
3.11.9. HRIF_DelMovePathJ 306
3.11.10. HRIF_ReadTrackProcess 307
3.11.11. HRIF_InitMovePathL 308
3.11.12. HRIF_PushMovePathL 310
3.11.13. HRIF_PushMovePaths 311
3.11.14. HRIF_MovePathL 313
3.11.15. HRIF_SetMovePathOverride 314
3.11.16. HRIF_EndPushPathPoints 315
3.11.17. HRIF_MovePathJOL 316
3.11.18. HRIF_GetMovePathJOLIndex 320
3.11.19. HRIF_ReadPathState 321
3.11.20. HRIF_ReadPathInfo 323
3.11.21. HRIF_ReadPathList 325
3.11.22. HRIF_UpdateMovePathJName 326
3.11.23. HRIF_UpdatePathName 327
3.11.24. HRIF_SetPathRefJoints 328
3.12. Servo Motion Control Instructions 329
3.12.1. HRIF_StartServo 329
3.12.2. HRIF_PushServoJ 331
3.12.3. HRIF_PushServoP 332
3.12.4. HRIF_SpeedJ 334
3.12.5. HRIF_SpeedL 335
3.13. Relative tracking motion control commands 337
3.13.1. HRIF_SetMoveTraceParams 337
3.13.2. HRIF_SetMoveTraceInitParams 338
3.13.3. HRIF_SetMoveTraceUcs 339
3.13.4. HRIF_SetTrackingState 340
3.14. Position follow motion instruction 341
3.14.1. HRIF_SetPoseTrackingMaxMotionLimit 341

3.14.2. HRIF_SetPoseTrackingPIDParams 342
3.14.3. HRIF_SetPoseTrackingTargetPos 344
3.14.4. HRIF_SetPoseTrackingState 346
3.14.5. HRIF_SetUpdateTrackingPose 347
3.15. other instructions 349
3.15.1. HRIF_HRAppCmd 349
3.15.2. HRIF_HRAppData 350
3.15.3. HRIF_WriteEndHoldingRegisters 351
3.15.4. HRIF_ReadEndHoldingRegisters 353

CHAPTER4 APPENDIX 355

4.1. reference document 355

Chapter1 overview

1.1. purpose of writing

1. For using Huayan C# Interface personnel provide interface usage instructions;
2. Provide reference documents for developers to write and maintain interfaces.

1.2. Terms and Conditions

All code examples in the documentation default to no interference within the robot's workspace.

1. SDK use:

   will RobotLibrarys.dll file and Newtonsoft.Json.dll Copy it to the corresponding project directory folder /bin/Debug .

2. SDK The normal use needs to include the dynamic library file, and the dynamic library version number query:

   click RobotLibrarys.dll file, select properties, and view the version information in "Details" .

*Note: Prior to C# SDK version v1.0.17.0, the RobotLibrarys.dll file was named HansRobotLibrarys.dll, and the class RobotAPI was named HansRobotAPI.

1.3. noun analysis

proper name	explain
ACS	Joint coordinates, in degrees (°)
PCS	Space coordinates in millimeters (mm) , degrees (°)
TCP	The system default tool coordinate system, the initial value is 0,0,0,0,0,0
UCS	user coordinate system
Base	The default user coordinate system of the system, the initial value is 0 , 0,0,0,0,0
Tool	System default tool coordinate system

1.4. type of data

type of data	length	illustrate
char	1 byte	character type
bool	1 byte	Boolean
int	4 bytes	unsigned integer
int	4 bytes	integer
double	8 bytes	double precision floating point
string	-	string
List	-	the list

Chapter2 Environment build

2.1. create project

Open Visual Studio 2022 and create C# Console application:

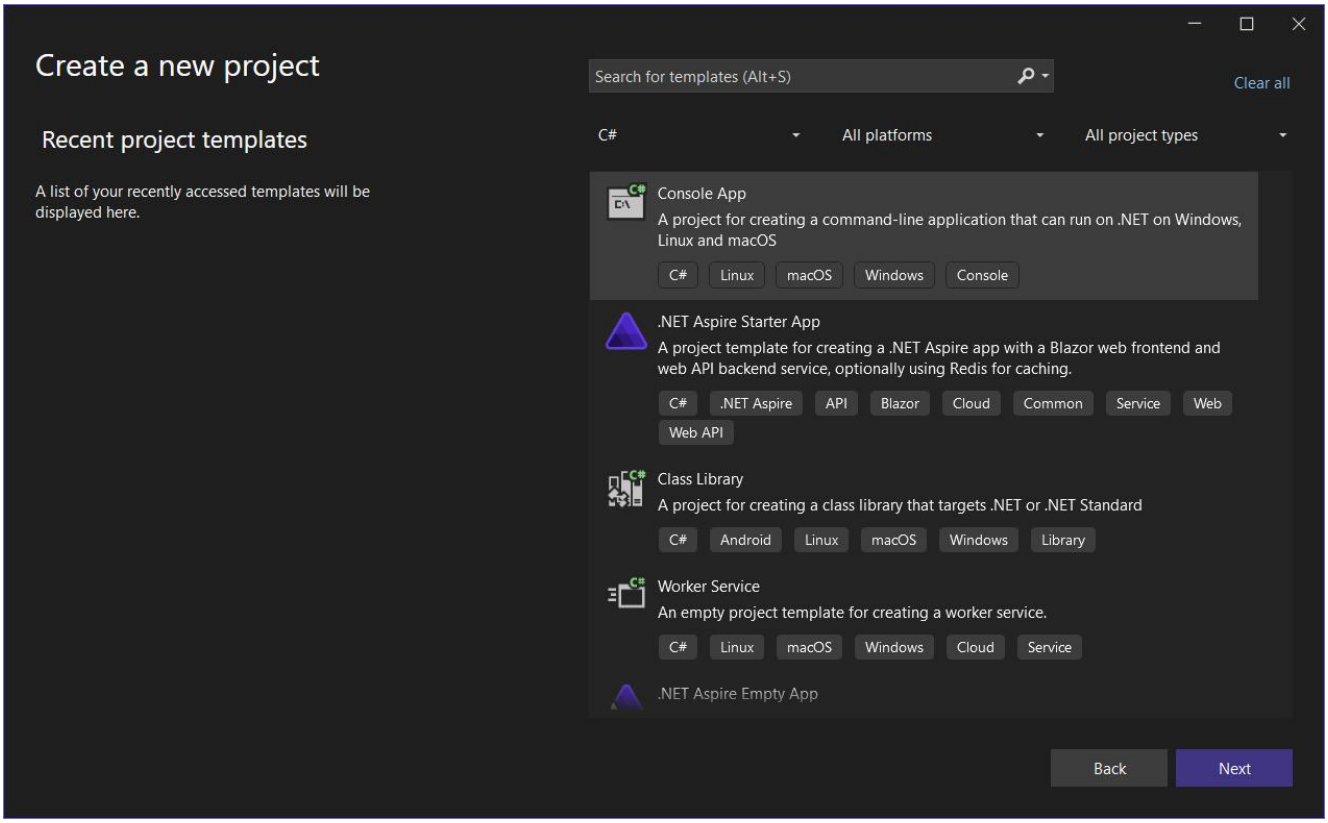


Figure 2-1 : Create a project

2.2. Add library files

Import the file, making sure the file is imported into the dependencies subordinate assembly.Right-click on Dependencies, select Add Project Reference. In the pop-up Reference Manager, select Browse, load the two library files (RobotLibrarys.dll and Newtonsoft.dll) of the C# SDK, then select OK to complete the library file loading.

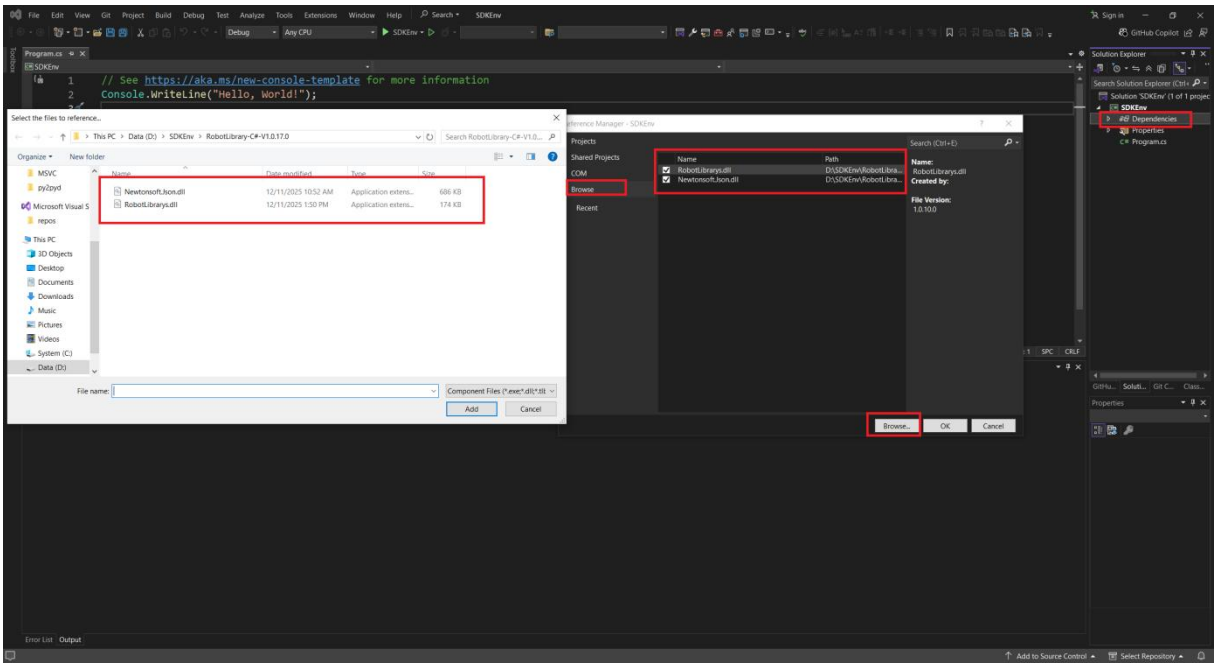
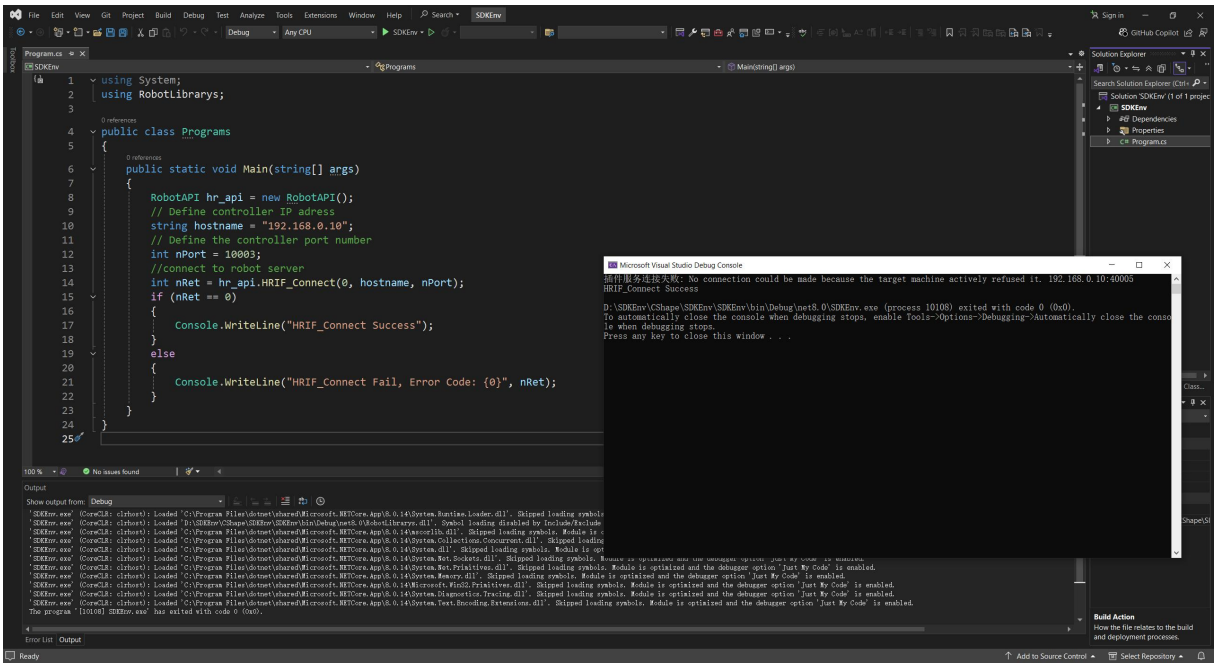


Figure 2-2 : Add library files

2.3. initialization

In the project, import the RobotLibrarys namespace, initialize the RobotAPI interface object (the instance name such as hr_api is customizable), configure the robot's IP address and port, and call the Connect function to establish communication with the robot. If the connection fails, check the network, IP/port configuration, or robot status; syntax errors in the code may also cause execution failure.



```
using System;
using RobotLibrarys;

public class Programs
{
    public static void Main(string[] args)
    {
        RobotAPI hr_api = new RobotAPI();
        // Define controller IP adress
        string hostname = "192.168.0.10";
        // Define the controller port number
        int nPort = 10003;
        //connect to robot server
        int nRet = hr_api.HRIF_Connect(0, hostname, nPort);
        if (nRet == 0)
        {
            Console.WriteLine("HRIF_Connect Success");
        }
        else
        {
            Console.WriteLine("HRIF_Connect Fail, Error Code: {0}", nRet);
        }
    }
}
```

Figure 2-3 : Run the program

2.4. Application Instructions for Extended Axis Interface

For certain machine models, the axis groups (in addition to the six native axes of the robot body) uniformly use the extended axis interface.

The data structure used by the extended axis interface is as follows:

```
public class JointsData
{
    public double[] joint = new double[10];
}
public struct Pose
{
    public double dX;
    public double dY;
    public double dZ;
    public double dRX;
    public double dRY;
    public double dRZ;
}
```

Figure 2-4 : Data Structure for Extended Axes

Chapter3 available interface

3.1. initialization

3.1.1. HRIF_Connect

3.1.1.1. Description: Connect to the robot server.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0 ~ 5	Electric box ID number, default=0
hostName	Controller IP address	string	-	Controller IP address, According to the actual IP address definition
nPort	The port number	int	10003	Controller port number, default = 10003

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.1.2. example

```
// Define controller IP address
string hostname = "192.168.0.10";
// Define the controller port number
int nPort = 10003;
// connect to robot server
int nRet= hr_api.HRIF_Connect(0, hostname, nPort);
```

3.1.2. HRIF_DisConnect

3.1.2.1. Description: Disconnect the robot server.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.2.2. example

```
// disconnect the robot server
int nRet = hr_api.HRIF_DisConnect(0);
```

3.1.3. HRIF_IsConnected

3.1.3.1. Description: Determine whether the controller is connected.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
bRet	return value	bool	false,true	false : the controller is not connected true : the controller is connected

3.1.3.2. example

```
// Whether the controller is connected
bool n Ret = hr_api.HRIF_IsConnected(0);
```

3.1.4. HRIF_ShutdownRobot

3.1.4.1. Description: The controller is powered off (the robot is powered off and the system shuts down).

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nCmdType	power outage type	int	1~2	1 : Stop motion - disable - cut off 48V power supply 2 : stop motion - disable enable - cut off 48V power supply - restart operation system

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.4.2. example

```
//Power off type
int nCmdType = 1;
// controller power off
int nRet = hr_api.HRIF_ShutdownRobot(0,nCmdType);
```

3.1.5. HRIF_Connect2Box

3.1.5.1. Description: Connect the controller electric box.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.5.2. example

```
// Connect the controller electric box
int nRet= hr_api.HRIF_Connect2Box(0);
```

3.1.6. HRIF_Electrify

3.1.6.1. Description: The robot is powered on.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.6.2. example

```
// Power on the robot
int nRet = hr_api.HRIF_Electrify(0);
```

3.1.7. HRIF_Blackout

3.1.7.1. Description: The robot lost power.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.7.2. example

```
// robot powered off
int nRet= hr_api.HRIF_BlackOut(0);
```

3.1.8. HRIF_Connect2Controller

3.1.8.1. Description: Connect the controller.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.8.2. example

```
// connect to the robot
int nRet= hr_api.HRIF_Connect2Controller(0);
```

3.1.9. HRIF_IsSimulateRobot

3.1.9.1. Description: Whether it is a simulated robot.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nSimulateRobot	Whether to simulate a robot	int	0/1	0: real robot 1: Simulation robot

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.9.2. example

```
// Define whether it is a simulated robot
int nSimulateRobot = 0;
// Determine whether it is a simulated robot
int nRet= hr_api.HRIF_IsSimulateRobot(0, ref nSimulateRobot);
```

3.1.10. HRIF_SetSimulation

3.1.10.1. Description: Sets the simulation state of the robot.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0 ~ 5	Robot ID number. Default value = 0.
nSimulation	Simulation Flag	int	0 / 1	0: Real robot 1: Simulated robot

*Note: This command can only be called successfully when the robot is in power-off mode. Otherwise, error code 20018 will be returned.

✓ return value

return value	name	type of data	Effective	content

			range	
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.10.2. example

// Set the current robot to simulation mode

int nRet = hr_api.HRIF_SetSimulation(0,0, 1);

3.1.11. HRIF_IsControllerStarted

3.1.11.1. Description: Whether the controller startup is complete.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nStarted	Whether the startup is complete	int	0/1	0: not activated 1: activated

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.11.2. example

```
// Define whether to start the completion variable
int nStarted = 0;
// Determine whether the robot is started
int nRet= hr_api.HRIF_IsControllerStarted(0, ref nStarted);
```

3.1.12.HR IF_GetErrorCodeStr

3.1.12.1. Description: Get the error code explanation.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	int	0~5	Electric box ID number, default=0
nErrorCode	error code	int	-	error code

✓ output variable

output variable	name	type of data	Effective range	content
strErrorMsg	Error code description	String Data	-	The specific description of the error code

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.12.2. example

```
// error code
int nErrorCode = 20018;
// error code description
String strErrorMsg = " " ;
// call interface
int nRet = hr_api.HRIF_GetErrorCodeStr(0,nErrorCode,ref strErrorMsg);
// Get error code description
```

3.1.13.HRIF_ReadVersion

3.1.13.1. Description: Read the controller version number.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
strVer	Overall version information	string	-	The format of the version number is separated by ".", and the meanings of the numbers are as follows: nCPSVersion nCodesysVersion nBoxVerMajor nBoxVerMid nBoxVerMin nAlgorithmVer nElfinFirmwareVer
nCPSVersion	CPS version	int	-	CPS version number
nCodesysVersion	Controller version	int	-	Controller version number
nBoxVerMajor	Electric box version	int	0~4	Electric box version number: 0: Analog electric box 1~4: Electric box version number
nBoxVerMid	Electrical Box Firmware Version	int	-	Electrical Box Firmware Version
nBoxVerMin	Electrical Box Firmware Version	int	-	Electrical Box Firmware Version
nAlgorithmVer	algorithm version	int	-	algorithm version
nElfinFirmwareVer	Firmware version	int	-	Firmware version

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.13.2. example

// Get the overall version number

```
string strVer = " ";
// Get the CPS version number
int nCPSVersion = 0;
// Get the controller version number
int nCodesysVersion = 0;
// Get the electric box version
int nBoxVerMajor = 0;
// Get the firmware version of the electrical box
int nBoxVerMid = 0;
// Get the firmware version of the electrical box
int nBoxVerMin = 0;
// Get algorithm version
int nAlgorithmVer = 0;
// Get the driver firmware version
int nElfinFirmwareVer = 0;
// Get the version number
int nRet= hr_api.HRIF_ReadVersion(0,0, ref strVer, ref nCPSVersion, ref nCodesysVersion, ref nBoxVerMajor, ref nBoxVerMid, ref nBoxVerMin, ref nAlgorithmVer, ref nElfinFirmwareVer);
```

3.1.14.HRIF_ReadRobotModel

3.1.14.1. Description: Read the robot type.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
strModel	robot type	string	-	robot type

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.1.14.2. example

```
// get robot type
string strModel = " ";
// read robot type
int nRet= hr_api.HRIF_ReadRobotModel(0, ref strModel);
```

3.2. Axes group control command

3.2.1. HRIF_GrpEnable

3.2.1.1. Description: Robot enable command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.1.2. example

```
// robot enable
int nRet = hr_api.HRIF_GrpEnable(0,0);
```

3.2.2. HRIF_GrpDisable

3.2.2.1. Description: Robot disable command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.2.2. example

```
// robot disable
int nRet = hr_api.HRIF_GrpDisable(0,0);
```

3.2.3. HRIF_GrpReset

3.2.3.1. Description: Robot reset command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.3.2. example

```
// robot reset
int nRet = hr_api.HRIF_GrpReset(0,0);
```

3.2.4. HRIF_GrpStop

3.2.4.1. Description: Robot stop motion command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.4.2. example

```
// robot stops moving
int nRet = hr_api.HRIF_GrpStop(0,0);
```

3.2.5. HRIF_Grpinterrupt

3.2.5.1. Description: Robot pause motion command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.5.2. example

```
// robot pauses motion
int nRet = hr_api.HRIF_GrpInterrupt(0,0);
```

3.2.6. HRIF_GrpContinue

3.2.6.1. Description: Robot continue motion command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.6.2. example

```
// robot continues to move
int nRet = hr_api.HRIF_GrpContinue(0,0);
```

3.2.7. HRIF_GrpCloseFreeDriver

3.2.7.1. Description: Turn off free drive.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.7.2. example

```
// Robot turns off free drive
int nRet= hr_api.HRIF_GrpCloseFreeDriver(0,0);
```

3.2.8. HRIF_GrpOpenFreeDriver

3.2.8.1. Description: Open the free drive.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.8.2. example

```
// The robot turns on free drive
int nRet = hr_api.HRIF_GrpOpenFreeDriver(0,0);
```

3.2.9. HRIF_OpenBrake

3.2.9.1. Description: Releases the brake.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nAxisID	Axis ID	int	0 ~ 5	Target axis ID for brake release, corresponding to joints J1-J6.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.9.2. example

```
// Release brake for joint 6
int nRet = hr_api.HRIF_OpenBrake(0,0,5);
```

3.2.10.HRIF_CloseBrake

3.2.10.1. Description: Engages the brake.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nAxisID	Axis ID	int	0 ~ 5	Target axis ID for brake release, corresponding to joints J1-J6.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.10.2. example

```
// Engages brake for joint 6
int nRet = hr_api.HRIF_CloseBrake(0,0,5);
```

3.2.11.HRIF_ReadBrakeStatus

3.2.11.1. Description: Reads the brake status (released/engaged) for all joints.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
stateJ1 ~ stateJ6	Brake Status	int	0/1	Brake status for joints 1-6

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.11.2. example

int stateJ1; int stateJ2; int stateJ3;

int stateJ4; int stateJ5; int stateJ6;

// Read brake status for all joints

int nRet = hr_api.HRIF_ReadBrakeStatus(0,0,ref stateJ1,ref stateJ2,ref stateJ3,ref stateJ4,ref stateJ5,ref stateJ6);

3.2.12.HRIF_SetTriStageSwitch

3.2.12.1. Description: Sets the state and functional mode of the three-stage switch.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nThreeStageStatus	Three-stage Switch State	int	0 / 1	0: Disable three-stage switch 1: Enable three-stage switch
nThreeStageMode	Three-stage Switch Mode	int	0 / 1	0: Zero-force teaching 1: Enable (normal operation)

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.12.2. example

// Enable three-stage switch

int nThreeStageStatus=1:

// Set mode to zero-force teaching

int nThreeStageMode=0:

// Configure the three-stage switch

int nRet = hr_api.HRIF_SetTriStageSwitch(0,0,nThreeStageStatus,nThreeStageMode);

3.2.13.HRIF_ReadTriStageSwitch

3.2.13.1. Description: Reads the current state and mode of the three-stage switch.

✓ input variable

input variable	name	type of data	Effective	content

			range	
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nThreeStageStatus	Three-stage Switch State	int	0 / 1	0: Disabled 1: Enabled
nThreeStageMode	Three-stage Switch Mode	int	0 / 1	0: Zero-force teaching 1: Enable (normal operation)

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.13.2. example

```
// Initialize Three-stage Switch State
int nThreeStageStatus=0:
// Initialize Three-stage Switch Mode
int nThreeStageMode=0:
// Read three-stage switch status and mode
int nRet = hr_api.HRIF_ReadTriStageSwitch(0,0,ref nThreeStageStatus,ref nThreeStageMode);
```

3.2.14.HRIF_MoveToSS

3.2.14.1. Description: Moves the robot to a predefined safe position.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.14.2. example

// Move to safe position

int nRet = hr_api.HRIF_MoveToSS(0,0);

3.2.15.HRIF_MoveAlignToZ

3.2.15.1. Description: Aligns the robot's tool Z-axis to a target orientation.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
TCP	TCP	string	-	Name of the Tool Center Point frame to use.
UCS	UCS	string	-	Name of the User Coordinate System frame to use.

✓ output variable

output variable	name	type of data	Effective range	content
bIsReached	Alignment Flag	bool	true / false	true: Z-axis alignment achieved false: Z-axis alignment not yet achieved
dJ1 ~ dJ6	Joint Coordinates	double	-	Joint positions during alignment: dJ1: Joint 1 coordinate [°] dJ2: Joint 2 coordinate [°] dJ3: Joint 3 coordinate [°] dJ4: Joint 4 coordinate [°] dJ5: Joint 5 coordinate [°] dJ6: Joint 6 coordinate [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.15.2. example

// Define tool coordinate variable

string TCP= "TCP";

// Define user coordinate variable

string UCS= "Base";

```
bool bIsReached = false;
// Define the joint positions where the computed Z-axis alignment is located
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0; double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
while (!bIsReached)
{
    int nRet = hr_api.HRIF_MoveAlignToZ(0, 0, TCP, UCS, bIsReached, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6);
    if (nRet != 0)
    {
        break;
    }
    // HRIF_MoveAlignToZ must be used in conjunction with HRIF_LongMoveEvent
    HRIF_LongMoveEvent(0, 0);
}
```

3.2.16.HRIF_LoadIdentify

3.2.16.1. Description: Initiates the payload identification process.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.16.2. example

```
// Start payload identification
int nRet = hr_api.HRIF_LoadIdentify(0,0);
```

3.2.17.HRIF_GetLoadIdentifyResult

3.2.17.1. Description: Retrieves the results of the payload identification process.

✓ input variable

input variable	name	type of data	Effective	content

			range	
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
status	Identification Status	int	0 ~ 7	0: Success 1: Trajectory 1 error 2: Trajectory 2 error 3: Trajectory 3 error 4: Identification in progress 5: Unexpected stop 6: Unexpected exit 7: Initialized, identification not started
progress	Progress	int	0 ~ 100	Current identification progress percentage
mass	Payload Mass	double	-	Identified payload mass [kg]
cx	Center of Mass X	double	-	X-coordinate of payload center of mass
cy	Center of Mass Y	double	-	Y-coordinate of payload center of mass
cz	Center of Mass Z	double	-	Z-coordinate of payload center of mass

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.2.17.2. example

```
// Initialize variables
int status=0;int progress=0;double mass=0.0;double dCx=0.0;double dCy=0.0;double dCz=0.0;
// Get payload identification results
int nRet= hr_api.HRIF_GetLoadIdentifyResult(0, 0, ref status, ref progress, ref mass, ref dCx, ref dCy, ref dCz);
```

3.3. script control command

3.3.1. HRIF_RunFunc

3.3.1.1. Description: Run the specified script function.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
strFuncName	function name	string	-	Specify the name of the script function, which corresponds to the function of the teach pendant interface
Param	Parameters Table	vector<string>	-	parameter list

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.3.1.2. example

```
// Specify the running function Func_1
string strFuncName = "Func_1";
// define input parameters
List<string> param = new List<string>();
// run the function Func_1
int nRet= hr_api.HRIF_RunFunc(0, strFuncName, param);
```

3.3.2. HRIF_StartScript

3.3.2.1. execute the compiled script file Func_main on the teach pendant page after calling function.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.3.2.2. example

```
// run the script Main function
int nRet = hr_api.HRIF_StartScript(0);
```

3.3.3. HRIF_StopScript

3.3.3.1. Description: Stop the script, and stop the teach pendant page after calling to execute the script file.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.3.3.2. example

```
// stop the currently running script
int nRet = hr_api.HRIF_StopScript(0);
```

3.3.4. HRIF_PauseScript

3.3.4.1. Description: Pause the script. After calling, the page of the teach pendant is paused and the script file is being executed.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.3.4.2. example

```
// Pause the currently running script
int nRet = hr_api.HRIF_PauseScript(0);
```

3.3.5. HRIF_ContinueScript

3.3.5.1. Description: Continue the script, and continue to run the script file that has been paused on the teach pendant page after calling it.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.3.5.2. example

```
// Continue running the currently paused script
int nRet = hr_api.HRIF_ContinueScript(0);
```

3.3.6. HRIF_SwitchScript

3.3.6.1. Description: Reads the name of the script program currently applied by the system.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nRbtID	Robot ID	int	0 ~ 5	Robot ID number.
sScriptName	Script Program Name	string	-	Name of the script program file, must have a .json extension.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.3.6.2. example

```
// Specify the script file "Script1.json"
string sScriptName= "Script1.json";
```

// Apply the specified script "Script1.json"

int nRet= hr_api.HRIF_SwitchScript(0, 0,sScriptName);

3.3.7. HRIF_ReadDefaultScript

3.3.7.1. Description: Reads the name of the script program currently applied by the system.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
sScriptName	Script Program Name	string	-	Name of the currently applied script program, with a .json extension.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.3.7.2. example

// Read the name of the currently running script program

int nRet= hr_api.HRIF_ReadDefaultScript(0,ref sScriptName);

3.4. Electric box control command

3.4.1. HRIF_ReadBoxInfo

3.4.1.1. Description: Read the electrical box information.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nConnected	Electric box connection status	int	0/1	0: not connected 1: connected
n48V_ON	48V voltage state	int	0/1	0: voltage<36V 1: Voltage>36V
db48OUT_Voltag	48V output voltage value	double	0~53	48V output voltage value
db48OUT_Current	48V output current value	double	-	48V output current value
nRemote BTN	Remote shutdown button status	int	0/1	0: no signal 1: There is a signal
nThreeStageBTN	Three button states	int	0/1	0: no signal 1: There is a signal

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.1.2. example

```
// Define whether the electrical box is connected
int nConnected = 0;
// 48V power-on state
int n48V_ON = 0;
// 48V voltage value
double db48OUT_Voltag = 0;
// 48V current value
```

```
double db48OUT_Current = 0;
// remote button state
int nRemoteBTN = 0;
// three button states
int nThreeStageBTN = 0;
// read electrical box information
int nRet= hr_api.HRIF_ReadBoxInfo(0,0, ref nConnected, ref n48V_ON, ref db48OUT_Voltag,
ref db48OUT_Current, ref nRemoteBTN, ref nThreeStageBTN);
```

3.4.2. HRIF_ReadBoxCI

3.4.2.1. Description: Read the electrical box control digital input status.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	Control digital input bit	int	0~7	Control Digital Input Bit Index

✓ output variable

output variable	name	type of data	Effective range	content
nVal	Control the corresponding bit state of the digital input	int	0/1	0: low level 1: high level

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.2.2. example

```
// read the result
int nVal = 0;
// Define the bits to be read
int nBit = 0;
// Read the nBit CI status of the electrical box
int nRet= hr_api.HRIF_ReadBoxCI(0, nBit, ref nVal);
```

3.4.3. HRIF_ReadBoxDI

3.4.3.1. Description: Read the status of the general digital input of the electrical box.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	General purpose digital input bit	int	0~7	General Digital Input Bit Index

✓ output variable

output variable	name	type of data	Effective range	content
nVal	General digital input corresponding bit state	int	0/1	0: low level 1: high level

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.3.2. example

```
// read the result
int nVal = 0;
// Define the bits to be read
int nBit = 0;
// Read the nBit DI status of the electrical box
int nRet= hr_api.HRIF_ReadBoxDI(0, nBit, ref nVal);
```

3.4.4. HRIF_ReadBoxCO

3.4.4.1. Description: Read the electrical box control digital output status.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	Control Digital Output Bits	int	0~7	Control Digital Output Bit Index

✓ output variable

output variable	name	type of data	Effective range	content
nVal	Control the corresponding bit state of the digital output	int	0/1	0: low level 1: high level

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.4.2. example

```
// read the result
int nVal = 0;
// Define the bits to be read
int nBit = 0;
// Read the nBit bit CO status of the electrical box
int nRet= hr_api.HRIF_ReadBoxCO(0, nBit, ref nVal);
```

3.4.5. HRIF_ReadBoxDO

3.4.5.1. Description: Read the status of the electrical box general digital output.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	General purpose digital output bits	int	0~7	General Digital Output Bit Index

✓ output variable

output variable	name	type of data	Effective range	content
nVal	General digital output corresponding bit status	int	0/1	0: low level 1: high level

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.5.2. example

```
// read the result
int nVal = 0;
// Define the bits to be read
int nBit = 0;
// Read the nBit bit DO status of the electrical box
int nRet= hr_api.HRIF_ReadBoxDO(0, nBit, ref nVal);
```

3.4.6. HRIF_ReadAllBoxCI

3.4.6.1. Description: Reads the status of all control digital inputs (CI) on the control box.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ output variable

output	name	type of data	Effective	content

variable			range	
status	Control Input Status Bits	IOStatus	-	Status values for 8 CI bits: status.Bit0 ~ status.Bit7: Status of each control input (0/1).

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.6.2. example

```
// Create CI status structure
IOStatus ciStatus = new IOStatus();

// Read status of all CI bits on the control box at once
int nRet = hr_api.HRIF_ReadAllBoxCI(0, ref ciStatus);
```

3.4.7. HRIF_ReadAllBoxDI

3.4.7.1. Description: Reads the status of all general-purpose digital inputs (DI) on the control box.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
status	General Input Status Bits	IOStatus	-	Status values for 8 DI bits: status.Bit0 ~ status.Bit7: Status of each general input (0/1).

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.7.2. example

```
// Create DI status structure
IOStatus diStatus = new IOStatus();
```

// Read status of all DI bits on the control box at once

int nRet = hr_api.HRIF_ReadAllBoxDI(0, ref diStatus);

3.4.8. HRIF_ReadAllBoxCO

3.4.8.1. Description: Reads the status of all control digital outputs (CO) on the control box.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
status	Control Output Status Bits	IOStatus	-	Status values for 8 CO bits: status.Bit0 ~ status.Bit7: Status of each control output (0/1).

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.8.2. example

// Create CO status structure

IOStatus coStatus = new IOStatus();

// Read status of all CO bits on the control box at once

int nRet = hr_api.HRIF_ReadAllBoxCO(0, ref coStatus);

3.4.9. HRIF_ReadAllBoxDO

3.4.9.1. Description: Reads the status of all general-purpose digital outputs (DO) on the control box.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

✓ output variable

output	name	type of data	Effective	content

variable			range	
status	General Output Status Bits	IOStatus	-	Status values for 8 DO bits: status.Bit0 ~ status.Bit7: Status of each general output (0/1).

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.9.2. example

```
// Create DO status structure
IOStatus doStatus = new IOStatus();

// Read status of all DO bits on the control box at once
int nRet = hr_api.HRIF_ReadAllBoxDO(0, ref doStatus);
```

3.4.10.HRIF_ReadBoxAI

3.4.10.1. Description: Read the electrical box analog input value.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	Analog input channel	int	0/1	0: Analog channel 0 1: Analog channel 1

✓ output variable

output variable	name	type of data	Effective range	content
dVal	Corresponding analog channel value	double	0~20	Current mode: 4-20mA Voltage mode: 0-10V

return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.10.2. example

```
// read the result
double dVal = 0;
// Define the bits to be read
int nBit = 0;
// Read the AI value of the nBit channel of the electrical box
int nRet= hr_api.HRIF_ReadBoxAI(0, nBit, ref dVal);
```

3.4.11. HRIF_ReadBoxAO

3.4.11.1. Description: Read the analog output value of the electrical box.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	Analog output channel	int	0/1	0: Analog channel 0 1: Analog channel 1

✓ output variable

output variable	name	type of data	Effective range	content
nMode	Corresponding analog channel mode	int	0/1	0: voltage mode 1: current mode:
dVal	Corresponding analog channel value	double	0~20	Current mode: 4-20mA Voltage mode: 0-10V

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.11.2. example

```
// Define the bits to be read
int nBit = 0;
// current mode to read
int nMode = 0;
// read the result
double dVal = 0;
// Read the AO value of the nBit channel of the electrical box
int nRet= hr_api.HRIF_ReadBoxAO(0, nBit, ref nMode, ref dVal);
```

3.4.12.HRIF_SetBoxCO

3.4.12.1. Description: Set the electrical box control digital output state.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	Control Digital Output Bits	int	0~7	Control Digital Output Bit Index
nVal	Control digital output state	int	0/1	0: low level 1: high level

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.12.2. example

```
// Define the bits that need to be set
int nBit = 5;
// define setting state
int nVal = 1;
// Set the CO state of the nBit bit of the electric box to nVal
int nRet= hr_api.HRIF_SetBoxCO(0, nBit, nVal);
```

3.4.13.HRIF_SetBoxDO

3.4.13.1. Description: Set the general digital output status of the electrical box.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	General purpose digital output bits	int	0~7	General Digital Output Bit Index
nVal	General purpose digital output status	int	0/1	0: low level 1: high level

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.13.2. example

```
// Define the bits that need to be set
int nBit = 5;
// define setting state
int nVal = 1;
// Set the nBit bit DO state of the electric box to nVal
int nRet= hr_api.HRIF_SetBoxDO(0, nBit,nVal);
```

3.4.14.HRIF_SetBoxAOMode

3.4.14.1. Description: Set the electrical box analog output mode.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	Analog channel	int	0/1	Analog channel
nMode	model	int	1/2	1: voltage mode 2: current mode

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.14.2. example

```
// Define the channel that needs to be set
int nBit = 0;
// define setting value
int nMode = 1;
// Set the nBit of the electric box Channel AO mode is nMode
int nRet= hr_api.HRIF_SetBoxAOMode(0, nBit , nMode);
```

3.4.15.HRIF_SetBoxAOVal

3.4.15.1. Description: Set the electrical box analog output value.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nBit	Analog channel	int	0/1	Analog channel
dVal	Corresponding analog value	double	0~20	Current mode: 4-20mA Voltage mode: 0-10V
nMode	model	int	1/2	1: voltage mode 2: current mode

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.15.2. example

```
// Define the channel that needs to be set
int nBit = 0;
// define setting value
double dVal = 5.5;
// set mode
int nMode = 1;
// Set the AO value of the nBit channel of the electrical box to dVal
int nRet= hr_api.HRIF_SetBoxAOVal(0, nBit, dVal, nMode);
```

3.4.16.HRIF_SetEndDO

3.4.16.1. Description: Set terminal digital output state.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nBit	terminal digital output bit	int	0~3	General purpose digital output bit index with only 4 DOs at the end
nVal	Terminal digital output state	int	0/1	0: low level 1: high level

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.16.2. example

```
// Define the bits that need to be set
int nBit = 1;
// define setting value
int nVal = 1;
// Set the end nBit bit DO status
int nRet= hr_api.HRIF_SetEndDO(0,0, nBit, nVal);
```

3.4.17.HRIF_ReadEndDI

3.4.17.1. Description: Read terminal digital input status.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nBit	end digit input	int	0~2	Terminal digital input corresponding bit state

✓ output variable

input variable	name	type of data	Effective range	content
nVal	Terminal digital input corresponding bit state	int	0/1	0: low level 1: high level

✓ return value：

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.17.2. example

```
// read the result
int nVal = 0;
// Define the bits to be read
int nBit = 0;
// Read the end nBit DI status
int nRet= hr_api.HRIF_ReadEndDI(0,0, nBit, ref nVal);
```

3.4.18.HRIF_ReadEndDO

3.4.18.1. Description: Read terminal digital output status.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nBit	terminal digital output bit	int	0~2	Bit index corresponding to terminal digital output
nVal	Terminal digital output corresponding bit state	int	0/1	0: low level 1: high level

✓ output variable

output variable	name	type of data	Effective range	content
nVal	Terminal digital output corresponding bit state	int	0/1	0: low level 1: high level

✓ return value：

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.18.2. example

```
// read the result
int nVal = 0;
// Define the bits to be read
int nBit = 0;
// Read the end nBit bit DO status
int nRet= hr_api.HRIF_ReadEndDO(0,0, nBit, ref nVal);
```

3.4.19.HRIF_ReadEndAI

3.4.19.1. Description: Read terminal analog input value.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nBit	Analog input channel	int	0/1	0: Analog channel 0 1: Analog channel 1

✓ output variable

output variable	name	type of data	Effective range	content
dVal	Corresponding analog channel value	double	0~20	Current mode: 4~20mA Voltage mode: 0-10V

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.19.2. example

```
// read the result
double dVal = 0;
// Define the bits to be read
int nBit = 0;
// Read the end nBit channel AI value
int nRet= hr_api.HRIF_ReadEndAI(0,0, nBit, ref dVal);
```

3.4.20.HRIF_ReadEndBTN

3.4.20.1. Description: Read the state of the terminal button. According to the type of terminal equipped, the meaning of each state will be different. Note: Some versions of the firmware/hardware do not support reading the end-effector button status. Calling this interface under such versions will return error code 39502 — "Parameter Error in Command Response".

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nBit1	Button 1 status	int	0/1	0: release 1: Press
nBit2	Button 2 state	int	0/1	0: release 1: Press
nBit3	Button 3 state	int	0/1	0: release 1: Press
nBit4	Button 4 state	int	0/1	0: release 1: Press

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.20.2. example

// Define the bits to be read

int nBit1 = 0; int nBit2 = 0;

int nBit3 = 0; int nBit4 = 0;

// Read the end nBit channel AI value

int nRet= hr_api.HRIF_ReadEndBTN(0,0, ref nBit1, ref nBit2, ref nBit3, ref nBit4);

3.4.21.HRIF_EnableEndBTN

3.4.21.1. Description: Enables or disables the end-effector I/O functionality.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nRbtID	Robot ID	int	0 ~ 5	Robot ID number.
nEnabled	End-Effector I/O Enable Status	int	0 / 1	0: Disable end-effector I/O functionality 1: Enable end-effector I/O functionality

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.21.2. example

```
// Enable end-effector I/O functionality
int nEnabled= 1;
// Call the interface
int nRet= hr_api.HRIF_EnableEndBTN(0,0,nEnabled);
```

3.4.22.HRIF_cdsSetIO

3.4.22.1. Description: Sets I/O states before the motion command reaches the target position.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nRbtID	Robot ID	int	0 ~ 5	Robot ID number.
nEndDOMask	End-DO Modification Bitmask	int	-	Bits 0-7 correspond to EndDO0~EndDO7. If a bit is 1, the corresponding EndDO value will be changed; otherwise, it remains unchanged.
nEndDOVal	Target End-DO States	int	-	Bits 0-7 correspond to EndDO0~EndDO7. 0: Low level 1: High level
nBoxDOMask	Box-DO	int	-	Bits 0-7 correspond to BoxDO0~BoxDO7. If a

	Modification Bitmask			bit is 1, the corresponding BoxDO value will be changed.
nBoxDOVal	Target Box-DO States	int	-	Bits 0-7 correspond to BoxDO0~BoxDO7. 0: Low level 1: High level
nBoxCOMask	Box-CO Modification Bitmask	int	-	Bits 0-7 correspond to BoxCO0~BoxCO7. If a bit is 1, the corresponding BoxCO value will be changed.
nBoxCOVal	Target Box-CO States	int	-	Bits 0-7 correspond to BoxCO0~BoxCO7. 0: Low level 1: High level
nBoxAOCH0_Mask	Box-AO Channel 0 Change Flag	int	0 / 1	0: No change required 1: Change required
nBoxAOCH0_Mode	Mode for Channel 0	int	1 / 2	1: Voltage mode 2: Current mode
nBoxAOCH1_Mask	Box-AO Channel 1 Change Flag	int	0 / 1	0: No change required 1: Change required
nBoxAOCH1_Mode	Mode for Channel 1	int	1 / 2	1: Voltage mode 2: Current mode
dbBoxAOCH0_Val	Analog Value for Channel 0	double	0 ~ 20	Current mode: 4-20 mA Voltage mode: 0-10 V
dbBoxAOCH1_Val	Analog Value for Channel 1	double	0 ~ 20	Current mode: 4-20 mA Voltage mode: 0-10 V

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.22.2. example

```
// Set EndDO1 to low level before reaching a waypoint
int nEndDOMask=1;
int nEndDOVal=0;
int nBoxDOMask=0;
int nBoxDOVal=0;
int nBoxCOMask=0;
int nBoxCOVal=0;
int nBoxAOCH0_Mask=0;
int nBoxAOCH0_Mode=0;
int nBoxAOCH1_Mask=0;
int nBoxAOCH1_Mode=0;
```

int dbBoxAOCH0_Val=0;

int dbBoxAOCH1_Val=0;

int nRet=hr_api.HRIF_cdsSetIO(0,0,nEndDOMask,nEndDOVal,nBoxDOMask,nBoxDOVal,nBoxCOMask, nBoxCOVal,nBoxAOCH0_Mask,nBoxAOCH0_Mode,nBoxAOCH1_Mask,nBoxAOCH1_Mode,dbBoxAOCH0_Val, dbBoxAOCH1_Val);

3.4.23.HRIF_SetMoveParamsAO

3.4.23.1. Description: Sets the analog output voltage for welding applications.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
nRbtID	Robot ID	int	0 ~ 5	Robot ID number.
nState	Channel Switch State	int	0 / 1	0: Turn off 1: Turn on
nIndex	Analog Voltage Channel	int	0 / 1	0: Channel AO0 1: Channel AO1
dInitAO	Initialization Voltage	double	0.0 ~ 10.0	Initialization voltage (V).
dWeldingAO	Welding Voltage	double	0.0 ~ 10.0	Welding voltage (V).

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.4.23.2. example

```
// Define channel switch state
int nState = 0;
// Define analog voltage channel
int nIndex = 0;
// Define initialization voltage
int dInitAO = 0;
// Define actual welding voltage
int dWeldingAO = 0;
```

// Set welding analog output voltage

```
int nRet = hr_api.HRIF_SetMoveParamsAO(0,0, nState, nIndex,dInitAO,dWeldingAO);
```

3.5. Status read and set instructions

3.5.1. HRIF_SetOverride

3.5.1.1. Description: Set the speed ratio.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dOverride	speed	double	0.01~1	Speed ratio to be set (0,0.01~1)

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.1.2. example

```
// The speed ratio that needs to be set
double dOverride = 0.5;
// set the current speed ratio
int nRet = hr_api.HRIF_SetOverride(0,0, dOverride);
```

3.5.2. HRIF_SetToolMotion

3.5.2.1. Description: Turn Tool on or off Coordinate system movement mode.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nState	state	int	0/1	0: off 1: open

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.2.2. example

```
// Tool motion state that needs to be set
int nState = 1;
// Set Tool motion state
int nRet= hr_api.HRIF_SetToolMotion(0,0, nState);
```

3.5.3. HRIF_SetPayload

3.5.3.1. Description: Set the current load parameters.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dMass	quality	double	0~allowable maximum load	Mass, ranging from 0 to the current maximum allowable load, unit [kg]
wxya	Center of mass X-direction offset	double	>0	Center of mass offset in X direction, unit [mm]
wxya	Center of mass Y-direction offset	double	>0	Center of mass offset in Y direction, unit [mm]
dZ	Center of mass Z direction offset	double	>0	Center of mass offset in Z direction, unit [mm]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.3.2. example

```
// need to set payload quality
double dMass = 3;
// need to set payload quality
double dX= 10;
// need to set payload quality
double dY= 20;
// need to set payload quality
double dZ= 30;
// set the current load
int nRet= hr_api.HRIF_SetPayload(0,0, dMass, dX, dY, dZ);
```

3.5.4. HRIF_SetJointMaxVel_nJ

3.5.4.1. Description: Sets the maximum motion velocity for joints. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Joints	Per-axis Maximum Velocity	JointsData	0.~ 200	Maximum velocity for axis i, unit: [°/s].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

* Note: Please refer to " HuayanRobot_ErrorCode.docx " for the error code .

3.5.4.2. example

```
// Define the maximum joint velocities to be set
JointsData joints = new JointsData();
for(int i = 0;i<10;i++)
{
    joints.joint[i] = 10;
}
// Set maximum joint velocities
int nRet=hr_api.HRIF_SetJointMaxVel_nJ(0,0,joints);
```

3.5.5. HRIF_SetJointMaxVel

3.5.5.1. Description: Set the maximum movement speed of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dJ1	J1 axis maximum speed	double	0~200	J1 axis maximum speed, unit [°/s]
dJ2	J2 axis maximum speed	double	0~200	J2 axis maximum speed, unit [°/s]
dJ3	J3 axis maximum speed	double	0~200	J3 axis maximum speed, unit [°/s]
dJ4	J4 axis maximum speed	double	0~200	J4 axis maximum speed, unit [°/s]
dJ5	J5 axis maximum speed	double	0~200	Maximum speed of J5 axis, unit [°/s]
dJ6	J6 axis maximum speed	double	0~200	J6 axis maximum speed, unit [°/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

* Note: Please refer to " HuayanRobot_ErrorCode.docx " for the error code .

3.5.5.2. example

```
// Define the maximum speed of the joint that needs to be set
double dJ1 = 180; double dJ2 = 180; double dJ3 = 180;
double dJ4 = 180; double dJ5 = 200; double dJ6 = 200;
// set the maximum joint speed
int nRet= hr_api.HRIF_SetJointMaxVel(0,0, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6);
```

3.5.6. HRIF_SetJointMaxAcc

3.5.6.1. Description: Set the maximum motion acceleration of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dJ1	J1 axis maximum acceleration	double	0~720	J1 axis maximum acceleration, unit [°/s ^{2}]
dJ2	J2 axis maximum acceleration	double	0~720	J2 axis maximum acceleration, unit [°/s ^{2}]
dJ3	J3 axis maximum acceleration	double	0~720	J3 axis maximum acceleration, unit [°/s ^{2}]
dJ4	J4 axis maximum acceleration	double	0~720	J4 axis maximum acceleration, unit [°/s ^{2}]
dJ5	J5 axis maximum acceleration	double	0~720	J5 axis maximum acceleration, unit [°/s ^{2}]
dJ6	J6 axis maximum acceleration	double	0~720	J6 axis maximum acceleration, unit [°/s ^{2}]

* Note: The effective range of joint acceleration needs to refer to the specific model.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.6.2. example

```
// Define the maximum acceleration of the joint that needs to be set
double dJ1 = 360; double dJ2 = 360; double dJ3 = 360;
double dJ4 = 360; double dJ5 = 360; double dJ6 = 360;
// Set the maximum joint acceleration
int nRet= hr_api.HRIF_SetJointMaxAcc(0,0, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6);
```

3.5.7. HRIF_SetJointMaxAcc_nJ

3.5.7.1. Description: Sets the maximum motion acceleration for joints. Note: This interface is specifically for the extended-axis version .

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Joints	Per-axis Maximum Acceleration	JointsData	0 ~ 200	Maximum acceleration for axis i, unit: [°/s].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

* Note: Please refer to " HuayanRobot_ErrorCode.docx " for the error code .

3.5.7.2. example

```
// Define the maximum joint accelerations to be set
JointsData joints = new();
for (int i = 0; i < 10; i++)
{
    joints.joint[i] = 100;
}
// Set maximum joint accelerations
int nRet= hr_api.HRIF_SetJointMaxAcc_nJ(0,0, joints);
```

3.5.8. HRIF_SetLinearMaxVel

3.5.8.1. Description: Set the maximum speed of linear motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dMaxVel	Maximum Linear Speed	double	0~3000	Maximum linear speed, default [2000], unit [mm/s]

* Note: The effective range of linear speed needs to refer to the specific model.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.8.2. example

```
// Define the need to set the maximum speed of the straight line
double dMaxVel = 100;
// set the maximum linear velocity
int nRet= hr_api.HRIF_SetLinearMaxVel(0,0, dMaxVel);
```

3.5.9. HRIF_SetLinearMaxAcc

3.5.9.1. Description: Set the maximum acceleration of linear motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dMaxAcc	Maximum Linear Acceleration	double	0~2500	Maximum linear acceleration, default [2500], unit [mm/s ^{2}]

* Note: The effective range of linear acceleration needs to refer to the specific model.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.9.2. example

```
// Define the maximum acceleration of the straight line that needs to be set
double dMaxAcc = 2500;
// Set the maximum linear acceleration
int nRet= hr_api.HRIF_SetLinearMaxAcc(0,0, dMaxAcc);
```

3.5.10.HRIF_SetMaxAcsRange

3.5.10.1. Description: Set the maximum motion range of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dMaxJ1-dMaxJ6	Joint maximum range	double	-	dMaxJ1: maximum motion range of joint 1, unit [°] dMaxJ2: maximum motion range of joint 2, unit [°] dMaxJ3: maximum motion range of joint 3, unit [°] dMaxJ4: maximum motion range of joint 4, unit [°] dMaxJ5: maximum motion range of joint 5, unit [°] dMaxJ6: maximum motion range of joint 6, unit [°]
dMinJ1-dMinJ6	Joint minimum range	double	-	dMinJ1: minimum motion range of joint 1, unit [°] dMinJ2: minimum motion range of joint 2, unit [°] dMinJ3: the minimum motion range of joint 3, unit [°] dMinJ4: the minimum motion range of joint 4, unit [°] dMinJ5: the minimum motion range of joint 5, unit [°] dMinJ6: the minimum motion range of joint 6, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.10.2. example

```
// Define the maximum motion range of the joint that needs to be set
double dMaxJ1 = 360; double dMaxJ2 = 135 ; double dMaxJ3 = 153 ;
double dMaxJ4 = 360; double dMaxJ5 = 180 ; double dMaxJ6 = 360;
// Define the minimum motion range of the joint that needs to be set
double dMinJ1 = -360; double dMinJ2 = -135 ; double dMinJ3 = -153 ;
double dMinJ4 = -360; double dMinJ5 = -180 ; double dMinJ6 = -360;
// Set joint motion range
int nRet= hr_api.HRIF_SetMaxAcsRange (0,0, dMaxJ1, dMaxJ2, dMaxJ3, dMaxJ4, dMaxJ5, dMaxJ6,
dMinJ1, dMinJ2, dMinJ3, dMinJ4, dMinJ5, dMinJ6);
```

3.5.11. HRIF_SetMaxAcsRange_nJ

3.5.11.1. Description: Sets the maximum motion range for joints. Note: This interface is specifically for the extended-axis version but is NOT supported in Series 7 versions.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
jointsMaxAcs	Joint Maximum Range	JointsData	-	jointsMaxAcs.joint[i]: Maximum motion range for joint i, unit: [°].
jointsMinAcs	Joint Minimum Range	JointsData	-	jointsMinAcs.joint[i]: Minimum motion range for joint i, unit: [°].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.11.2. example

```
// Define the joint motion ranges to be set (example for a 7-axis robot)
JointsData jointsMaxAcs = new JointsData();
JointsData jointsMinAcs = new JointsData();
jointsMaxAcs.joint[0] = 360;jointsMinAcs.joint[0] = -360;jointsMaxAcs.joint[1] = 135;jointsMinAcs.joint[1] = -135;
jointsMaxAcs.joint[2] = 153;jointsMinAcs.joint[2] = -153;jointsMaxAcs.joint[3] = 360;jointsMinAcs.joint[3] = -360;
jointsMaxAcs.joint[4] = 180;jointsMinAcs.joint[4] = -180;jointsMaxAcs.joint[5] = 360;jointsMinAcs.joint[5] = -360;
```

```
jointsMaxAcs.joint[6] = 720;jointsMinAcs.joint[6] = -720;
// Set joint motion ranges
int nRet = hr_api.HRIF_SetMaxAcsRange_nJ(0, 0,jointsMaxAcs,jointsMinAcs);
```

3.5.12.HRIF_SetMaxPcsRange

3.5.12.1. Description: Set the maximum motion range of the space.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dMaxX-dMaxRz	X-Rz maximum range	double	-	XYZ maximum range, unit [mm] RxRyRz maximum range, unit [°], not enabled, default is 0
dMinX-dMinRz	X-Rz minimum range	double	-	XYZ minimum range, unit [mm] RxRyRz minimum range, unit [°], not enabled, default is 0
dUcs_X-dUcs_Rz	based on user coordinates	double	-	dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.12.2. example

```
// Define the maximum motion range of the space that needs to be set
double dMaxX = 1200; double dMaxY = 1200; double dMaxZ = 1200;
double dMaxRx = 180; double dMaxRy = 180; double dMaxRz = 180;
// Define the minimum motion range of the space that needs to be set
double dMinX = -1200; double dMinY = -1200; double dMinZ = -1200;
double dMinRx = -180; double dMinRy = -180; double dMinRz = -180;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
```

// Set joint motion range

int nRet= hr_api.HRIF_SetMaxPcsRange(0,0, dMaxX, dMaxY, dMaxZ, dMaxRx, dMaxRy, dMaxRz, dMinX, dMinY, dMinZ, dMinRx, dMinRy, dMinRz, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_R z);

3.5.13.HRIF_ReadMaxRange

3.5.13.1. Description: Reads the workspace limits of the robotic arm.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default = 0
rbtID	Robot ID	unsigned int	0~5	Robot ID number, default = 0

✓ output variable

output variable	name	type of data	Effective range	content
Positive_J1 ~ Positive_J6	Joint Maximum Range	double	-	Maximum motion range for joints 1-6, unit: [°]. Positive_J1: Joint 1 max range Positive_J2: Joint 2 max range Positive_J3: Joint 3 max range Positive_J4: Joint 4 max range Positive_J5: Joint 5 max range Positive_J6: Joint 6 max range
Negative_J1 ~ Negative_J6	Joint Minimum Range	double	-	Minimum motion range for joints 1-6, unit: [°]. Negative_J1: Joint 1 min range Negative_J2: Joint 2 min range Negative_J3: Joint 3 min range Negative_J4: Joint 4 min range Negative_J5: Joint 5 min range Negative_J6: Joint 6 min range
Positive_X~Positive_Rz	Cartesian Maximum Range	double	-	XYZ max limits [mm] RxRyRz orientation max limits [°]
Negative_X~ Negative_Rz	Cartesian Minimum Range	double	-	XYZ min limits [mm] RxRyRz orientation min limits [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.13.2. example

double Positive_J1 = 0, Positive_J2 = 0, Positive_J3 = 0, Positive_J4 = 0, Positive_J5 = 0, Positive_J6 = 0, Negative_J1 = 0, Negative_J2 = 0, Negative_J3 = 0, Negative_J4 = 0, Negative_J5 = 0, Negative_J6 = 0,

Positive_X = 0, Positive_Y = 0, Positive_Z = 0, Positive_Rx = 0, Positive_Ry = 0, Positive_Rz = 0,
Negative_X = 0, Negative_Y = 0, Negative_Z = 0, Negative_Rx = 0, Negative_Ry = 0, Negative_Rz = 0;

// // Read robotic arm workspace limits

int nRet = hr_api.HRIF_ReadMaxRange(0, 0, ref Positive_J1, ref Positive_J2, ref Positive_J3, ref Positive_J4, ref Positive_J5, ref Positive_J6, ref Negative_J1, ref Negative_J2, ref Negative_J3, ref Negative_J4, ref Negative_J5, ref Negative_J6, ref Positive_X, ref Positive_Y, ref Positive_Z, ref Positive_Rx, ref Positive_Ry, ref Positive_Rz, ref Negative_X, ref Negative_Y, ref Negative_Z, ref Negative_Rx, ref Negative_Ry, ref Negative_Rz);

3.5.14.HRIF_ReadMaxRange_nJ

3.5.14.1. Description: Reads the workspace limits of the robotic arm. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default = 0
rbtID	Robot ID	unsigned int	0~5	Robot ID number, default = 0

✓ output variable

output variable	name	type of data	Effective range	content
positiveJoints	Joint Maximum Range	JointsData	-	positiveJoints.joint[i]: Maximum motion range for joint i, unit: [°].
negativeJoints	Joint Minimum Range	JointsData	-	negativeJoints.joint[i]: Minimum motion range for joint i, unit: [°].
Positive_X, Positive_Y, Positive_Z, Positive_Rx, Positive_Ry, Positive_Rz	Cartesian Maximum Range	double	-	XYZ max limits [mm]; RxRyRz max limits [°].
Negative_X, Negative_Y, Negative_Z, Negative_Rx, Negative_Ry, Negative_Rz	Cartesian Minimum Range	double	-	XYZ min limits [mm]; RxRyRz min limits [°].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.14.2. example

```
JointsData positiveJpints = new JointsData();
positiveJpints.joint[0] = 0; positiveJpints.joint[1] = 0; positiveJpints.joint[2] = 0; positiveJpints.joint[3] = 0; positiveJpints.joint[4] = 0;
positiveJpints.joint[5] = 0; positiveJpints.joint[6] = 0; positiveJpints.joint[7] = 0; positiveJpints.joint[8] = 0; positiveJpints.joint[9] = 0;
JointsData negativeJpints = new JointsData();
negativeJpints.joint[0] = 0; negativeJpints.joint[1] = 0; negativeJpints.joint[2] = 0; negativeJpints.joint[3] = 0; negativeJpints.joint[4] = 0;
```

```
negativeJpints.joint[5] = 0; negativeJpints.joint[6] = 0; negativeJpints.joint[7] = 0; negativeJpints.joint[8] = 0; negativeJpints.joint[9] = 0;
double Positive_X = 0, Positive_Y = 0, Positive_Z = 0, Positive_Rx = 0, Positive_Ry = 0, Positive_Rz = 0, Negative_X = 0, Negative_Y = 0, Negative_Z = 0, Negative_Rx = 0, Negative_Ry = 0, Negative_Rz = 0;
// // Read robotic arm workspace limits
int nRet = hr_api.HRIF_ReadMaxRange_nJ(0, 0, ref positiveJpints, ref negativeJpints, ref Positive_X, ref Positive_Y, ref Positive_Z, ref Positive_Rx, ref Positive_Ry, ref Positive_Rz, ref Negative_X, ref Negative_Y, ref Negative_Z, ref Negative_Rx, ref Negative_Ry, ref Negative_Rz);
```

3.5.15.HRIF_ReadOverride

3.5.15.1. Description: Read the current speed ratio.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default = 0
rbtID	Robot ID	unsigned int	0~5	Robot ID number, default = 0

✓ output variable

output variable	name	type of data	Effective range	content
dOverride	speed	double	0~ 1	Save the obtained current speed ratio

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.15.2. example

```
// define speed ratio variable
double dOverride = 0;
// read the current speed ratio
int nRet = HRIF_ ReadOverride (0, 0, ref dOverride);
```

3.5.16.HRIF_ReadJointMaxVel

3.5.16.1. Description: Read the maximum movement speed of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1	J1 axis maximum speed	double	0~200	J1 axis maximum speed, unit [°/s]
dJ2	J2 axis maximum speed	double	0~200	J2 axis maximum speed, unit [°/s]
dJ3	J3 axis maximum speed	double	0~200	J3 axis maximum speed, unit [°/s]
dJ4	J4 axis maximum speed	double	0~200	J4 axis maximum speed, unit [°/s]
dJ5	J5 axis maximum speed	double	0~200	Maximum speed of J5 axis, unit [°/s]
dJ6	J6 axis maximum speed	double	0~200	J6 axis maximum speed, unit [°/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.16.2. example

```
// Define the maximum movement speed variable of the joint
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// read the maximum joint speed
int nRet= hr_api.HRIF_ReadJointMaxVel(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);
```

3.5.17.HRIF_ReadJointMaxVel_nJ

3.5.17.1. Description: Reads the maximum motion velocity for joints. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
joints	Per-axis Maximum Velocity	JointsData	0~ 200	joints.joint[i]: Maximum velocity for axis i, unit: [°/s].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.17.2. example

```
// Define variable for joint maximum velocities
JointsData joints = new JointsData();
// Read maximum joint velocities
int nRet = hr_api.HRIF_ReadJointMaxVel_nJ(0, 0, ref joints);
```

3.5.18.HRIF_ReadJointMaxAcc

3.5.18.1. Description: Read the maximum motion acceleration of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1	J1 axis maximum acceleration	double	0~720	J1 axis maximum acceleration, unit [°/s ^{2}]
dJ2	J2 axis maximum acceleration	double	0~720	J2 axis maximum acceleration, unit [°/s ^{2}]
dJ3	J3 axis maximum acceleration	double	0~720	J3 axis maximum acceleration, unit [°/s ^{2}]
dJ4	J4 axis maximum acceleration	double	0~720	J4 axis maximum acceleration, unit [°/s ^{2}]
dJ5	J5 axis maximum acceleration	double	0~720	J5 axis maximum acceleration, unit [°/s ^{2}]
dJ6	J6 axis maximum acceleration	double	0~720	J6 axis maximum acceleration, unit [°/s ^{2}]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.18.2. example

```
// Define the maximum motion acceleration variable of the joint
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// read the maximum joint acceleration
int nRet= hr_api.HRIF_ReadJointMaxAcc(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);
```

3.5.19.HRIF_ReadJointMaxAcc_nJ

3.5.19.1. Description: Reads the maximum motion acceleration for joints. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
joints	Per-axis Maximum Acceleration	JointsData	0~ 720	joints.joint[i]: Maximum acceleration for axis i, unit: [°/s^2].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.19.2. example

```
// Define variable for joint maximum accelerations
JointsData joints = new JointsData();
// Read maximum joint accelerations
int nRet = hr_api.HRIF_ReadJointMaxAcc_nJ(0, 0, ref joints);
```

3.5.20.HRIF_ReadJointMaxJerk

3.5.20.1. Description: Read the maximum motion jerk of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1	J1 axis maximum jerk	double	0~3600	J1 axis maximum jerk, unit [°/s ^{3}]
dJ2	J2 axis maximum jerk	double	0~3600	J2 axis maximum jerk, unit [°/s ^{3}]
dJ3	J3 axis maximum jerk	double	0~3600	J3 axis maximum jerk, unit [°/s ^{3}]
dJ4	J4 axis maximum jerk	double	0~3600	J4 axis maximum jerk, unit [°/s ^{3}]
dJ5	J5 axis maximum jerk	double	0~3600	J5 axis maximum jerk, unit [°/s ^{3}]
dJ6	J6 axis maximum jerk	double	0~3600	J6 axis maximum jerk, unit [°/s ^{3}]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.20.2. example

// Define the joint maximum motion jerk variable

double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;

double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;

// read the maximum joint jerk

int nRet= hr_api.HRIF_ReadJointMaxJerk(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);

3.5.21.HRIF_ReadJointMaxJerk_nJ

3.5.21.1. Description: Reads the maximum motion jerk for joints. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
joints	Per-axis Maximum Jerk	JointsData	0.0 ~ 3600.0	joints.joint[i]: Maximum jerk for axis i, unit: [°/s³].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.21.2. example

```
// Define variable for joint maximum jerk
JointsData joints = new JointsData();
// Read maximum joint jerk
int nRet = hr_api.HRIF_ReadJointMaxJerk_nJ(0, 0, ref joints);
```

3.5.22.HRIF_ReadLinearMaxSpeed

3.5.22.1. Description: Read the maximum speed parameter of linear motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dMaxVel	Maximum Linear Speed	double	0~3000	Maximum linear speed, default [20 00], unit [mm/s]
dMaxAcc	Maximum Linear Acceleration	double	0~2500	Maximum linear acceleration, default [2500], unit [mm/s^2]
dMaxJerk	Maximum linear jerk	double	0~10000000	Maximum linear jerk, default [10000000], unit [mm/s^3]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.22.2. example

```
// Define the maximum movement speed variable of the straight line
double dMaxVel = 0; double dMaxAcc = 0; double dMaxJerk = 0;
// read the maximum linear speed
int nRet= hr_api.HRIF_ReadLinearMaxSpeed (0,0, ref dMaxVel , ref dMaxAcc, ref dMaxJerk);
```

3.5.23.HRIF_ReadEmergencyInfo

3.5.23.1. Description: Read emergency stop information.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nESTO_Error	emergency stop error	int	0/1	There are two emergency stop circuits. When the signals of the two circuits are different, it is considered that there is an error in the emergency stop circuit, and the value is 1.
NESTO	emergency stop signal	int	0/1	Emergency stop signal, when an emergency stop occurs, the 48V output power supply to the body will be cut off, but the 220V to 48V power supply will not be cut off
nSafetyGuard_Error	Safety light curtain error	int	0/1	There are two circuits in the safety light curtain circuit. When the two signals are not the same, it is considered that there is an error in the circuit of the safety light curtain, and the value is 1.
nSafetyGuard	safety light curtain signal	int	0/1	Safety light curtain signal, when a safety light curtain occurs, the robot will stop moving, and will no longer accept motion commands, and will not cut off the main body power supply

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.23.2. example

// Define the maximum movement speed variable of the straight line

int nESTO_Error = 0; int nESTO = 0; int nSafetyGuard_Error = 0; int nSafetyGuard = 0;

// read the maximum linear speed

int nRet = hr_api.HRIF_ReadEmergencyInfo (0,0, ref nESTO_Error, ref nESTO, ref nSafetyGuard_Error, ref nSafetyGuard);

3.5.24.HRIF_ReadRobotState

3.5.24.1. Description: Read the current robot status flags.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nMovingState	exercise state	int	0/1	0: The robot is not in motion 1: The robot is moving
nEnableState	enable state	int	0/1	0: The robot is not enabled 1: The robot is enabled
nErrorState	error state	int	0/1	0: No error occurred 1: An error occurred
nErrorCode	error code	int	>= 0	error code
nErrorAxis	wrong axis id	int	0~5	wrong axis id
Breaking	brake status	int	0/1	Brake state (after the brake is released, the shaft may fall due to the action of gravity) 0: Brake 1: Release the brake
nPause	suspended state	int	0/1	0: not in paused state 1: Already paused
nEmergencyStop	emergency stop	int	0/1	0: not in emergency stop state 1: already in emergency stop state
nSaftyGuard	Safety Light Curtain Status	int	0/1	0: not in the safety light curtain state 1: Already in the safety light curtain state
nElectrify	power-on state	int	0/1	0: Not powered on 1: Powered on
nIsConnectToBox	Electric box connection status	int	0/1	0: Electric box is not connected 1 : Electric box is connected
nBlending Done	Waypoint Motion Status	int	0/1	0: the waypoint is in motion 1: Waypoint movement completed
nInPos	In place	int	0/1	0: The actual position of the robot has not moved to the command position 1: Move in place (there is basically no error between the command and the actual position)

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.24.2. example

```
// Define the robot state variables that need to be read
int nMovingState = 0; int nEnableState = 0; int nErrorState = 0; int nErrorCode = 0;
int nErrorAxis = 0; int nBreaking = 0; int nPause = 0; int nEmergencyStop = 0;
int nSaftyGuard = 0; int nElectrify = 0; int nIsConnectToBox = 0; int nBlendingDone = 0; int nInPos = 0;
// read status
int nRet= hr_api.HRIF_ReadRobotState(0,0, ref nMovingState, ref nEnableState, ref nErrorState, ref nErrorCode,
ref nErrorAxis, ref nBreaking, ref nPause, ref nEmergencyStop, ref nSaftyGuard, ref nElectrify,
ref nIsConnectToBox, ref nBlendingDone, ref nInPos);
```

3.5.25.HRIF_ReadRobotFlags

3.5.25.1. Description: Read the current robot status flags.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nMovingState	exercise state	int	0/1	0: The robot is not in motion 1: The robot is moving
nEnableState	enable state	int	0/1	0: The robot is not enabled 1: The robot is enabled
nErrorState	error state	int	0/1	0: No error occurred 1: An error occurred
nErrorCode	error code	int	>= 0	error code
nErrorAxis	wrong axis id	int	0~5	wrong axis id
Breaking	brake status	int	0/1	Brake state (after the brake is released, the shaft may fall due to the action of gravity) 0: Brake 1: Release the brake
nPause	suspended state	int	0/1	0: not in paused state 1: Already paused
nBlending Done	Waypoint Motion Status	int	0/1	0: the waypoint is in motion 1: Waypoint movement completed

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.25.2. example

```
// Define the robot state variables that need to be read
int nMovingState = 0; int nEnableState = 0; int nErrorState = 0; int nErrorCode = 0;
int nErrorAxis = 0; int nBreaking = 0; int nPause = 0; int nBlendingDone = 0;
// read status
int nRet= hr_api.HRIF_ ReadRobotFlags (0,0, ref nMovingState, ref nEnableState, ref nErrorState, ref nErrorCode,
ref nErrorAxis, ref nBreaking, ref nPause, ref nBlendingDone);
```

3.5.26.HRIF_ReadCurWaypoint ID

3.5.26.1. Description: Read the current motion ID number of WayPoint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
strCurWaypoint ID	Current ID	string	-	The current motion ID of the waypoint, and WayPoint, MoveJ, The waypoint IDs set in MoveL and MoveC are the same

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.26.2. example

```
// Define the waypoint movement ID variable
string strCurWaypointID = " ";
// read waypoint ID
int nRet= hr_api.HRIF_ReadCurWaypointID(0,0, ref strCurWaypointID);
```

3.5.27.HRIF_ReadPointList

3.5.27.1. Description: Reads the list of saved point names from the system.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective	content

			range	
PointList	Point Name List	List<string>	-	Contains all retrieved point names.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.27.2. example

```
// Define variable to store the list of point names
List<string> PointList=new List<string>();
// Read the point name list
int nRet = hr_api.HRIF_ReadPointList(0,0, ref PointList);
```

3.5.28.HRIF_ReadAxisErrorCode

3.5.28.1. Description: Read the error code.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nErrorCode	current error code	int	>= 0	Current error code, when there is no error code = 0
nJ1	J1 axis error code	int	>= 0	J1 axis error code, when there is no error code = 0
nJ2	J2 axis error code	int	>= 0	J2 axis error code, when there is no error code = 0
nJ3	J3 axis error code	int	>= 0	J3 axis error code, when there is no error code = 0
wxya	J4 axis error code	int	>= 0	J4 axis error code, when there is no error code = 0
nJ5	J5 axis error code	int	>= 0	J5 axis error code, when there is no error code = 0
nJ6	J6 axis error code	int	>= 0	J6 axis error code, when there is no error code = 0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.28.2. example

```
// define error code variable
int nErrorCode = 0;
// Define the error code variables of each axis
int nJ1= 0; int nJ2= 0; int nJ3= 0; int nJ4= 0; int nJ5= 0; int nJ6= 0;
// read error code
i nt nRet= hr_api.HRIF_ReadAxisErrorCode(0,0,ref nErrorCode, ref nJ1, ref nJ2, ref nJ3, ref nJ4, ref nJ5, ref nJ6);
```

3.5.29.HRIF_ReadAxisErrorCode_nJ

3.5.29.1. Description: Reads the error codes for the system and individual axes. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nErrorCode	Current System Error Code	int	Integer >= 0	Current overall system error code. Returns 0 if no error is present.
joints	Per-axis Error Codes	JointsData	Integer >= 0	joints.joint[i]: Error code for axis i. Returns 0 if no error is present for that axis.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.29.2. example

```
// // Define variables for error codes
int nErrorCode = 0;
// Per-axis error codes
JointsData jointsData = new JointsData();
// Read error codes
int nRet = hr_api.HRIF_ReadAxisErrorCode_nJ(0, 0, ref nErrorCode, ref jointsData);
```

3.5.30.HRIF_ReadCurFSM

3.5.30.1. Description: Read the current state machine.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nCurFSM	current state machine	int	>= 0	State machine, you can refer to the state machine list
strCurFSM	Current state machine description	string	-	Current state machine description

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.30.2. example

```
// define state machine variables
int nCurFSM = 0;
// Define state machine field variables
string strCurFSM = " ";
// read the current state machine
int nRet= hr_api.HRIF_ReadCurFSM(0,0, ref nCurFSM, ref strCurFSM);
```

3.5.31.HRIF_ReadCurFSMFromCPS

3.5.31.1. Description: Read the current state machine.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nCurFSM	current state machine	int	>= 0	State machine, you can refer to the state machine list

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.31.2. example

```
// define state machine variables
int nCurFSM = 0;
// read the current state machine
int nRet= hr_api.HRIF_ReadCurFSMFromCPS(0,0, ref nCurFSM);
```

3.5.32. HRIF_ReadPointByName

3.5.32.1. Description: Read the point information according to the point name.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
pointName	point name	string	-	point name

✓ output variable

output variable	name	type of data	Effective range	content
dJ1-dJ6	joint coordinates	double	-	dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°] dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6: joint 6 coordinates, unit [°]
dX -dRz	Cartesian coordinates	double	-	dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]
dTcp_X-dTcp_Rz	current tool coordinates	double	-	dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
dUcs_X-dUcs_Rz	current user coordinates	double	-	dUcs_X: X coordinate, unit [mm] dUcs_Y : Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°]

				dUcs_Rz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.5.32.2. example

```
//Define the coordinate name
string pointName = "Point_1";
// Define the joint position variable
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// define space position variable
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Read the point information according to the point name
int nRet = hr_api.HRIF_ReadPointByName(0, 0, pointName, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6, ref dX, ref dY, ref dZ, ref dRx, ref dRy, ref dRz, ref dTcp_X, ref dTcp_Y, ref dTcp_Z, ref dTcp_Rx, ref dTcp_Ry, ref dTcp_Rz, ref dUcs_X, ref dUcs_Y, ref dUcs_Z, ref dUcs_Rx, ref dUcs_Ry, ref dUcs_Rz);
```

3.5.33.HRIF_ReadPointByName_nJ

3.5.33.1. Description： Reads position information based on point name. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
pointName	Point Name	string		Point name to read

✓ output variable

Output Variables	Name	Data Type	Valid Range	Description
joints	Joint Coordinates	JointsData	-	joints.joint[i]: Coordinate of joint i, unit: [°].
point	Cartesian Coordinates	Pose	-	point.dX: X coordinate [mm] point.dY: Y coordinate [mm] point.dZ: Z coordinate [mm] point.dRX: Rx orientation [°] point.dRY: Ry orientation [°] point.dRZ: Rz orientation [°]
TCP	Tool Frame	Pose	-	TCP.X: X offset [mm] TCP.Y: Y offset [mm] TCP.Z: Z offset [mm] TCP.RX: Rx orientation [°] TCP.RY: Ry orientation [°] TCP.RZ: Rz orientation [°]
UCS	User Coordinate System	Pose	-	UCS.X: X offset [mm] UCS.Y: Y offset [mm] UCS.Z: Z offset [mm] UCS.RX: Rx orientation [°] UCS.RY: Ry orientation [°] UCS.RZ: Rz orientation [°]

✓ return value

Return Values	Name	Data Type	Valid Range	Description
nRet	Return Values	Int	Integer value >= 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.5.33.2. example

```
// Define point name
string pointName = "Point_1";
// Joint coordinates
JointsData joints1 = new JointsData();
// Cartesian pose
Pose point = new Pose();
// Tool frame
Pose TCP = new Pose();
// User coordinate system
Pose UCS = new Pose();
// Read point information by name
int nRet=hr_api.HRIF_ReadPointByName_nJ(0,0, pointname,ref joints1,ref point,ref TCP,ref UCS);
```

3.5.34.HRIF_EnterSafeGuard

3.5.34.1. Description：Forces entry into or exit from a soft emergency stop state by simulating the safety light curtain input.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
bFlag	Status Flag	int	0 / 1	0: Exit soft emergency stop 1: Enter soft emergency stop

✓ return value

Return Values	Name	Data Type	Valid Range	Description
nRet	Return Values	Int	Integer value >= 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

*Note: This command forces entry into/exit from a soft emergency stop by simulating the activation/deactivation of the safety light curtain. After entering the soft emergency stop, the teach pendant or front-end interface will display a safety light curtain active state. After using this command to exit the soft emergency stop, it is necessary to click the "Light Curtain Restore" button on the teach pendant or front-end interface.

3.5.34.2. example

```
int flag=1;
// Robot enters soft emergency stop
int nRet=hr_api.HRIF_EnterSafeGuard(0,0,flag);
```

3.5.35.HRIF_ReadMaxPayload

3.5.35.1. Description：Reads the maximum allowable payload at the robot's end-effector.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

Output Variables	Name	Data Type	Valid Range	Description

dbMaxPayload	Maximum Payload	double	-	Maximum allowable payload at the end-effector, unit: kg.

✓ return value

Return Values	Name	Data Type	Valid Range	Description
nRet	Return Values	Int	Integer value >= 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.5.35.2. Example

```
double dbMaxPayload = 0;
// Read maximum end-effector payload
int nRet = hr_api.HRIF_ReadMaxPayload(0,0,ref dbMaxPayload);
```

3.5.36. HRIF_ReadPayload

3.5.36.1. Description：Reads the current configured payload parameters.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

Output Variables	Name	Data Type	Valid Range	Description
dMass	Mass	double	0 ~ Current max allowed payload	Mass of the configured payload, unit: kg.
dX	Center of Mass X Offset	double	> 0.0	X-direction offset of the center of mass relative to the tool flange, unit: mm.
dY	Center of Mass Y Offset	double	> 0.0	Y-direction offset of the center of mass, unit: mm.
dZ	Center of Mass Z Offset	double	> 0.0	Z-direction offset of the center of mass, unit: mm.

✓ return value

Return Values	Name	Data Type	Valid Range	Description
nRet	Return Values	Int	Integer value >= 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.5.36.2. example

```
double dMass = 0;
```

```
double dX = 0;
double dY = 0;
double dZ = 0;
// Read current payload parameters
int nRet = hr_api.HRIF_ReadPayload(0,0,ref dMass,ref dX,ref dY,ref dZ);
```

3.5.37.HRIF_SetCollideLevel

3.5.37.1. Description：Sets the safety risk (collision sensitivity) level.

✓ input variable

Input Variables	Name	Data Type	Valid Range	Description
boxID	Control Box ID	unsigned int	0~5	Controller Box ID number, Default value = 0
rbtID	Robot ID	unsigned int	0~5	Robot ID number, Default value = 0
nSafeLevel	Safety Risk Level	int	0 ~ 5	Safety risk level: 0: Low safety risk (least sensitive) ... 5: High safety risk (most sensitive)

✓ return value

Return Values	Name	Data Type	Valid Range	Description
nRet	Return Values	Int	Integer value >= 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.5.37.2. example

```
// Define safety risk level
int nSafeLevel = 3;
// Set safety risk level
int nRet= hr_api.HRIF_ SetCollideLevel(0,0, nSafeLevel);
```

3.6. Position, speed, current reading instruction

3.6.1. HRIF_ReadActPos_nJ

3.6.1.1. Description: Reads the current actual position information of the robot. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
joints	Joint Coordinates	JointsData	-	joints.joint[i]: Actual coordinate of joint i, unit: [°].
point	Cartesian Coordinates	Pose	-	point.dX: X coordinate [mm] point.dY: Y coordinate [mm] point.dZ: Z coordinate [mm] point.dRX: Rx orientation [°] point.dRY: Ry orientation [°] point.dRZ: Rz orientation [°]
TCP	Active Tool Frame	Pose	-	TCP.X: X offset [mm] TCP.Y: Y offset [mm] TCP.Z: Z offset [mm] TCP.RX: Rx orientation [°] TCP.RY: Ry orientation [°] TCP.RZ: Rz orientation [°]
UCS	Active User Coordinate System	Pose	-	UCS.X: X offset [mm] UCS.Y: Y offset [mm] UCS.Z: Z offset [mm] UCS.RX: Rx orientation [°] UCS.RY: Ry orientation [°] UCS.RZ: Rz orientation [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.1.2. example

```
// Joint coordinates
JointsData joints1 = new JointsData();
// define space position variable
Pose point = new Pose();
// Active tool frame
Pose TCP = new Pose();
// Active user coordinate system
Pose UCS = new Pose();
// Read current actual position information
int nRet = hr_api.HRIF_ReadActPos_nJ(0, 0, ref joints1, ref point, ref TCP, ref UCS);
```

3.6.2. HRIF_ReadActPos

3.6.2.1. Description: Read the current actual location information.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dX -dRz	Cartesian coordinates	double	-	dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]
dJ1-dJ6	joint coordinates	double	-	dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°] dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6: joint 6 coordinates, unit [°]

dTcp_X-dTcp_Rz	current tool coordinates	double	-	dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]
dUcs_X-dUcs_Rz	current user coordinates	double	-	dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.2.2. example

```
// define space position variable
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// Define the joint position variable
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Read the current actual position information
int nRet= hr_api.HRIF_ReadActPos(0,0, ref dX, ref dY, ref dZ, ref dRx, ref dRy, ref dRz, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6, ref dTcp_X, ref dTcp_Y, ref dTcp_Z, ref dTcp_Rx, ref dTcp_Ry, ref dTcp_Rz, ref dUcs_X, ref dUcs_Y, ref dUcs_Z, ref dUcs_Rx, ref dUcs_Ry, ref dUcs_Rz);
```

3.6.3. HRIF_ReadCmdJointPos

3.6.3.1. Description: Read joint command position.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1-dJ6	joint coordinates	double	-	dJ1: Command position of joint 1, unit [°] dJ2: command position of joint 2, unit [°] dJ3: Command position of joint 3, unit [°] dJ4: Command position of joint 4, unit [°] dJ5: Command position of joint 5, unit [°] dJ6: Command position of joint 6, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.3.2. example

```
// define the joint command position
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// read joint command position
i nt nRet= hr_api.HRIF_ReadCmdJointPos(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);
```

3.6.4. HRIF_ReadCmdJointPos_nJ

3.6.4.1. Description: Reads the commanded joint positions. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective	content

			range	
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
JointsRead	Joint Coordinates	JointsData	-	JointsRead.joint[i]: Commanded position for joint i, unit: [°].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.4.2. example

```
// Define variable for commanded joint positions
JointsData jointsRead = new JointsData();
// Read variable for commanded joint positions
int nRet = hr_api.HRIF_ReadCmdJointPos_nJ(0, 0,ref jointsRead);
```

3.6.5. HRIF_ReadActJointPos_nJ

3.6.5.1. Description: Reads the actual joint positions from the motor feedback. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
JointsRead	Joint Coordinates	JointsData	-	JointsRead.joint[i]: Actual feedback position for joint i, unit: [°].

✓ return value

return value	name	type of data	Effective range	content

nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.5.2. example

```
// Define variable for actual joint positions
JointsData jointsRead = new JointsData();
// Read actual joint positions
int nRet=hr_api.HRIF_ReadActJointPos_nJ(0,0,ref jointsRead);
```

3.6.6. HRIF_ReadActJointPos

3.6.6.1. Description: Read the actual position of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1-dJ6	joint coordinates	double	-	dJ1: actual position of joint 1, unit [°] dJ2: actual position of joint 2, unit [°] dJ3: actual position of joint 3, unit [°] dJ4: actual position of joint 4, unit [°] dJ5: actual position of joint 5, unit [°] dJ6: actual position of joint 6, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.6.2. example

```
// Define the actual position of the joint
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// read the actual position of the joint
int nRet= hr_api.HRIF_ReadActJointPos(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);
```

3.6.7. HRIF_ReadCmdTcpPos

3.6.7.1. Description: Read command TCP position.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dX -dRz	TCP coordinates	double	-	dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.7.2. example

```
// TCP command position
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// read TCP command position
int nRet= hr_api.HRIF_ReadCmdTcpPos(0,0, ref dX, ref dY, ref dZ, ref dRx, ref dRy, ref dRz);
```

3.6.8. HRIF_ReadActTcpPos

3.6.8.1. Description: Read the actual TCP position.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dX -dRz	TCP coordinates	double	-	dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.8.2. example

```
// TCP actual position
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// read TCP actual position
int nRet= hr_api.HRIF_ReadActTcpPos(0,0, ref dX, ref dY, ref dZ, ref dRx, ref dRy, ref dRz);
```

3.6.9. HRIF_ReadCmdJointVel

3.6.9.1. Description: Read joint command velocity.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1-dJ6	joint speed	double	-	dJ1: Joint 1 command speed, unit [°/s] dJ2: Joint 2 command speed, unit [°/s] dJ3: Joint 3 command speed, unit [°/s] dJ4: command speed of joint 4, unit [°/s] dJ5: Joint 5 command speed, unit [°/s] dJ6: Joint 6 command speed, unit [°/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.9.2. example

```
// Define the actual position of the joint
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// read the actual position of the joint
int nRet= hr_api.HRIF_ReadCmdJointVel(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);
```

3.6.10. HRIF_ReadCmdJointVel_nJ

3.6.10.1. Description: Reads the commanded joint velocities. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective	content

			range	
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
jointsRead	Joint Velocities	JointsData	-	jointsRead.joint[i]: Commanded velocity for joint i, unit: [°/s].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.10.2. example

```
// Define variable for joint commanded velocities
JointsData jointsRead = new JointsData();
// Read commanded joint velocities
int nRet = hr_api.HRIF_ReadCmdJointVel_nJ(0, 0, ref jointsRead);
```

3.6.11. HRIF_ReadActJointVel

3.6.11.1. Description: Read the actual speed of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1-dJ6	joint speed	double	-	dJ1: actual speed of joint 1, unit [°/s] dJ2: actual speed of joint 2, unit [°/s] dJ3: actual speed of joint 3, unit [°/s] dJ4: actual speed of joint 4, unit [°/s] dJ5: actual speed of joint 5, unit [°/s] dJ6: actual speed of joint 6, unit [°/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.11.2. example

```
// Define the actual speed of the joint
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// read the actual speed of the joint
int nRet= hr_api.HRIF_ReadActJointVel(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);
```

3.6.12. HRIF_ReadActJointVel_nJ

3.6.12.1. Description: Reads the actual joint velocities from the motor feedback. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective	content

			range	
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
jointsRead	Joint Velocities	JointsData	-	jointsRead.joint[i]: Actual feedback velocity for joint i, unit: [°/s].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.12.2. example

```
// Define variable for actual joint velocities
JointsData jointsRead = new JointsData();
// Read actual joint velocities
int nRet = hr_api.HRIF_ReadActJointVel_nJ(0, 0, ref jointsRead);
```

3.6.13.HRIF_ReadCmdTcpVel

3.6.13.1. Description: Read command TCP speed.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dX -dRz	TCP speed	double	-	dX: X speed, unit [mm/s] dY: Y speed, unit [mm/s] dZ: Z speed, unit [mm/s] dRx: Rx speed, unit [°/s] dRy: Ry speed, unit [°/s] dRz: Rz speed, unit [°/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.13.2. example

```
// TCP command speed
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// read TCP command speed
int nRet= hr_api.HRIF_ReadCmdTcpVel(0,0, ref dX, ref dY, ref dZ, ref dRx, ref dRy, ref dRz);
```

3.6.14.HRIF_ReadActTcpVel

3.6.14.1. Description: Read actual TCP speed.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dX - dRz	TCP speed	double	-	dX: X speed, unit [mm/s] dY: Y speed, unit [mm/s] dZ: Z speed, unit [mm/s] dRx: Rx speed, unit [°/s] dRy: Ry speed, unit [°/s] dRz: Rz speed, unit [°/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.14.2. example

```
// TCP actual speed
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// read TCP actual speed
int nRet= hr_api.HRIF_ReadActTcpVel(0,0, ref dX, ref dY, ref dZ, ref dRx, ref dRy, ref dRz);
```

3.6.15.HRIF_ReadCmdJointCur

3.6.15.1. Description: Read joint command current.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1-dJ6	joint current	double	-	dJ1: joint 1 command current, unit [A] dJ2: joint 2 command current, unit [A] dJ3: joint 3 command current, unit [A] dJ4: joint 4 command current, unit [A] dJ5: joint 5 command current, unit [A] dJ6: joint 6 command current, unit [A]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.15.2. example

```
// Define joint command current
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// read joint command current
int nRet= hr_api.HRIF_ReadCmdJointCur(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);
```

3.6.16.HRIF_ReadCmdJointCur_nJ

3.6.16.1. Description: Reads the commanded joint currents. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
jointsRead	Joint Currents	JointsData	-	jointsRead.joint[i]: Commanded current for joint i, unit: [A].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.16.2. example

```
// Define variable for commanded joint currents
JointsData jointsRead = new JointsData();
// Read commanded joint currents
int nRet = hr_api.HRIF_ReadCmdJointCur_nJ(0, 0, ref jointsRead);
```

3.6.17.HRIF_ReadActJointCur

3.6.17.1. Description: Read the actual current of the joint.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dJ1-dJ6	joint current	double	-	dJ1: joint 1 actual current, unit [A] dJ2: joint 2 actual current, unit [A] dJ3: joint 3 actual current, unit [A] dJ4: joint 4 actual current, unit [A] dJ5: joint 5 actual current, unit [A] dJ6: actual current of joint 6, unit [A]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.17.2. example

```
// Define the actual current of the joint
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// read the actual joint current
int nRet= hr_api.HRIF_ReadActJointCur(0,0, ref dJ1, ref dJ2, ref dJ3, ref dJ4, ref dJ5, ref dJ6);
```

3.6.18.HRIF_ReadActJointCur_nJ

3.6.18.1. Description: Reads the actual joint currents from motor/servo feedback. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0

rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
jointsRead	Joint Currents	JointsData	-	jointsRead.joint[i]: Actual feedback current for joint i, unit: [A].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.18.2. example

```
// Define variable for actual joint currents
JointsData jointsRead = new JointsData();
// Read actual joint currents
int nRet = hr_api.HRIF_ReadActJointCur_nJ(0, 0, ref jointsRead);
```

3.6.19.HRIF_ReadTcpVelocity

3.6.19.1. Description: read TCP terminal speed.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dCmdVel	command speed	double	-	Command speed, unit [mm/s]
dActVel	actual speed	double	-	Actual speed, unit [mm/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.6.19.2. example

```
// Define the TCP speed variable
double dCmdVel = 0;
double dActVel = 0;
// Read the current end TCP speed
int nRet= hr_api.HRIF_ReadTcpVelocity(0,0, ref dCmdVel, ref dActVel);
```

3.7. Coordinate Transformation Calculation Instructions

3.7.1. HRIF_Quaternion2RPY

3.7.1.1. Description: Convert quaternion to Euler angle.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
wxya	W	double	-	W
wxya	Xi	double	-	Xi
wxya	Yj	double	-	Yj
wxya	Z	double	-	Z

✓ output variable

output variable	name	type of data	Effective range	content
wxya	Euler angle Rx	double	>=-180, =<180	Euler angle Rx
wxya	Euler angle Ry	double	>=-180, =<180	Euler angle Ry
wxya	Euler angle Rz	double	>=-180, =<180	Euler angle Rz

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.1.2. example

```
// The quaternion variable that needs to be converted
double dQuaW = 0; double dQuaX = 0; double dQuaY = 0; double dQuaZ = 0;
// Converted Euler angle result
double dRx = 0; double dRy = 0; double dRz = 0;
// convert
int nRet= hr_api.HRIF_Quaternion2RPY(0,0, dQuaW, dQuaX, dQuaY, dQuaZ, ref dRx, ref dRy, ref dRz);
```

3.7.2. HR IF _RPY2Quaternion

3.7.2.1. Description: Convert Euler angles to quaternions.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Rx	Euler angle Rx	double	> = -180, = <180	Euler angle Rx
Ry	Euler angle Ry	double	> = -180, = <180	Euler angle Ry
Rz	Euler angle Rz	double	> = -180, = <180	Euler angle Rz

✓ output variable

output variable	name	type of data	Effective range	content
wxya	W	double	-	W
wxya	Xi	double	-	Xi
wxya	Yj	double	-	Yj
wxya	Z	double	-	Z

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.2.2. example

```
// The converted Euler angle variable that needs to be converted
double dRx = 0; double dRy = 0; double dRz = 0;
// converted quaternion variable
double dQuaW = 0; double dQuaX = 0; double dQuaY = 0; double dQuaZ = 0;
// convert
int nRet= hr_api.HRIF_RPY2Quaternion (0,0, dRx, dRy, dRz, ref dQuaW, ref dQuaX, ref dQuaY, ref dQuaZ);
```

3.7.3. HRIF_GetInverseKin

3.7.3.1. Description: Inverse kinematics solution, the corresponding joint coordinate position is calculated from the specified user coordinate system position and Cartesian coordinates in the tool coordinate system.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dCoord_X-dCoord_Rz	The target Cartesian position for which the inverse solution needs to be calculated	double	-	The target Cartesian position for which the inverse solution needs to be calculated: dCoord_X: X coordinate, unit [mm] dCoord_Y: Y coordinate, unit [mm] dCoord_Z: Z coordinate, unit [mm] dCoord_Rx: Rx coordinates, unit [°] dCoord_Ry: Ry coordinate, unit [°] dCoord_Rz: Rz coordinate, unit [°]
dTcp_X-dTcp_Rz	tool coordinates	double	-	Whether the target position contains tool coordinates (all values = 0 if tool coordinates are not included): dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
dUcs_X-dUcs_Rz	user coordinates	double	-	Whether the target location contains user coordinates (all values = 0 if user coordinates are not included): dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz: Rz coordinates, unit [°]
dJ1-dJ6	Refer to the joint coordinates.	double	-	dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°]

	When there are multiple solutions in the inverse solution, the final solution needs to be selected according to the reference joint coordinates.			dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6: joint 6 coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dTargetJ1-dTargetJ6	joint coordinates	double	-	dTargetJ1: joint 1 coordinates, unit [°] dTargetJ2: Joint 2 coordinates, unit [°] dTargetJ3: joint 3 coordinates, unit [°] dTargetJ4: joint 4 coordinates, unit [°] dTargetJ5: Joint 5 coordinates, unit [°] dTargetJ6: Joint 6 coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.3.2. example

```
// Define the spatial position variable that needs to be converted
double dCoord_X = 420; double dCoord_Y = 0; double dCoord_Z = 445;
double dCoord_Rx = 180; double dCoord_Ry = 0; double dCoord_Rz = 180;
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Define the reference joint position variable
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
```

double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;

// Define the conversion result

double dTargetJ1 = 0; double dTargetJ2 = 0; double dTargetJ3 = 0;

double dTargetJ4 = 0; double dTargetJ5 = 0; double dTargetJ6 = 0;

// Find the inverse

int nRet = hr_api.HRIF_GetInverseKin(0,0, dCoord_X, dCoord_Y, dCoord_Z, dCoord_Rx, dCoord_Ry, dCoord_Rz, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6, dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz, ref dTargetJ1, ref dTargetJ2, ref dTargetJ3, ref dTargetJ4, ref dTargetJ5 , ref dTargetJ6);

3.7.4. HRIF_GetInverseKin_nJ

3.7.4.1. Description: Performs inverse kinematics calculation. Computes the corresponding joint coordinates from a specified Cartesian position in a given user coordinate system (UCS) and tool frame. Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Coord	Target Cartesian Position for IK	Pose	-	Target Cartesian pose for which to compute the inverse kinematics: Coord.dX: X coordinate [mm] Coord.dY: Y coordinate [mm] Coord.dZ: Z coordinate [mm] Coord.dRX: Rx orientation [°] Coord.dRY: Ry orientation [°] Coord.dRZ: Rz orientation [°]
Tcp	Tool Frame	Pose	-	Tool frame to be considered. Set all values to 0 if the target pose does not include a tool offset. Tcp.dX: X offset [mm] Tcp.dY: Y offset [mm] Tcp.dZ: Z offset [mm] Tcp.dRX: Rx orientation [°] Tcp.dRY: Ry orientation [°] Tcp.dRZ: Rz orientation [°]
Ucs	User Coordinate System	Pose	-	User coordinate system to be considered. Set all values to 0 if the target pose is defined in the base frame. Ucs.dX: X offset [mm]

				Ucs.dY: Y offset [mm] Ucs.dZ: Z offset [mm] Ucs.dRX: Rx orientation [°] Ucs.dRY: Ry orientation [°] Ucs.dRZ: Rz orientation [°]
joints	Reference Joint Coordinates	JointsData	-	joints.joint[i]: Reference coordinate for joint i, unit: [°]. Used to select the final solution when multiple inverse kinematics solutions exist.

✓ output variable

output variable	name	type of data	Effective range	content
Target	Joint Coordinates	JointsData	-	Target.joint[i]: Computed coordinate for joint i, unit: [°].

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.4.2. example

```
// // Define the target Cartesian position variable
Pose Coord = new Pose();
Coord.dX = 420; Coord.dY = 0; Coord.dZ = 445;
Coord.dRX = 180; Coord.dRY = 0; Coord.dRZ = 180;
// // Define the tool frame variable
Pose Tcp= new Pose();
Tcp.dX = 0; Tcp.dY = 0; Tcp.dZ = 0;
Tcp.dRX = 0; Tcp.dRY = 0; Tcp.dRZ = 0;
// Define the user coordinate system variable
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Define reference joint position variable
JointsData joints= new JointsData();
for(int i = 0;i<10;i++)
{
    joints.joint[i] = 0;
}
joints.joint[2] = 90;
joints.joint[4] = 90;
// Define variable for the IK result
```

```
JointsData Target=new JointsData();
// Compute inverse kinematics
int nRet = hr_api.HRIF_GetInverseKin_nJ(0, 0,Coord,joints,Tcp,Ucs,ref Target);
```

3.7.5. HRIF_GetForwardKin

3.7.5.1. Description: Kinematics positive solution, calculated from the joint coordinate position to specify the Cartesian coordinate position in the user coordinate system and tool coordinate system.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dJ1-dJ6	Need to calculate the joint coordinates of the positive solution	double	-	dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°] dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6: joint 6 coordinates, unit [°]
dTcp_X-dTcp_Rz	tool coordinates	double	-	Whether the target position contains tool coordinates (all values = 0 if tool coordinates are not included): dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
dUcs_X-dUcs_Rz	user coordinates	double	-	Whether the target location contains user coordinates (all values = 0 if user coordinates are not included): dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz: Rz coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content

dTargetX-dTargetRz	Target Cartesian coordinates	double	-	Compute the target Cartesian position of the positive solution: dTargetX: X coordinate, unit [mm] dTargetY: Y coordinate, unit [mm] dTargetZ: Z coordinate, unit [mm] dTargetRx: Rx coordinates, unit [°] dTargetRy: Ry coordinate, unit [°] dTargetRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.5.2. example

```
// Define the joint position variables that need to be converted
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Define the transformed spatial position result
double dTargetX = 0; double dTargetY = 0; double dTargetZ = 0;
double dTargetRx = 0; double dTargetRy = 0; double dTargetRz = 0;
// seek positive solution
int nRet= hr_api.HRIF_GetForwardKin(0,0, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6, dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs _Rz, ref dTargetX, ref dTargetY, ref dTargetZ, ref dTargetRx, ref dTargetRy, ref dTargetRz);
```

3.7.6. HRIF_GetForwardKin_nJ

3.7.6.1. Description: Forward kinematics, calculates the Cartesian coordinate position under the specified user coordinate system and tool coordinate system from the joint coordinate position. Note: This is the interface used for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Joints	Joint coordinates for forward kinematics calculation	JointsData	-	Joints.joint[i]: Joint i coordinate, unit [°]
Tcp	Tool coordinate	Pose	-	Whether the target position includes tool coordinates (if not, all values = 0): Tcp.dX: X coordinate, unit [mm] Tcp.dY: Y coordinate, unit [mm] Tcp.dZ: Z coordinate, unit [mm] Tcp.dRX: Rx coordinate, unit [°] Tcp.dRY: Ry coordinate, unit [°] Tcp.dRZ: Rz coordinate, unit [°]
Ucs	User coordinate	Pose	-	Whether the target position includes user coordinates (if not, all values = 0): Ucs.dX: X coordinate, unit [mm] Ucs.dY: Y coordinate, unit [mm] Ucs.dZ: Z coordinate, unit [mm] Ucs.dRX: Rx coordinate, unit [°] Ucs.dRY: Ry coordinate, unit [°] Ucs.dRZ: Rz coordinate, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
Target	Target Cartesian coordinates	Pose	-	Calculated target Cartesian position from forward kinematics: Target.dX: X coordinate, unit [mm] Target.dY: Y coordinate, unit [mm] Target.dZ: Z coordinate, unit [mm] Target.dRX: Rx coordinate, unit [°] Target.dRY: Ry coordinate, unit [°] Target.dRZ: Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.6.2. example

```
// Define the joint position variable to be converted
JointsData joints = new JointsData();
for(int i = 0;i<10;i++)
{
    joints.joint[i] = 0;
}
// Define the tool coordinate variable
Pose Tcp= new Pose();
Tcp.dX = 0; Tcp.dY=0 ;Tcp.dZ = 0;
Tcp.dRX = 0; Tcp.dRY=0; Tcp.dRZ = 0;
// Define the user coordinate variable
Pose Ucs= new Pose();
Ucs.dRX=0; Ucs.dRY=0; Ucs.dRZ = 0;
Ucs.dX=0; Ucs.dY=0; Ucs.dZ=0;
// Define the converted spatial position result
Pose Target =new Pose();
// Calculate forward kinematics
int nRet= hr_api.HRIF_GetForwardKin_nJ(0,0,joints,Tcp,Ucs,ref Target);
```

3.7.7. HRIF_Base2UcsTcp

3.7.7.1. Description: Convert the Cartesian coordinate position of the default TCP in the base coordinate system to the Cartesian coordinate position of the specified user coordinate system and the specified TCP.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dCoord_X-dCoord_Rz	Cartesian position based on base coordinate system	double	-	Cartesian positions that need to be transformed: dCoord_X: X coordinate, unit [mm] dCoord_Y: Y coordinate, unit [mm] dCoord_Z: Z coordinate, unit [mm] dCoord_Rx: Rx coordinates, unit [°] dCoord_Ry: Ry coordinate, unit [°] dCoord_Rz: Rz coordinate, unit [°]
dTcp_X-dTcp_Rz	tool coordinates	double	-	Whether the target position contains tool coordinates (all values = 0 if tool coordinates are not included): dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
dUcs_X-dUcs_Rz	user coordinates	double	-	Whether the target location contains user coordinates (all values = 0 if user coordinates are not included): dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz : Rz coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content

dTargetX-dTargetRz	Target Cartesian coordinates	double	-	Specify the Cartesian position in user and tool coordinates: dTargetX: X coordinate, unit [mm] dTargetY: Y coordinate, unit [mm] dTargetZ: Z coordinate, unit [mm] dTargetRx: Rx coordinates, unit [°] dTargetRy: Ry coordinate, unit [°] dTargetRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.7.2. example

```
// Define the spatial position variable that needs to be converted
double dCoord_X = 0; double dCoord_Y = 0; double dCoord_Z = 0;
double dCoord_Rx = 0; double dCoord_Ry = 0; double dCoord_Rz = 0;
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Define the transformed spatial position result
double dTargetX = 0; double dTargetY = 0; double dTargetZ = 0;
double dTargetRx = 0; double dTargetRy = 0; double dTargetRz = 0;
// Convert base coordinates to user coordinates
int nRet = hr_api.HRIF_Base2UcsTcp(0, 0, dCoord_X, dCoord_Y, dCoord_Z, dCoord_Rx, dCoord_Ry, dCoord_Rz,
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,
ref dTargetX, ref dTargetY, ref dTargetZ, ref dTargetRx, ref dTargetRy, ref dTargetRz);
```

3.7.8. HRIF_UcsTcp2Base

3.7.8.1. Description: Convert the Cartesian coordinate position of the specified user coordinate system and the specified TCP to the Cartesian coordinate position of the default TCP in the base coordinate system.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dCoord_X-dCoord_Rz	Cartesian position	double	-	Specify the Cartesian position in user and tool coordinates: dCoord_X: X coordinate, unit [mm] dCoord_Y: Y coordinate, unit [mm] dCoord_Z: Z coordinate, unit [mm] dCoord_Rx: Rx coordinates, unit [°] dCoord_Ry: Ry coordinate, unit [°] dCoord_Rz: Rz coordinate, unit [°]
dTcp_X-dTcp_Rz	tool coordinates	double	-	Whether the target position contains tool coordinates (all values = 0 if tool coordinates are not included): dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
dUcs_X-dUcs_Rz	user coordinates	double	-	Whether the target location contains user coordinates (all values = 0 if user coordinates are not included): dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z : Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz: Rz coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dTargetX-dTargetRz	Target Cartesian coordinates	double	-	Cartesian coordinate position in the base coordinate system: dTargetX: X coordinate, unit [mm] dTargetY: Y coordinate, unit [mm] dTargetZ: Z coordinate, unit [mm] dTargetRx: Rx coordinates, unit [°] dTargetRy: Ry coordinate, unit [°] dTargetRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.8.2. example

```
// Define the spatial position variable that needs to be converted
double dCoord_X = 0; double dCoord_Y = 0; double dCoord_Z = 0;
double dCoord_Rx = 0; double dCoord_Ry = 0; double dCoord_Rz = 0;
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Define the transformed spatial position result
double dTargetX = 0; double dTargetY = 0; double dTargetZ = 0;
double dTargetRx = 0; double dTargetRy = 0; double dTargetRz = 0;
// Convert user coordinates to base coordinates
int nRet = hr_api.HRIF_UcsTcp2Base(0, 0, dCoord_X, dCoord_Y, dCoord_Z, dCoord_Rx, dCoord_Ry, dCoord_Rz,
dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz,
ref dTargetX, ref dTargetY, ref dTargetZ, ref dTargetRx, ref dTargetRy, ref dTargetRz);
```

3.7.9. HRIF_PoseAdd

3.7.9.1. Description: Point addition calculation, using matrix left multiplication operation (the second point is left multiplied by the first point).

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X-dPose1_Rz	space coordinate 1	double	-	Space coordinates to be calculated 1: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinates, unit [°] dPose1_Ry: Ry coordinates, unit [°] dPose1_Rz: Rz coordinates, unit [°]
dPose2_X-dPose2_Rz	space coordinates 2	double	-	Space coordinates to be calculated 2: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm] dPose2_Rx: Rx coordinates, unit [°] dPose2_Ry: Ry coordinates, unit [°] dPose2_Rz: Rz coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dTargetX-dTargetRz	Calculation results	double	-	Calculation results: dTargetX: X coordinate, unit [mm] dTargetY: Y coordinate, unit [mm] dTargetZ: Z coordinate, unit [mm] dTargetRx: Rx coordinates, unit [°] dTargetRy: Ry coordinate, unit [°] dTargetRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer	nRet=0: return the function call successfully

			value >=0	nRet>0: return the error code of the call failure

3.7.9.2. example

// Define the space coordinates 1 that need to be calculated

double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;

double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;

// Define the space coordinates 2 that need to be calculated

double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;

double dPose2_Rx =180; double dPose2_Ry = 0; double dPose2_Rz = 180;

// Calculation results

double dTargetX = 0; double dTargetY = 0; double dTargetZ = 0;

double dTargetRx = 0; double dTargetRy = 0; double dTargetRz = 0;

// Calculation results

int nRet= hr_api.HRIF_PoseAdd(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz, dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, ref dTargetX, ref dTargetY, ref dTargetZ, ref dTargetRx, ref dTargetRy, ref dTargetRz);

3.7.10.HRIF_PoseSub

3.7.10.1. Description: Point subtraction calculation, with the second point as the reference point.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X-dPose1_Rz	space coordinate 1	double	-	Space coordinates to be calculated 1: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinates, unit [°] dPose1_Ry: Ry coordinates, unit [°] dPose1_Rz: Rz coordinates, unit [°]
dPose2_X-dPose2_Rz	space coordinates 2	double	-	Space coordinates to be calculated 2: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm] dPose2_Rx: Rx coordinates, unit [°] dPose2_Ry: Ry coordinates, unit [°] dPose2_Rz: Rz coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dTargetX-dTargetRz	Calculate coordinates	double	-	Calculation results: dTargetX: X coordinate, unit [mm] dTargetY: Y coordinate, unit [mm] dTargetZ: Z coordinate, unit [mm] dTargetRx: Rx coordinates, unit [°] dTargetRy: Ry coordinate, unit [°] dTargetRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.10.2. example

```
// Define the space coordinates 1 that need to be calculated
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
// Define the space coordinates 2 that need to be calculated
double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;
double dPose2_Rx =180; double dPose2_Ry = 0; double dPose2_Rz = 180;
// Calculation results
double dTargetX = 0; double dTargetY = 0; double dTargetZ = 0;
double dTargetRx = 0; double dTargetRy = 0; double dTargetRz = 0;
// Calculation results
int nRet= hr_api.HRIF_PoseSub(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,
dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, ref dTargetX, ref dTargetY,
ref dTargetZ, ref dTargetRx, ref dTargetRy, ref dTargetRz);
```

3.7.11. HRIF_PoseTrans

3.7.11.1. Description: Convert the Cartesian coordinate position Pose1 in the base coordinate system to the Cartesian coordinate position in the user coordinate system Pose2.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X-dPose1_Rz	Coordinate position 1	double	-	Coordinate position 1: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinates, unit [°] dPose1_Ry: Ry coordinates, unit [°] dPose1_Rz: Rz coordinates, unit [°]
dPose2_X-dPose2_Rz	Coordinate position 2	double	-	Coordinate position 2: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm] dPose2_Rx: Rx coordinates, unit [°] dPose2_Ry: Ry coordinates, unit [°] dPose2_Rz: Rz coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dTargetX-dTargetRz	Calculate coordinates	double	-	Calculation results: dTargetX: X coordinate, unit [mm] dTargetY: Y coordinate, unit [mm] dTargetZ: Z coordinate, unit [mm] dTargetRx: Rx coordinates, unit [°] dTargetRy: Ry coordinate, unit [°] dTargetRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer	nRet=0: return the function call successfully

			value >=0	nRet>0: return the error code of the call failure

3.7.11.2. example

```
// Define the space coordinates 1 that need to be calculated
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
// Define the space coordinates 2 that need to be calculated
double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;
double dPose2_Rx =180; double dPose2_Ry = 0; double dPose2_Rz = 180;
// Calculation results
double dTargetX = 0; double dTargetY = 0; double dTargetZ = 0;
double dTargetRx = 0; double dTargetRy = 0; double dTargetRz = 0;
// Calculation results
int nRet= hr_api.HRIF_PoseTrans(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,
dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, ref dTargetX, ref dTargetY,
ref dTargetZ, ref dTargetRx, ref dTargetRy, ref dTargetRz);
```

3.7.12.HRIF_PoseInverse

3.7.12.1. Description: Coordinate kinematics inverse transformation.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X-dPose1_Rz	space coordinate 1	double	-	Space coordinates to be calculated 1: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinates, unit [°] dPose1_Ry: Ry coordinates, unit [°] dPose1_Rz: Rz coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dTargetX-dTargetRz	Calculate coordinates	double	-	Calculation results: dTargetX: X coordinate, unit [mm] dTargetY: Y coordinate, unit [mm] dTargetZ: Z coordinate, unit [mm] dTargetRx: Rx coordinates, unit [°] dTargetRy: Ry coordinate, unit [°] dTargetRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.12.2. example

```
// Define the space coordinates 1 that need to be calculated
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
// Calculation results
double dTargetX = 0; double dTargetY = 0; double dTargetZ = 0;
double dTargetRx = 0; double dTargetRy = 0; double dTargetRz = 0;
```

// Calculation results

int nRet= hr_api.HRIF_PoseInverse(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz, ref dTargetX, ref dTargetY, ref dTargetZ, ref dTargetRx, ref dTargetRy, ref dTargetRz);

3.7.13.HRIF_PoseDist

3.7.13.1. Description: Calculate point distance.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X-dPose1_Rz	space coordinate 1	double	-	Space coordinates to be calculated 1: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinates, unit [°] dPose1_Ry: Ry coordinates, unit [°] dPose1_Rz: Rz coordinates, unit [°]
dPose2_X-dPose2_Rz	space coordinates 2	double	-	Space coordinates to be calculated 2: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm] dPose2_Rx: Rx coordinates, unit [°] dPose2_Ry: Ry coordinates, unit [°] dPose2_Rz: Rz coordinates, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dDistance	point distance	double	-	Point distance, unit [mm]
dAngle	attitude distance	double	-	Attitude distance, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.13.2. example

// Define the space coordinates 1 that need to be calculated

double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;

double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;

// Define the space coordinates 2 that need to be calculated

double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;

double dPose2_Rx =180; double dPose2_Ry = 0; double dPose2_Rz = 180;

// define calculation result

double dDistance = 0;

double dAngle = 0;

// Calculation results

int nRet= hr_api.HRIF_PoseDist(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz, dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, ref dDistance, ref dAngle);

3.7.14.HRIF_Poseinterpolate

3.7.14.1. Description: Linear interpolation calculation of spatial position.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X-dPose1_Rz	space coordinate 1	double	-	Space coordinates to be calculated 1: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinates, unit [°] dPose1_Ry: Ry coordinates, unit [°] dPose1_Rz: Rz coordinates, unit [°]
dPose2_X-dPose2_Rz	space coordinates 2	double	-	Space coordinates to be calculated 2: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm] dPose2_Rx: Rx coordinates, unit [°] dPose2_Ry: Ry coordinates, unit [°] dPose2_Rz: Rz coordinates, unit [°]
dAlpha	imputation ratio	double	0-1	dAlpha = 0: dPose3 = dPose1 dAlpha = 1: dPose3 = dPose2 0-1: According to the position of dPose1 to dPose2, the ratio is dAlpha and returns to dPose3

✓ output variable

output variable	name	type of data	Effective range	content
dTargetX-dTargetRz	Calculate coordinates	double	-	Calculation results: dTargetX: X coordinate, unit [mm] dTargetY : Y coordinate, unit [mm] dTargetZ: Z coordinate, unit [mm] dTargetRx: Rx coordinates, unit [°] dTargetRy: Ry coordinate, unit [°] dTargetRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.14.2. example

```
// Define the space coordinates 1 that need to be calculated
double dPose1_X = 420; double dPose1_Y = 0; double dPose1_Z = 445;
double dPose1_Rx = 180; double dPose1_Ry = 0; double dPose1_Rz = 180;
// Define the space coordinates 2 that need to be calculated
double dPose2_X = 420; double dPose2_Y = 50; double dPose2_Z = 445;
double dPose2_Rx =180; double dPose2_Ry = 0; double dPose2_Rz = 180;
// interpolation ratio
double dAlpha = 0.5;
// Calculation results
double dTargetX = 0; double dTargetY = 0; double dTargetZ = 0;
double dTargetRx = 0; double dTargetRy = 0; double dTargetRz = 0;
// Calculation results
int nRet= hr_api.HRIF_PoseInterpolate(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz,
dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, dAlpha, ref d TargetX, ref dTargetY,
ref dTargetZ, ref dTargetRx, ref dTargetRy, ref dTargetRz);
```

3.7.15.HRIF_PoseDefdFrame

3.7.15.1. Description: Rotate and calculate based on the center of the trajectory, p1, p2, p3 are the feature points of the trajectory selected before rotation, p4, p5, p6 are the feature points of the trajectory selected after rotation, and the calculation result is expressed as the user coordinate system of the rotation feature.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X - dPose1_Z	space coordinate 1	double	-	Space coordinates to be calculated 1: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm]
dPose2_X - dPose2_Z	space coordinates 2	double	-	Space coordinates to be calculated 2: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm]
dPose3_X - dPose3_Z	Space coordinates 3	double	-	Space coordinates to be calculated 3: dPose3_X: X coordinate, unit [mm] dPose3_Y: Y coordinate, unit [mm] dPose3_Z: Z coordinate, unit [mm]
dPose4_X - dPose4_Z	Space Coordinates 4	double	-	Space coordinates to be calculated 4: dPose4_X: X coordinate, unit [mm] dPose4_Y: Y coordinate, unit [mm] dPose4_Z: Z coordinate, unit [mm]
dPose5_X - dPose5_Z	space coordinates 5	double	-	Space coordinates to be calculated 5: dPose5_X: X coordinate, unit [mm] dPose5_Y: Y coordinate, unit [mm] dPose5_Z: Z coordinate, unit [mm]
dPose6_X - dPose6_Z	Space coordinates 6	double	-	Space coordinates to be calculated 6: dPose6_X : X coordinate, unit [mm] dPose6_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm]

✓ output variable

output variable	name	type of data	Effective range	content
dUcs_X-dUcs_Rz	Calculation results	double	-	Calculation results: dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm]

				dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.7.15.2. example

```
// Define the space coordinates 1 that need to be calculated
double dPose1_X = 0; double dPose1_Y = 0; double dPose1_Z = 0;
// Define the space coordinates 2 that need to be calculated
double dPose2_X = 0; double dPose2_Y = 0; double dPose2_Z = 0;
// Define the space coordinates 3 that need to be calculated
double dPose3_X = 0; double dPose3_Y = 0; double dPose3_Z = 0;
// Define the space coordinates that need to be calculated 4
double dPose4_X = 0; double dPose4_Y = 0; double dPose4_Z = 0;
// Define the space coordinates that need to be calculated 5
double dPose5_X = 0; double dPose5_Y = 0; double dPose5_Z = 0;
// Define the space coordinates to be calculated 6
double dPose6_X = 0; double dPose6_Y = 0; double dPose6_Z = 0;
// Calculation result
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Calculation results
int nRet= hr_api.HRIF_PoseDefdFrame(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose2_X, dPose2_Y, dPose2_Z, dPose3_X, dPose3_Y, dPose3_Z, dPose4_X, dPose4_Y, dPose4_Z, dPose5_X, dPose5 _Y, dPose5_Z, dPose6_X, dPose6_Y, dPose6_Z, ref dUcs_X, ref dUcs_Y, ref dUcs_Z, ref dUcs_Rx, ref dUcs_Ry, ref dUcs_Rz);
```

3.8. Tool coordinates and user coordinates read and write commands

3.8.1. HRIF_SetTCP

3.8.1.1. Description: Set the current tool coordinates, do not write to the configuration file, and become invalid after restarting.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dTcp_X-dTcp_Rz	tool coordinates	double	-	Tool coordinates that need to be set: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.1.2. example

```
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// set tool coordinates
int nRet= hr_api.HRIF_SetTCP(0,0, dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz);
```

3.8.2. HRIF_SetUCS

3.8.2.1. Description: Set the current user coordinates, do not write to the configuration file, and will fail after restarting.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dUcs_X-dUcs_Rz	user coordinates	double	-	User coordinates that need to be set: dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.2.2. example

```
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// set user coordinates
int nRet= hr_api.HRIF_SetUCS(0,0, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.8.3. HRIF_ReadCurTCP

3.8.3.1. Description: Read the currently set tool coordinates.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dTcp_X-dTcp_Rz	tool coordinates	double	-	The tool coordinates read: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.3.2. example

```
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// read tool coordinates
int nRet= hr_api.HRIF_ReadCurTCP(0,0, ref dTcp_X, ref dTcp_Y, ref dTcp_Z, ref dTcp_Rx, ref dTcp_Ry,
ref dTcp_Rz);
```

3.8.4. HRIF_ReadCurUCS

3.8.4.1. Description: Read the currently set user coordinate value.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dUcs_X-dUcs_Rz	user coordinates	double	-	Read user coordinates: dUcs_X : X coordinate, unit [mm] dUcs_Y : Y coordinate, unit [mm] dUcs_Z : Z coordinate, unit [mm] dUcs_Rx : Rx coordinate, unit [°] dUcs_Ry : Ry coordinate, unit [°] dUcs_Rz : Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.4.2. example

```
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// read user coordinates
int nRet= hr_api.HRIF_ReadCurUCS(0,0, ref dUcs_X, ref dUcs_Y, ref dUcs_Z, ref dUcs_Rx, ref dUcs_Ry,
ref dUcs_Rz);
```

3.8.5. HRIF_SetTCPByName

3.8.5.1. Description: Set the value in the tool coordinates list to the current tool coordinates by name, and the corresponding name is the name of the tool taught by TCP on the teach pendant configuration page.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTcpName	Tool Coordinate Name	string	-	The tool coordinate name that needs to be set

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.5.2. example

```
// The name of the tool coordinates that need to be delivered
string sTcpName = "TCP";
// set tool coordinates
int nRet= hr_api.HRIF_SetTCCPByName(0,0, sTcpName);
```

3.8.6. HRIF_SetUCSByName

3.8.6.1. Description: Set the value in the user coordinate list to the current user coordinate by name, and the corresponding name is the name of the user coordinate teaching on the teach pendant configuration page.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sUcsName	User Coordinate Name	string	-	The user coordinate name that needs to be set

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.6.2. example

```
// The name of the user coordinates that need to be delivered
string sUcsName = "Base";
// set user coordinates
int nRet= hr_api.HRIF_SetUCSByName(0,0, sUcsName);
```

3.8.7. HRIF_ReadTCPByName

3.8.7.1. Description: Read the specified TCP by name Coordinates, the corresponding name is the teach pendant configuration page TCP The taught tool name.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinates to be read

✓ output variable

output variable	name	type of data	Effective range	content
dTcp_X-dTcp_Rz	tool coordinates	double	-	The tool coordinates read: dTcp_X : X coordinate, unit [mm] dTcp_Y : Y coordinate, unit [mm] dTcp_Z : Z coordinate, unit [mm] dTcp_Rx : Rx coordinate, unit [°] dTcp_Ry : Ry coordinate, unit [°] dTcp_Rz : Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.7.2. example

```
// The name of the tool coordinates that need to be read
string sTcpName = "TCP";
// Define the read tool coordinate result variable
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// read tool coordinates
int nRet= hr_api.HRIF_ReadTCPByName(0,0,sTcpName, ref dTcp_X, ref dTcp_Y, ref dTcp_Z, ref dTcp_Rx,
ref dTcp_Ry, ref dTcp_Rz);
```

3.8.8. HRIF_ReadUCSByName

3.8.8.1. Description: Read the specified UCS by name Coordinates, the corresponding name is the name of the user coordinates taught by the user coordinates on the teach pendant configuration page.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sUcsName	User Coordinate Name	string	-	The name of the user coordinates that need to be read

✓ output variable

output variable	name	type of data	Effective range	content
dUcs_X-dUcs_Rz	user coordinates	double	-	Read user coordinates: dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz : Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.8.2. example

```
// The name of the user coordinates that need to be read
string sUcsName = "Base";
// Define the read user coordinate variable
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0 ;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// read user coordinates
int nRet= hr_api.HRIF_ReadUCSByName(0,0, sUcsName , ref dUcs_X, ref dUcs_Y, ref dUcs_Z, ref dUcs_Rx, ref dUcs_Ry, ref dUcs_Rz);
```

3.8.9. HRIF_ReadTCPList

3.8.9.1. Description: Reads the list of saved TCP names in the system.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
TCPList	TCP Name List	List<string>	-	Saves all read TCP names

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.9.2. example

```
// Define the variable to save the TCP list
List<string> TCPList = new List<string>();
// Read the TCP name list
int nRet = hr_api.HRIF_ReadTCPList(0,0, ref TCPList);
```

3.8.10. HRIF_ReadUCSList

3.8.10.1. Description: Reads the list of saved UCS names in the system.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
UCSList	UCS Name	List<string>	-	Saves all read UCS names

	List			

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.10.2. example

```
// Define the variable to save the UCS list
List<string> UCSList = new List<string>();
// Read the user coordinate system name list
int nRet = hr_api.HRIF_ReadUCSList(0,0, ref UCSList);
```

3.8.11. HRIF_ConfigTCP

3.8.11.1. Description: Creates a new TCP with the specified name and values.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTcpName	Tool coordinate name	string	-	Tool coordinate name to be created
dTcp_X-dTcp_Rz	Tool coordinate	double	-	Tool coordinate to be set: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinate, unit [°] dTcp_Ry: Ry coordinate, unit [°] dTcp_Rz: Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.11.2. example

```
// Define tool coordinate name
string sTcpName = "TCP_A";
// Define tool coordinate variables
```

double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 10;

double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;

// Create a new TCP with the specified name and values

int nRet = hr_api.HRIF_ConfigTCP(0,0, sTcpName , dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz);

3.8.12.HRIF_CalUcsLine

3.8.12.1. Description: Calculates UCS using the two-point line method.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X- dPose1_Rz	Spatial coordinate 1	double	-	Pose of point 1 in the Base coordinate system with the system default TCP: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinate, unit [°] dPose1_Ry: Ry coordinate, unit [°] dPose1_Rz: Rz coordinate, unit [°]
dPose2_X- dPose2_Rz	Spatial coordinate 2	double	-	Pose of point 2 in the Base coordinate system with the system default TCP: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm] dPose2_Rx: Rx coordinate, unit [°] dPose2_Ry: Ry coordinate, unit [°] dPose2_Rz: Rz coordinate, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dRetPose_X- dRetPose_Rz	Calculation result	double	-	Calculated UCS pose: dRetPose_X: X coordinate, unit [mm] dRetPose_Y: Y coordinate, unit [mm] dRetPose_Z: Z coordinate, unit [mm] dRetPose_Rx: Rx coordinate, unit [°] dRetPose_Ry: Ry coordinate, unit [°] dRetPose_Rz: Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.12.2. example

```
// Define spatial coordinate 1 for calculation
double dPose1_X = 10; double dPose1_Y = 0; double dPose1_Z = 0;double dPose1_Rx = 10; double dPose1_Ry = 0; double dPose1_Rz = 0;
// Define spatial coordinate 2 for calculation
double dPose2_X = 0; double dPose2_Y = 10; double dPose2_Z = 0;double dPose2_Rx = 0; double dPose2_Ry = 10; double dPose2_Rz = 0;
// Define calculation result variables
double dRetPose_X = 0; double dRetPose_Y = 0; double dRetPose_Z = 0;
double dRetPose_Rx = 0; double dRetPose_Ry = 0; double dRetPose_Rz = 0;
// Get calculation result. After function call, nRet value is 0, and dRetPose_X~dRetPose_Rz values are: 10,0,0,0,-7.107,-45
int nRet= hr_api.HRIF_CalUcsLine(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz, dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz,ref dRetPose_X,ref dRetPose_Y, ref dRetPose_Z, ref dRetPose_Rx, ref dRetPose_Ry, ref dRetPose_Rz);
```

3.8.13.HRIF_ConfigUCS

3.8.13.1. Description: Creates a new UCS with the specified name and values.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sUcsName	User coordinate name	string	-	User coordinate name to be created
dUcs_X-dUcs_Rz	User coordinate	double	-	User coordinate to be set: dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinate, unit [°] dUcs_Ry: Ry coordinate, unit [°] dUcs_Rz: Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.13.2. example

```
// Define user coordinate name
string sUcsName = " Point_A";
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 10;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Create a new UCS with the specified name and values
int nRet = hr_api.HRIF_ConfigUCS(0,0, sUcsName , dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.8.14.HRIF_CalTcp3P

3.8.14.1. Description: Calculates TCP using the three-point plane method.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X- dPose1_Rz	Spatial coordinate 1	double	-	Pose of point 1 in the Base coordinate system with the system default TCP: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinate, unit [°] dPose1_Ry: Ry coordinate, unit [°] dPose1_Rz: Rz coordinate, unit [°]
dPose2_X- dPose2_Rz	Spatial coordinate 2	double	-	Pose of point 2 in the Base coordinate system with the system default TCP: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm] dPose2_Rx: Rx coordinate, unit [°] dPose2_Ry: Ry coordinate, unit [°] dPose2_Rz: Rz coordinate, unit [°]
dPose3_X- dPose3_Rz	Spatial coordinate 3	double	-	Position of point 3 in the Base coordinate system with the system default TCP:

				dPose3_X: X coordinate, unit [mm] dPose3_Y: Y coordinate, unit [mm] dPose3_Z: Z coordinate, unit [mm] dPose3_Rx: Rx coordinate, unit [°] dPose3_Ry: Ry coordinate, unit [°] dPose3_Rz: Rz coordinate, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dRetPose_X- dRetPose_Rz	Calculation result	double	-	Calculated UCS pose: dRetPose_X: X coordinate, unit [mm] dRetPose_Y: Y coordinate, unit [mm] dRetPose_Z: Z coordinate, unit [mm] dRetPose_Rx: Rx coordinate, unit [°], generally 0 dRetPose_Ry: Ry coordinate, unit [°], generally 0 dRetPose_Rz: Rz coordinate, unit [°], generally 0
quality	Result quality	int	0/1/2	0: Good 1: Poor (calculation result better not used) 2: Abnormal

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.14.2. example

```
// Define spatial coordinate 1 for calculation
double dPose1_X = 10; double dPose1_Y = 0; double dPose1_Z = 0;double dPose1_Rx = 10; double dPose1_Ry = 0;
double dPose1_Rz = 0;
// Define spatial coordinate 2 for calculation
double dPose2_X = 0; double dPose2_Y = 10; double dPose2_Z = 0;double dPose2_Rx = 10; double dPose2_Ry = 0;
double dPose2_Rz = 0;
// Define spatial coordinate 3 for calculation
double dPose3_X = 0; double dPose3_Y = 10; double dPose3_Z = 0;double dPose3_Rx = 10; double dPose3_Ry = 0;
double dPose3_Rz = 0;
// Define calculation result variables
double dRetPose_X = 0; double dRetPose_Y = 0; double dRetPose_Z = 0;
double dRetPose_Rx = 0; double dRetPose_Ry = 0; double dRetPose_Rz = 0;
// Define calculation result quality variable
int quality = 0;
```

// Get calculation result. After function call, nRet value is 20031 (point calculation failed).

int nRet= hr_api.HRIF_CalTcp3P(0,0, dPose1_X, dPose1_Y, dPose1_Z, dPose1_Rx, dPose1_Ry, dPose1_Rz, dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Rx, dPose2_Rz, dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry, dPose3_Rz, ref dRetPose_X, ref dRetPose_Y, ref dRetPose_Z, ref dRetPose_Rx,ref dRetPose_Ry, ref dRetPose_Rz, ref quality);

3.8.15.HRIF_CalTcp4P

3.8.15.1. Description: Calculates TCP using the four-point plane method.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dPose1_X- dPose1_Rz	Spatial coordinate 1	double	-	Pose of point 1 in the Base coordinate system with the system default TCP: dPose1_X: X coordinate, unit [mm] dPose1_Y: Y coordinate, unit [mm] dPose1_Z: Z coordinate, unit [mm] dPose1_Rx: Rx coordinate, unit [°] dPose1_Ry: Ry coordinate, unit [°] dPose1_Rz: Rz coordinate, unit [°]
dPose2_X- dPose2_Rz	Spatial coordinate 2	double	-	Pose of point 2 in the Base coordinate system with the system default TCP: dPose2_X: X coordinate, unit [mm] dPose2_Y: Y coordinate, unit [mm] dPose2_Z: Z coordinate, unit [mm] dPose2_Rx: Rx coordinate, unit [°] dPose2_Ry: Ry coordinate, unit [°] dPose2_Rz: Rz coordinate, unit [°]
dPose3_X- dPose3_Rz	Spatial coordinate 3	double	-	Position of point 3 in the Base coordinate system with the system default TCP: dPose3_X: X coordinate, unit [mm] dPose3_Y: Y coordinate, unit [mm] dPose3_Z: Z coordinate, unit [mm] dPose3_Rx: Rx coordinate, unit [°] dPose3_Ry: Ry coordinate, unit [°] dPose3_Rz: Rz coordinate, unit [°]
dPose4_X- dPose4_Rz	Spatial coordinate 3	double	-	Position of point 4 in the Base coordinate system with the system default TCP: dPose4_X: X coordinate, unit [mm]

				dPose4_Y: Y coordinate, unit [mm] dPose4_Z: Z coordinate, unit [mm] dPose4_Rx: Rx coordinate, unit [°] dPose4_Ry: Ry coordinate, unit [°] dPose4_Rz: Rz coordinate, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content
dRetPose_X- dRetPose_Rz	Calculation result	double	-	Calculated UCS pose: dRetPose_X: X coordinate, unit [mm] dRetPose_Y: Y coordinate, unit [mm] dRetPose_Z: Z coordinate, unit [mm] dRetPose_Rx: Rx coordinate, unit [°], generally 0 dRetPose_Ry: Ry coordinate, unit [°], generally 0 dRetPose_Rz: Rz coordinate, unit [°], generally 0
quality	Result quality	int	0/1/2	0: Good 1: Poor (calculation result better not used) 2: Abnormal
errorIndex_P1~4	Source point error indication	int	0/1	0: Normal 1: Abnormal

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.15.2. example

// Define spatial coordinate 1 for calculation

double dPose1_X = 10; double dPose1_Y = 0; double dPose1_Z = 0;double dPose1_Rx = 10; double dPose1_Ry = 0; double dPose1_Rz = 0;

// Define spatial coordinate 2 for calculation

double dPose2_X = 0; double dPose2_Y = 10; double dPose2_Z = 0;double dPose2_Rx = 10; double dPose2_Ry = 0; double dPose2_Rz = 0;

// Define spatial coordinate 3 for calculation

double dPose3_X = 0; double dPose3_Y = 10; double dPose3_Z = 0;double dPose3_Rx = 10; double dPose3_Ry = 0; double dPose3_Rz = 0;

// Define spatial coordinate 4 for calculation

double dPose4_X = 0; double dPose4_Y = 10; double dPose4_Z = 0;double dPose4_Rx = 10; double dPose4_Ry = 0; double dPose4_Rz = 0;

// Define calculation result variables

double dRetPose_X = 0; double dRetPose_Y = 0; double dRetPose_Z = 0;

double dRetPose_Rx = 0; double dRetPose_Ry = 0; double dRetPose_Rz = 0;

// Define calculation result quality variables

int quality = 0; int errorIndex_P1=0; int errorIndex_P2=0; int errorIndex_P3=0; int errorIndex_P4=0;

// Get calculation result. After function call, nRet value is 0, quality value is 2, errorIndex_P1~P3 values are 1, errorIndex_P4 value is 0.

int nRet= hr_api.HRIF_CalTcp4P(0,0,dPose1_X,dPose1_Y,dPose1_Z,dPose1_Rx,dPose1_Ry,dPose1_Rz,dPose2_X, dPose2_Y, dPose2_Z, dPose2_Rx, dPose2_Ry, dPose2_Rz, dPose3_X, dPose3_Y, dPose3_Z, dPose3_Rx, dPose3_Ry, dPose3_Rz, dPose4_X, dPose4_Y, dPose4_Z, dPose4_Rx, dPose4_Ry, dPose4_Rz,ref dRetPose_X, ref dRetPose_Y,ref dRetPose_Z,ref dRetPose_Rx,ref dRetPose_Ry,ref dRetPose_Rz,ref quality,ref errorIndex_P1,ref errorIndex_P2,ref errorIndex_P3,ref errorIndex_P4);

3.8.16.HRIF_CalTCPOrt

3.8.16.1. Description: Calibrates tcp orientation.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
UCS_X- UCS_Rz	UCS orientation	double	-	Currently set ucs orientation: UCS_X: X coordinate, unit [mm] UCS_Y: Y coordinate, unit [mm] UCS_Z: Z coordinate, unit [mm] UCS_Rx: Rx coordinate, unit [°] UCS_Ry: Ry coordinate, unit [°] UCS_Rz: Rz coordinate, unit [°]
TCP_X- TCP_Rz	Default TCP pose	double	-	Pose of point 2 in the Base coordinate system with the system default TCP: TCP_X: X coordinate, unit [mm] TCP_Y: Y coordinate, unit [mm] TCP_Z: Z coordinate, unit [mm] TCP_Rx: Rx coordinate, unit [°] TCP_Ry: Ry coordinate, unit [°] TCP_Rz: Rz coordinate, unit [°]

✓ output variable

output variable	name	type of data	Effective range	content

RX-RY- RZ	Calculation result	double	-	Calculated UCS orientation: Rx: Rx coordinate, unit [°], generally 0 Ry: Ry coordinate, unit [°], generally 0 Rz: Rz coordinate, unit [°], generally 0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.16.2. example

```
// Define default TCP pose
double TCP_x=0;double TCP_y=0;double TCP_z=0;
double TCP_Rx=0;double TCP_Ry=0;double TCP_Rz=0;
// Define UCS orientation
double UCS_x=84.303;double UCS_y=341.461;double UCS_z=457.954;
double UCS_Rx=179.863;double UCS_Ry=1.916;double UCS_Rz=-107.973;
double RX = 0, RY = 0, RZ = 0;
// Perform calibration
int nRet=hr_api.HRIF_CalTCPOrt(0, 0,UCS_x,UCS_y,UCS_z,UCS_Rx,UCS_Ry,UCS_Rz,
                               TCP_x,TCP_y,TCP_z,TCP_Rx,TCP_Ry,TCP_Rz,ref RX,ref RY,ref RZ);
```

3.8.17.HRIF_GetInverseDynamics

3.8.17.1. Description: Inverse dynamics, calculates corresponding joint torque from specified joint position, joint velocity, joint acceleration.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
dPose_J1-dPose_J6	Joint position for inverse dynamics calculation	double	-	Joint coordinates for which to calculate inverse dynamics: dPose_J1: Joint 1 coordinate, unit [°] dPose_J2: Joint 2 coordinate, unit [°] dPose_J3: Joint 3 coordinate, unit [°] dPose_J4: Joint 4 coordinate, unit [°] dPose_J5: Joint 5 coordinate, unit [°] dPose_J6: Joint 6 coordinate, unit [°]
dJ1-dJ6	Joint velocity for inverse	double	-	dVel_J1: Joint 1 velocity, unit [°/s] dVel_J2: Joint 2 velocity, unit [°/s]

	dynamics calculation			dVel_J3: Joint 3 velocity, unit [°/s] dVel_J4: Joint 4 velocity, unit [°/s] dVel_J5: Joint 5 velocity, unit [°/s] dVel_J6: Joint 6 velocity, unit [°/s]
dTcp_X-dTcp_Rz	Joint acceleration for inverse dynamics calculation	double	-	dAcc_J1: Joint 1 acceleration, unit [°/s²] dAcc_J2: Joint 2 acceleration, unit [°/s²] dAcc_J3: Joint 3 acceleration, unit [°/s²] dAcc_J4: Joint 4 acceleration, unit [°/s²] dAcc_J5: Joint 5 acceleration, unit [°/s²] dAcc_J6: Joint 6 acceleration, unit [°/s²]

✓ output variable

output variable	name	type of data	Effective range	content
dTorq_J1- dTorq_J6	Calculated joint torque	double	-	dTorq_J1: Joint 1 torque, unit [Nm] dTorq_J2: Joint 2 torque, unit [Nm] dTorq_J3: Joint 3 torque, unit [Nm] dTorq_J4: Joint 4 torque, unit [Nm] dTorq_J5: Joint 5 torque, unit [Nm] dTorq_J6: Joint 6 torque, unit [Nm]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.17.2. example

```
// Define joint parameters
double dPose_J1 = 0, dPose_J2 = 0, dPose_J3 = 90, dPose_J4 = 0, dPose_J5 = 90, dPose_J6 = 0;
double dVel_J1 = 0, dVel_J2 = 0, dVel_J3 = 0, dVel_J4 = 0, dVel_J5 = 0, dVel_J6 = 0;
double dAcc_J1 = 0, dAcc_J2 = 0, dAcc_J3 = 0, dAcc_J4 = 0, dAcc_J5 = 0, dAcc_J6 = 0;
double dTorq_J1 = 0, dTorq_J2 = 0, dTorq_J3 = 0, dTorq_J4 = 0, dTorq_J5 = 0, dTorq_J6 = 0;
// Get inverse dynamics calculation result
int nRet = hr_api.HRIF_GetInverseDynamics(0, 0,
        dPose_J1, dPose_J2, dPose_J3, dPose_J4, dPose_J5, dPose_J6,
        dVel_J1, dVel_J2, dVel_J3, dVel_J4, dVel_J5, dVel_J6,
        dAcc_J1, dAcc_J2, dAcc_J3, dAcc_J4, dAcc_J5, dAcc_J6,
        ref dTorq_J1, ref dTorq_J2, ref dTorq_J3, ref dTorq_J4, ref dTorq_J5, ref dTorq_J6);
```

3.8.18.HRIF_SetBaseInstallingAngle

3.8.18.1. Description: Sets the base installation angle.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nRotation	Base rotation angle	int	-360~360	Set base rotation angle
nTilt	Base tilt angle	int	-360~360	Set base tilt angle

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.18.2. example

```
// Define base rotation angle
int nRotation= 0;
// Define base tilt angle
int nTilt= 0;
// Set base installation angle
int nRet = hr_api.HRIF_SetBaseInstallingAngle(0,0, nRotation,nTilt);
```

3.8.19.HRIF_GetBaseInstallingAngle

3.8.19.1. Description: Reads the base installation angle.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nRotation	Base rotation	int	-360~360	Read base rotation angle

	angle			
nTilt	Base tilt angle	int	-360~360	Read base tilt angle

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.19.2. example

```
// Define base rotation angle
int nRotation= 0;
// Define base tilt angle
int nTilt= 0;
// Set base installation angle
int nRet = hr_api.HRIF_GetBaseInstallingAngle(0,0,ref nRotation,ref nTilt);
```

3.8.20.HRIF_SetDftTCP

3.8.20.1. Description: Sets the current default TCP..

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Name	TCP name	String		Tcp name

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.20.2. example

```
string name="TCP";
int nNet=hr_api.HRIF_SetDftTCP(0,0,name);
```

3.8.21.HRIF_ReadActCoord

3.8.21.1. Description: Reads the base installation angle.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
TcpName	TCP name	String		Tcp name
UcsName	Ucs name	String		Ucs name

✓ output variable

output variable	name	type of data	Effective range	content
J1~J6	Joint coordinates	double	~	Current joint coordinates
dX-dRz	Coordinates	double	~	Coordinates under the specified Ucs and Tcp: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinate, unit [°] dTcp_Ry: Ry coordinate, unit [°] dTcp_Rz: Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.21.2. example

```
// Define TCP name
string TcpName="TCP";
// Define UCS name
string UcsName="Base";
// Define joint variables
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// Define user coordinate variables
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// Obtain the current joint coordinate values and values in the user coordinate system
```

int nRet=hr_api.HRIF_ReadActCoord(0,0,TcpName,UcsName,ref dJ1,ref dJ2,ref dJ3,ref dJ4,ref dJ5,ref dJ6,ref dX,ref dY,ref dZ,ref dRx,ref dRy,ref dRz);

3.8.22.HRIF_HRIF_ReadActCoord_nJ

3.8.22.1. Description: Obtains the joint coordinate values and user coordinates of the robotic arm in its current state by inputting TCP and UCS names. Note: This is the interface used for the extended-axis version, but it is not supported in Series 7 versions.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
TcpName	TCP name	String		Tcp name
UcsName	Ucs name	String		Ucs name

✓ output variable

output variable	name	type of data	Effective range	content
Joints	Current joint coordinates	JointsData	-	Joints.dJoint[i]: Joint coordinate, unit [°]
dX-dRz	Coordinates	double	~	Coordinates under the specified Ucs and Tcp: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinate, unit [°] dTcp_Ry: Ry coordinate, unit [°] dTcp_Rz: Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.8.22.2. example

```
// Define TCP name
string TcpName="TCP";
// Define UCS name
string UcsName="Base";
// Joint position
```

```
JointsData joints = new JointsData();
joints.joint[0] = 0; joints.joint[1] = 0; joints.joint[2] = 0; joints.joint[3] = 0; joints.joint[4] = 0;
joints.joint[5] = 0; joints.joint[6] = 0; joints.joint[7] = 0; joints.joint[8] = 0; joints.joint[9] = 0;
// Define user coordinate variables
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// Obtain the current joint coordinate values and values in the user coordinate system
int nRet=hr_api.HRIF_ReadActCoord_nJ(0,0,TcpName,UcsName,joints,dX,dY,dZ,dRx,dRy,dRz);
```

3.9. Force control command

3.9.1. HRIF_SetForceControlState

3.9.1.1. Description: Set the force control state , and the robot will jump to the motion state after executing the command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nState	Force control state	int	0/1-	Force control state 0: Disable force control 1: Turn on force control

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.1.2. example

```
// define setting state
int nState = 1;
// Set force control status
int nRet= hr_api.HRIF_SetForceControlState(0,0, nState);
```

3.9.2. HRIF_ReadForceControlState

3.9.2.1. Description: Read the current force control status.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
nState	Force control state	int	0 ~3	Force control status: 0: Off state 1: Turn on the power control to explore the state 2: force control to explore the completion status 3: Force control free drive state

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.2.2. example

```
// read force control status
int nState = 1;
// read force control status
int nRet= hr_api.HRIF_ReadForceControlState(0,0, ref nState);
```

3.9.3. HRIF_SetForceToolCoordinateMotion

3.9.3.1. Description: Set the direction of the force control coordinate system as Tool Coordinate direction mode.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nMode	model	int	0/1	Set the force control coordinate system as Tool direction: 0: off 1: open

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.3.2. example

```
// define setting state
int nMode = 1;
// Set the state of the force control coordinate system
int nRet= hr_api.HRIF_SetForceToolCoordinateMotion(0,0, nMode);
```

3.9.4. HRIF_ForceControlInterrupt

3.9.4.1. Description: Pause force control motion, only pause force control function, not pause motion and script.(This interface function has been disabled.)

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.4.2. example

```
// Set force control pause state
int nRet= hr_api.HRIF_ForceControlInterrupt(0,0);
```

3.9.5. HRIF_ForceControlContinue

3.9.5.1. Description: Continue to force control motion, only continue to force control motion functions, do not continue motion and scripts.(This interface function has been disabled.)

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.5.2. example

```
// Set force control pause state
int nRet = hr_api.HRIF_ForceControlContinue(0,0);
```

3.9.6. HRIF_SetForceZero

3.9.6.1. Description: The force control is cleared, and the force sensor is recalibrated on the basis of the original data.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.6.2. example

```
// Clear force control data
int nRet = hr_api.HRIF_SetForceZero(0,0);
```

3.9.7. HRIF_SetMaxSearchVelocities

3.9.7.1. Description: Set the maximum speed of force control exploration.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dMaxLinearVelocity	linear speed	double	>0	The maximum speed of the search line, unit [mm/s]
dMaxAngularVelocity	attitude angular velocity	double	>0	The maximum speed of the seeking attitude, unit [°/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.7.2. example

```
//Set force control to search for linear speed
double dMaxLinearVelocity = 10;
// Set the force control to find the attitude angular velocity
double dMaxAngularVelocity = 5;
// Set force control search speed
int nRet= hr_api.HRIF_SetMaxSearchVelocities(0,0, dMaxLinearVelocity, dMaxAngularVelocity);
```

3.9.8. HRIF_SetMaxSearchDistance

3.9.8.1. Description: Sets the maximum search distance for force control in each degree of freedom.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

Dis_X	X-direction search boundary	double	10~1000	X-direction search boundary, unit: mm
Dis_Y	Y-direction search boundary	double	10~1000	Y-direction search boundary, unit: mm
Dis_Z	Z-direction search boundary	double	10~1000	Z-direction search boundary, unit: mm
Dis_RX	RX-direction search boundary	double	1~360	RX-direction search boundary, unit: °
Dis_RY	RY-direction search boundary	double	1~360	RY-direction search boundary, unit: °
Dis_RZ	RZ-direction search boundary	double	1~360	RZ-direction search boundary, unit: °

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.8.2. example

```
// Define the maximum search distance for force control in each degree of freedom
double Dis_X = 300; double Dis_Y = 300; double Dis_Z = 300;
double Dis_RX = 20; double Dis_RY = 20; double Dis_RZ = 20;
// Set the maximum search distance for force control in each degree of freedom
int nRet = hr_api.HRIF_SetMaxSearchDistance(0,0,Dis_X,Dis_Y,Dis_Z,Dis_RX,Dis_RY,Dis_RZ);
```

3.9.9. HRIF_ReadFTMotionFreedom

3.9.9.1. Description: Reads the end-effector degrees of freedom for force-controlled free drive.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

input variable	name	type of data	Effective range	content
nX-nRz	Degrees of freedom in each direction	int	0/1	Search degree of freedom switch in each direction: 0: Off 1: On

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.9.2. example

```
int nX; int nY; int nZ;
int nRx; int nRy; int nRz;
// // Read the end-effector degrees of freedom for force-controlled free drive
int nRet = hr_api.HRIF_ReadFTMotionFreedom(0,0,nX,nY,nZ,nRx,nRy,nRz);
```

3.9.10.HRIF_SetControlFreedom

3.9.10.1. Description: Set the degree of freedom for force control exploration.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nX-nRz	degrees of freedom in all directions	int	0/1	Search degrees of freedom switch in each direction: 0: off 1: open

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.10.2. example

```
// define force control degree of freedom state
int nX = 0; int nY = 0; int nZ = 1;
int nRx = 0; int nRy = 0; int nRz = 0;
// Set force control degree of freedom state
int nRet= hr_api.HRIF_SetControlFreedom(0,0, nX, nY, nZ, nRx, nRy, nRz);
```

3.9.11. HRIF_SetForceControlStrategy

3.9.11.1. Description: Set the force control strategy.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nState	Control Strategy	int	0~2	Force control strategy: 0: constant force mode 1: soft mode 2: Soft obstacle mode

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.11.2. example

```
// define force control strategy
int nState = 0;
// Set the force control strategy to constant force mode
int nRet= hr_api.HRIF_SetForceControlStrategy(0,0, nState);
```

3.9.12.HRIF_SetFreeDrivePositionAndOrientation

3.9.12.1. Description: Set the installation position and attitude of the force sensor center relative to the flange.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	Cartesian coordinates	double	-	The installation position and attitude of the force sensor relative to the flange: dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.12.2. example

```
// Define the installation position and attitude of the force sensor
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// Set the installation position and attitude of the force sensor
int nRet= hr_api.HRIF_SetFreeDrivePositionAndOrientation(0,0, dX, dY, dZ, dRx, dRy, dRz);
```

3.9.13.HRIF_SetPIDControlParams

3.9.13.1. Description: Set the force control search PID parameters.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
f	PID parameter	double	-	PID parameter Fp
dFi	PID parameter	double	-	PID parameter Fi
f	PID parameter	double	-	PID parameter Fd
dT	PID parameter	double	-	PID parameter Tp
dT	PID parameter	double	-	PID parameter Ti
wxya	PID parameter	double	-	PID parameter Td

✓ return value

return value	**name**	**type of data**	**Effective range**	**content**
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.13.2. example

```
// set PID parameters
double dFp = 1.0; double dFi = 0.1; double dFd = 0;
double dTp = 1.0; double dTi = 0.1; double dTd = 0;
// set PID parameters
int nRet= hr_api.HRIF_SetPIDControlParams(0,0, dFp, dFi, dFd, dTp, dTi, dTd);
```

3.9.14.HRIF_SetMassParams

3.9.14.1. Description: Set the inertia control parameters.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	Inertia Control Parameters	double	-	Inertia control parameters: dX: X direction dY: Y direction dZ: Z direction dRx: Rx direction dRy: Ry direction dRz: Rz direction

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.14.2. example

```
// set the quality parameters
double dX = 40; double dY = 40; double dZ = 40;
double dRx = 10; double dRy = 10; double dRz = 10;
// set the quality parameters
int nRet= hr_api.HRIF_SetMassParams(0,0, dX, dY, dZ, dRx, dRy, dRz);
```

3.9.15.HRIF_SetDampParams

3.9.15.1. Description: Sets damping (b) control parameters.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	Damping Control Parameters	double	-	Damping control parameters: dX: X direction dY: Y direction dZ: Z direction dRx: Rx direction dRy: Ry direction dRz: Rz direction

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.15.2. example

```
// set damping parameters
double dX = 800; double dY = 800; double dZ = 800;
double dRx = 40; double dRy = 40; double dRz = 40;
// set damping parameters
int nRet= hr_api.HRIF_SetDampParams(0,0, dX, dY, dZ, dRx, dRy, dRz);
```

3.9.16.HRIF_SetStiffParams

3.9.16.1. Description: Set the stiffness (k) control parameter.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	Stiffness Control Parameters	double	-	Stiffness control parameters: dX: X direction dY: Y direction dZ: Z direction dRx: Rx direction dRy: Ry direction dRz: Rz direction

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.16.2. example

```
// set the stiffness parameter
double dX = 1000; double dY = 1000; double dZ = 1000;
double dRx = 100; double dRy = 100; double dRz = 100;
// set the stiffness parameter
int nRet= hr_api.HRIF_SetStiffParams(0,0, dX, dY, dZ, dRx, dRy, dRz);
```

3.9.17.HRIF_SetForceControlGoal

3.9.17.1. Description: Set the force control target force.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	force control target force	double	-	Force control target force: dX: X direction, unit [N] dY: Y direction, unit [N] dZ: Z direction, unit [N] dRx: Rx direction, unit [NM] dRy: Ry direction, unit [NM] dRz: Rz direction, unit [NM]
dDistance1-dDistance6	Force Control Target Distance	double	-	Not enabled yet, default: 0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.17.2. example

```
// Set force control target force
double dX = 0; double dY = 0; double dZ = 10;
double dRx = 0; double dRy = 0; double dRz = 0;
// Set force control distance
double dDistance1 = 0; double dDistance2 = 0; double dDistance3 = 0;
double dDistance4 = 0; double dDistance5 = 0; double dDistance6 = 0;
// Set the force control target force to 10N in the Z direction
int nRet= hr_api.HRIF_SetForceControlGoal(0,0, dX, dY, dZ, dRx, dRy, dRz, dDistance1, dDistance2, dDistance3, dDistance4, dDistance5, dDistance6);
```

3.9.18.HRIF_SetControlGoal

3.9.18.1. Description: Set the force control target force and target distance (the force control target distance is not enabled yet).

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dWrench_X-dWrench_Rz	force control target force	double	-	Force control target force: dWrenchX: X direction, unit [N] dWrenchY: Y direction, unit [N] dWrenchZ: Z direction, unit [N] dWrenchRx: Rx direction, unit [NM] dWrenchRy: Ry direction, unit [NM] dWrenchRz: Rz direction, unit [NM]
dDistance_X-dDistance_Rz	Force Control Target Distance	double	-	Force control target distance: dDistanceX: X direction, unit [N] dDistanceY: Y direction, unit [N] dDistanceZ: Z direction, unit [N] dDistanceRx: RX direction, unit [NM] dDistanceRy: RY direction, unit [NM] dDistanceRz: RZ direction, unit [NM] Not enabled yet, default: 0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.18.2. example

// Set force control target force

double dWrench_X = 0; double dWrench_Y = 0; double dWrench_Z = 10;

double dWrench_Rx = 0; double dWrench_Ry = 0; double dWrench_Rz = 0;

// Set force control target distance

double dDistance_X = 0; double dDistance_Y = 0; double dDistance_Z = 10;

double dDistance_Rx = 0; double dDistance_Ry = 0; double dDistance_Rz = 0;

// Set the force control target force to 10N in the Z direction

int nRet= hr_api.HRIF_SetControlGoal(0,0, dWrench_X, dWrench_Y, dWrench_Z, dWrench_Rx, dWrench_Ry, dWrench_Rz, dDistance_X, dDistance_Y, dDistance_Z, dDistance_Rx, dDistance_Ry, dDistance_Rz);

3.9.19.HRIF_SetForceDataLimit

3.9.19.1. Description: Set the force control limit range, the controller will be powered off after the force sensor exceeds this range.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dMax_X-dMax_Rz	Maximum force range	double	-	Maximum force range: dMax_X: X direction, unit [N] dMax_Y: Y direction, unit [N] dMax_Z: Z direction, unit [N] dMax_Rx: Rx direction, unit [NM] dMax_Ry: Ry direction, unit [NM] dMax_Rz: Rz direction, unit [NM]
dMin_X-dMin_Rz	Minimum force range	double	-	Force minimum range: dMin_X: X direction, unit [N] dMin_Y: Y direction, unit [N] dMin_Z: Z direction, unit [N] dMin_Rx: Rx direction, unit [NM] dMin_Ry: Ry direction, unit [NM] dMin_Rz: Rz direction, unit [NM]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.19.2. example

```
// set force max
double dMax_X = 500; double dMax_Y = 500; double dMax_Z = 500;
double dMax_Rx = 500; double dMax_Ry = 500; double dMax_Rz = 500;
// set minimum force
double dMin_X = -500; double dMin_Y = -500; double dMin_Z = -500;
double dMin_Rx = -500; double dMin_Ry = -500; double dMin_Rz = -500;
// Set the force sensor data limit range
int nRet= hr_api.HRIF_SetForceDataLimit(0,0, dMax_X, dMax_Y, dMax_Z, dMax_Rx, dMax_Ry, dMax_Rz, dMin_X, dMin_Y, dMin_Z, dMin_Rx, dMin_Ry, dMin_Rz);
```

3.9.20.HRIF_SetForceDistanceLimit

3.9.20.1. Description: Set the force control deformation range.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dAllowDistance	Maximum distance allowed	double		Allowable maximum deformation distance, unit [mm]
dStrengthLevel	The power term of the deviation distance between the position and the boundary setting	double	2/3	The power term of the distance from the position to the boundary setting: 2: Resistance is proportional to the squared term from the boundary; 3: Resistance is proportional to the cubic term off bounds;

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.20.2. example

```
// Set the maximum distance allowed
double dAllowDistance = 10;
// Set the power item of the distance between the position and the boundary setting
double dStrengthLevel = 2;
// Set force control deformation range
int nRet= hr_api.HRIF_SetForceDistanceLimit(0,0, dAllowDistance, dStrengthLevel);
```

3.9.21.HRIF_SetForceFreeDriveMode

3.9.21.1. Description: Set to enable or disable force control free drive mode.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nMode	whether to open	int	0/1	0: off 1: open

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.21.2. example

```
// Set to enable force control free drive
int nMode = 1;
// Set to enable force control free drive
int nRet = hr_api.HRIF_SetForceFreeDriveMode(0, 0, nMode);
```

3.9.22.HRIF_ReadFTFreeDriveSpeedMode

3.9.22.1. Description: Read the force-controlled free-drive speed mode.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0

✓ output variable

nMode	Current speed mode	int	0~3	0: Default mode 1: Slow mode 2: Compliance mode 3: Welding mode

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer ≥ 0	nRet = 0: Function call successfulnRet > 0: Error code indicating call failure

3.9.22.2. example

```
// Free-drive speed mode
int nState = 1;
// Successful return
int nRet = hr_api.HRIF_ReadFTFreeDriveSpeedMode(0, 0, ref nState);
```

3.9.23.HRIF_SetFTFreeDriveSpeedMode

3.9.23.1. Description: Set the force-controlled free-drive speed mode.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0
nMode	Current speed mode	int	0~3	0: Default mode 1: Slow mode 2: Compliance mode 3: Welding mode

✓ return value

return value	name	type of data	Effective	content

			range	
nRet	Return value	int	Integer ≥ 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.23.2. example

```
// Set free-drive speed mode
int nMode = 1;
// Successful return
int nRet = hr_api.HRIF_SetFTFreeDriveSpeedMode(0, 0, nMode);
```

3.9.24.HRIF_ReadFTCabData

3.9.24.1. Description: Read calibrated force sensor data.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0

✓ output variable

output variable	name	type of data	Effective range	content
dX - dRz	Calibrated force sensor data	double	-	dX: X coordinate, unit [N] dY: Y coordinate, unit [N] dZ: Z coordinate, unit [N] dRx: Rx coordinate, unit [NM dRy: Ry coordinate, unit [NM] dRz: Rz coordinate, unit [NM]

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer ≥ 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.24.2. example

```
// Read force sensor data
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// Read force sensor data
int nRet= hr_api.HRIF_ReadFTCabData(0,0, ref dX, ref dY, ref dZ, ref dRx, ref dRy, ref dRz);
```

3.9.25.HRIF_SetFreeDriveMotionFreedom

3.9.25.1. Description: Set the end-effector degrees of freedom for force-controlled free-drive motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0
nX - nRz	Degrees of	int	0/1	Degree-of-freedom enable switch for each

	freedom in each direction			direction:0: Disabled1: Enabled

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer > 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.25.2. example

```
// Define force-controlled free-drive degrees of freedom states
int nX = 0; int nY = 0; int nZ = 1;
int nRx = 0; int nRy = 0; int nRz = 0;
// Set force-controlled free-drive degrees of freedom states
int nRet = hr_api. HRIF_SetFreeDriveMotionFreedom(0,0, nX, nY, nZ, nRx, nRy, nRz);
```

3.9.26.HRIF_SetFTFreeFactor

3.9.26.1. Description: Set translational compliance and rotational compliance.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0
dLinear	Translational compliance	double	0~100	Translational compliance for force-controlled free-drive
dAngular	Rotational compliance	double	0~100	Rotational compliance for force-controlled free-drive

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer > 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.26.2. example

```
// Define translational compliance for force-controlled free-drive
double dLinear = 50;
// Define rotational compliance for force-controlled free-drive
double dAngular = 50;
// Set translational and rotational compliance for force-controlled free-drive
int nRet =    hr_api. HRIF_SetFTFreeFactor(0,0, dLinear, dAngular);
```

3.9.27.HRIF_SetTangentForceBounds

3.9.27.1. Description: Set the maximum and minimum tangential force in the X/Y directions and the maximum lifting speed.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0
dMax	Minimum obstacle-crossing force	double	1~100	Minimum force in the X/Y directions to trigger obstacle crossing, unit: N
dMin	Deprecated	double	1~100	Deprecated parameter
dVel	Lifting speed	double	5~50	Maximum lifting speed, unit: mm/s

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer > 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.27.2. example

```
// Define the maximum tangential force in the X/Y directions
double dMax = 25;
// Define the minimum tangential force in the X/Y directions
double dMin = 15;
// Define the maximum lifting speed for obstacle crossing
double dVel = 30;
// Set the maximum and minimum tangential force in the X/Y directions and the maximum lifting speed
int nRet=    hr_api. HRIF_SetTangentForceBounds(0,0, dMax, dMin, dVel);
```

3.9.28.HRIF_SetFreeDriveCompensateForce

3.9.28.1. Description: Set the magnitude and vector direction of the directional compensation force in FreeDrive mode.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0

rbtID	Robot ID	int	0~5	Robot ID number, default = 0
dForce	Compensation force	double	0~500	Magnitude of the compensation force, unit: N
dVector	Force direction vector	double	-	Vector direction of the compensation force [x, y, z]

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer > 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.28.2. example

```
// Define the magnitude of the compensation force
double dForce= 25;
// Define the direction of the compensation force
double[] dVector = new double[] { 0.2, 0.4, 0.8 };
// Set the magnitude and vector direction of the directional compensation force in FreeDrive mode
int nRet = hr_api.HRIF_SetFreeDriveCompensateForce(0, 0, dForce, dVector);
```

3.9.29.HRIF_SetDepthThresholdForDampingArea

3.9.29.1. Description: Set the distance threshold at which the virtual wall starts to generate damping.

✓ **input variable**

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0
dDepth	Threshold value	double	-	Threshold distance that triggers the virtual wall, unit: mm

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer > 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.29.2. example

```
// Define the threshold value
double dDepth = 100;
// Set the distance threshold at which the virtual wall starts to generate damping
```

int nRet = hr_api.HRIF_SetDepthThresholdForDampingArea(0, 0, dDepth);

3.9.30.HRIF_AddSafePlane

3.9.30.1. Description: Add a virtual wall plane. The virtual wall plane is a safety plane; the functions differ accordingly. If the name of the added safety plane already exists, error 20006 will be reported. The last parameter indicates whether to activate it; if not activated, UCS existence will not be checked.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0
Name	Plane name	string	—	Name of the plane to be added
UcsName	User coordinate name	string	—	The user coordinate system to be used
Mode	Safety mode	int	0/1	Normal / Reduced mode
Display	Display	int	0/1	Whether to display the target plane
Switch	Enable	int	0/1	Whether to enable the target plane

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer > 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.30.2. example

```
// Define the plane name
string Name = "planel";
// Define the user coordinate system name
string UcsName = "Plane_1";
// Define the safety mode
int Mode = 0;
// Define whether to display
int Display = 0;
// Define whether to enable
int Switch = 1;
// Add the virtual wall plane
int nRet = hr_api.HRIF_AddSafePlane(0, 0, Name, UcsName , Mode, Display, Switch);
```

3.9.31.HRIF_UpdateSafePlane

3.9.31.1. Description: Modify and update the attributes of a virtual wall plane. The virtual wall plane is a safety plane; the functions differ accordingly. The last parameter indicates whether to activate it; if not activated, UCS existence will not be checked.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0
Name	Plane name	string	—	Name of the plane to be updated
UcsName	User coordinate name	string	—	The user coordinate system to be used
Mode	Safety mode	int	0/1	Normal / Reduced mode
Display	Display	int	0/1	Whether to display the target plane
Switch	Enable	int	0/1	Whether to enable the target plane

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer > 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.31.2. example

```
// Define the plane name
string Name = "planel";
//Define the user coordinate system name
string UcsName = "Plane_1";
//Define the safety mode
int Mode = 0;
//Define whether to display
int Display = 0;
//Define whether to enable
int Switch = 1;
// Update the virtual wall plane
int nRet = hr_api.HRIF_UpdateSafePlane(0, 0, Name, UcsName , Mode, Display, Switch);
```

3.9.32.HRIF_DelSafePlane

3.9.32.1. Description: Delete a virtual wall plane. The virtual wall plane is a safety plane; the functions differ accordingly. If the specified safety plane name does not exist, error 20006 will be reported.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	Electrical Box ID	int	0~5	Electrical Box ID number, default = 0
rbtID	Robot ID	int	0~5	Robot ID number, default = 0
Name	Plane name	string	—	Name of the plane to be deleted

✓ return value

return value	name	type of data	Effective range	content
nRet	Return value	int	Integer > 0	nRet = 0: Function call successful nRet > 0: Error code indicating call failure

3.9.32.2. example

```
// Define the plane name to be deleted
string Name = "planel";
//Delete the virtual wall plane
int nRet = hr_api.HRIF_DelSafePlane(0, 0, Name );
```

3.9.33.HRIF_ReadSafePlaneList

3.9.33.1. Description： Returns a list of all safety plane names. Virtual wall planes are safety planes with different functionalities.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
BordersName	All Virtual Wall Plane Names	List<string>	-	Virtual wall plane names

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.33.2. Example

```
// Define list to receive all virtual wall plane names
List<string> BordersName = new List<string>();
// Read all current virtual wall plane names
int nRet = hr_api.HRIF_ReadSafePlaneList(0, 0, ref BordersName );
```

3.9.34.HRIF_ReadSafePlane

3.9.34.1. Description：Returns detailed parameters of the specified safety plane. Virtual wall planes are safety planes with different functionalities. Returns empty if the specified safety plane does not exist.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Name	Plane Name	string	-	Name of the plane to read information from

✓ output variable

output variable	name	type of data	Effective range	content
UcsName	User Coordinate Name	string	-	User coordinate system used
Mode	Safety Mode	int	0/1	Normal/Reduced mode
Display	Display	int	0/1	Whether to display the target plane
Switch	Enable	int	0/1	Whether to enable the target plane

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.34.2. Example

```
// Define plane name to read
string BorderName = "planel";
// Define variables to receive target plane information
string UcsName = "";
int Mode = 0;
int Display = 0;
int Switch = 0;
// Read detailed information of target virtual wall plane
int nRet = hr_api.HRIF_ReadSafePlane(0, 0, BorderName, ref UcsName, ref Mode, ref Display, ref Switch);
```

3.9.35.HRIF_SetFTWrenchThresholds

3.9.35.1. Description：Sets the force control free drive activation thresholds (force and torque).

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dForceThreshold	Force	double	0~100	Free drive force activation threshold, unit [N]
dTorqueThreshold	Torque	double	0~10	Free drive torque activation threshold, unit [N·m]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.35.2. Example

```
// Define force threshold
double dForceThreshold;
// Define torque threshold
double dTorqueThreshold;
// Set force control free drive activation thresholds (force and torque)
int nRet = hr_api.HRIF_SetFTWrenchThresholds(0,0, dForceThreshold, dTorqueThreshold);
```

3.9.36.HRIF_SetMaxFreeDriveVel

3.9.36.1. Description：Sets the maximum linear velocity and attitude angular velocity for force control free drive.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dMaxLinearVelocity	Linear Velocity	double	1~500	Maximum linear velocity for free drive, unit [mm/s]
dMaxAngularVelocity	Angular Velocity	double	1~180	Maximum attitude angular velocity for free drive, unit [°/s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.36.2. Example

```
// Define maximum free drive linear velocity
double dMaxLinearVelocity = 250;
// Define maximum free drive angular velocity
double dMaxAngularVelocity = 20;
// Set maximum linear velocity and attitude angular velocity for force control free drive
int nRet = hr_api.HRIF_SetMaxFreeDriveVel(0,0, dMaxLinearVelocity, dMaxAngularVelocity);
```

3.9.37.HRIF_SetSteadyContactDeviationRange

3.9.37.1. Description：Sets the maximum linear velocity and attitude angular velocity for force control free drive.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Pos_X~Pos_Z	Positive Direction Distance Boundary	double	10~1000	Positive direction boundary (consistent with the probing direction), unit: mm
Pos_RX~Pos_RZ	Positive Direction Angle Boundary	double	1~360	Positive direction boundary (consistent with the probing direction), unit: °
Neg_X ~ Neg_Z	Negative Direction Distance Boundary	double	-1000~-10	Negative direction boundary (opposite to the probing direction), unit: mm
Neg_RX ~ Neg_RZ	Negative Direction Angle Boundary	double	-360~-1	Negative direction boundary (opposite to the probing direction), unit: °

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.37.2. 示例

```
double Pos_X = 100; double Pos_Y = 100; double Pos_Z = 100;
double Pos_RX = 20; double Pos_RY = 20; double Pos_RZ = 20;

double Neg_X = -100; double Neg_Y = -100; double Neg_Z = -100;
double Neg_RX = -20; double Neg_RY = -20; double Neg_RZ = -20;
// Set Steady Contact Deviation Range
int nRet = hr_api.HRIF_SetSteadyContactDeviationRange(0,0,Pos_X,Pos_Y,Pos_Z,Pos_RX,Pos_RY,Pos_RZ,
Neg_X,Neg_Y,Neg_Z,Neg_RX,Neg_RY,Neg_RZ);
```

3.9.38.HRIF_SetFTCalibration

3.9.38.1. Description：Calculates the force control calibration results.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
pointnum	Calibration Point Count	int	8~16	Number of calibration points, 8~16 points
Position	Calibrated Position Data	List<Pose>	-	Calibrated Position Data
Force	Calibrated Force Data	List<Pose>	-	Calibrated Force Data

✓ return value

return value	Name	Data Type	Valid Range	Description
nRet	return value	int	Integer value >= 0	nRet = 0: Function call successful nRet >0: Error code indicating call failure

3.9.38.2. Example

```
// Set number of points
int pointnum=8;
List<Pose> Force = new List<Pose>();
List<Pose> Position = new List<Pose>();
for(int i = 0;i<pointnum;i++)
{
    position[i].dX =0;
    position[i].dY =0;
    position[i].dZ =0;
    position[i].dRX =0;
    position[i].dRY =0;
    position[i].dRZ =0;
```

```
        force[i].dX =0;
        force[i].dY =0;
        force[i].dZ =0;
        force[i].dRX =0;
        force[i].dRY =0;
        force[i].dRZ =0;
}
// Set calibrated data
int nRet = hr_api.HRIF_SetFTCalibration(0,0,pointnum,Position,Force);
```

3.9.39.HRIF_ReadForceData

3.9.39.1. Description：Reads the raw force sensor data.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
forcedata	Raw Force Data	double	-	Raw force data provided by force sensor: dX: Raw force data in x-direction dY: Raw force data in y-direction dZ: Raw force data in z-direction dRX: Raw force data in RX-direction dRY: Raw force data in RY-direction dRZ: Raw force data in RZ-direction

✓ return value

return value	Name	Data Type	Valid Range	Description
nRet	return value	int	Integer value >= 0	nRet = 0: Function call successful nRet >0: Error code indicating call failure

3.9.39.2. Example

```
Pose force = new Pose();
// Read current raw force sensor data
int nRet=hr_api.HRIF_ReadForceData(0, 0, ref force);
```

3.9.40.HRIF_SetInitializeForceSensor

3.9.40.1. Description：Sets the force control calibration results.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Fx0-Fz0	Calibrated Force	double	-	Calibrated force in x-z directions
Mx0-Mz0	Moment/Torque	double	-	Moment/Torque in x-z directions
X-Z	Center of Mass Offset	double	-	Center of mass offset in x-z directions
G	Gravity	double	-	Gravity
InstallRotationAngel	Installation Angle	double	-	Machine installation angle

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.40.2. Example

```
// Set calibrated force in x, y, z directions
double Fx0 = 1;
double Fy0 =2 ;
double Fz0 = 3;
// Set moment/torque in x, y, z directions
double Mx0 =4 ;
double My0 =5 ;
double Mz0 = 6;
// Set gravity
double G = 7;
// Set center of mass offset in x, y, z directions
double X = 8;
double Y = 9;
double Z =10 ;
// Set installation angle
```

double InstallRotationAngel =11 ;

int Ret = hr_api.HRIF_SetInitializeForceSensor(0, 0, Fx0,Fy0,Fz0,Mx0,My0,Mz0,G,X,Y,Z,InstallRotationAngel);

3.9.41.HRIF_GetLastCalibParams

3.9.41.1. Description：Gets the force control calibration results.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

Output Variables	Name	Data Type	Valid Range	Description
Fx0-Fz0	Calibrated Force	double	-	Calibrated force in x-z directions
Mx0-Mz0	Moment/Torque	double	-	Moment/Torque in x-z directions
X-Z	Center of Mass Offset	double	-	Center of mass offset in x-z directions
G	Gravity	double	-	Gravity
InstallRotationAngel	Installation Angle	double	-	Machine installation angle

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.41.2. Example

```
// Get calibrated force in x, y, z directions
double Fx0 = 1;
double Fy0 =2 ;
double Fz0 = 3;
// Get moment/torque in x, y, z directions
double Mx0 =4 ;
double My0 =5 ;
double Mz0 = 6;
// Get gravity
double G = 7;
// Get center of mass offset in x, y, z directions
double X = 8;
double Y = 9;
```

```
double Z =10 ;
// Get installation angle
double InstallRotationAngel =11 ;
int Ret = hr_api.HRIF_GetLastCalibParams(0,0,Fx0,Fy0,Fz0,Mx0,My0,Mz0,G,X,Y,Z,InstallRotationAngel);
```

3.9.42.HRIF_SetFTUCS

3.9.42.1. Description： Sets the force control user coordinate without writing to the configuration file. Returns to default base after restart.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dUcs_X-dUcs_Rz	User Coordinates	double	-	User coordinates to be set: dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinate, unit [°] dUcs_Ry: Ry coordinate, unit [°] dUcs_Rz: Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.42.2. Example

```
// Define force control user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Set force control user coordinates
int nRet= hr_api.HRIF_SetFTUCS(0,0, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.9.43.HRIF_ReadFTUCS

3.9.43.1. Description：Reads the force control user coordinate values.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
dUcs_X-dUcs_Rz	User Coordinates	double	-	Retrieved user coordinates: dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinate, unit [°] dUcs_Ry: Ry coordinate, unit [°] dUcs_Rz: Rz coordinate, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.9.43.2. Example

```
// Define force control user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// Read force control user coordinatesint
int nRet= hr_api.HRIF_ReadFTUCS(0,0, sUcsName , ref dUcs_X, ref dUcs_Y, ref dUcs_Z, ref dUcs_Rx,
ref dUcs_Ry, ref dUcs_Rz);
```

3.10. General Motion Control Instructions

3.10.1.HRIF_MoveRelJ

3.10.1.1. Description: Relative movement of joints.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nAxis ID	Axis ID	int	0~5	The target axis ID of the movement, corresponding to the joints J1-J6
nDirection	direction	int	0/1	0: negative direction 1: positive direction
dDistance	Movement distance	double	-	relative movement distance

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.1.2. example

```
//The target axis ID of the movement, corresponding to joints J1-J6
int nAxisId = 2;
//direction
int nDirection = 1;
//movement distance
double dDistance = 10;
// Joint relative movement
int nRet= hr_api.HRIF_MoveRelJ(0,0, nAxisId, nDirection, dDistance);
```

3.10.2.HRIF_MoveRelL

3.10.2.1. Relative movement in space.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nAxis ID	Axis ID	int	0~5	The target axis ID of the movement, corresponding to the space coordinate X-Rz
nDirection	direction	int	0/1	0: negative direction 1: positive direction
dDistance	Movement distance	double	-	relative movement distance
nToolMotion	Motion coordinate type	int	0/1	0: Move according to the currently selected user coordinates 1: Press Tool coordinate movement

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.2.2. example

```
//The target axis ID of the movement, corresponding to the space coordinate X-Rz
int nAxisId = 2;
//direction
int nDirection = 1;
//movement distance
double dDistance = 10;
//Motion coordinate type
int nToolMotion = 0;
//Spatial relative movement
int nRet= hr_api.HRIF_MoveRelL(0,0, nAxisId, nDirection, dDistance, nToolMotion);
```

3.10.3.HRIF_WayPointRel

3.10.3.1. Description: Relative movement of waypoints.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nType	type of exercise	int	0/1	0: joint relative movement 1: Linear relative motion
nPointList	Whether to use point list points	int	0/1	0: Do not use the point in the point list 1: Use the point in the point list
dPos_X - dPos_Rz _	Spatial location	double		nPointList = 1 : point space position d Pos_ X: X coordinate, unit [mm] d Pos_ Y: Y coordinate, unit [mm] d Pos_ Z: Z coordinate, unit [mm] d Pos_ Rx: Rx coordinates, unit [°] d Pos_ Ry: Ry coordinates, unit [°] d Pos_ Rz: Rz coordinates, unit [°] nPointList = 0 , Both are 0
dPos_ J 1 - dPos_ J 6	joint position	double		nPointList = 1 : point joint position d Pos_ J1: joint 1 coordinates, unit [°] d Pos_ J2: joint 2 coordinates, unit [°] d Pos_ J3: Joint 3 coordinates, unit [°] d Pos_ J4: Joint 4 coordinates, unit [°] d Pos_ J5: Joint 5 coordinates, unit [°] d Pos_ J6: Joint 6 coordinates, unit [°] nPointList = 0 , Both are 0
nrelMoveType	relative motion type	int	0/1	0: absolute value 1: Overlay value
nAxisMask_1- nAxisMask_6	Whether to exercise	int	0/1	Whether each axis/direction moves 0 : no motion 1: Sports
dTarget_1-dTarget_ 6	Movement distance	double	-	nType = 0 and nAxisMask = 1 : the absolute distance or superimposed distance of movement in this direction nType = 1 and nAxisMask = 1 : the axis moves the absolute distance or the

				superimposed distance nAxisMask = 0 : no effect
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name " TCP " can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name " Base " can be used
dVelocity	speed	double	-	Maximum movement speed, unit [°/s] during joint movement, X, Y, Z during space movement Unit [mm/s], Rx, Ry, Rz Unit [°/s]
dAcc	acceleration	double	-	Maximum acceleration of movement, unit [°/s ^{2}] during joint movement, X, Y, Z during space movement Unit [mm/s ^{2}], Rx, Ry, Rz Unit [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nIsUseJoint	Whether to use joint coordinates	int	0	The default is 0 (other values are invalid), use the joint angle
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection of DI stop is turned on. During the waypoint movement, if the state of DI indexed by the nIOBit bit of the electric box = nIOState, the robot stops moving, otherwise it moves to the target point to complete the movement
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	Detected DI status	int	0/1	Checked DI status, invalid when nIsSeek= 0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to " 1 " , "2" , " 3 " in sequence.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.3.2. example

```
//Motion type
```

```
int nType = 0;
//Whether to use point list point
int nPointList = 1;
//Spatial location
double dPos_X = 420; double dPos_Y = 0; double dPos_Z = 445;
double dPos_Rx = 180; double dPos_Ry = 0; double dPos_Rz = 180;
//joint position
double dPos_J1 = 0; double dPos_J2 = 0; double dPos_J3 = 90;
double dPos_J4 = 0; double dPos_J5 = 90; double dPos_J6 = 0;
//relative movement type
int nrelMoveType = 0;
//whether to exercise
int nAxisMask_1 = 0; int nAxisMask_2 = 0; int nAxisMask_3 = 1;
int nAxisMask_4 = 0; int nAxisMask_5 = 1; int nAxisMask_6 = 0;
//movement distance
double dTarget_1 = 0; double dTarget_2 = 0; double dTarget_3 = 0;
double dTarget_4 = 0; double dTarget_5 = 0; double dTarget_6 = 0;
//Tool coordinate name
string sTcpName = "TCP";
//user coordinate name
string sUcsName = "Base";
//speed
double dVelocity = 5;
//acceleration
double dAcc = 360;
// transition radius
double dRadius = 50;
//Whether to use joint coordinates
int nIsUseJoint = 0;
//Whether to detect DI stop
int nIsSeek = 1;
//Detected DI index
int nIOBit = 0;
//Detected DI status
int nIOState = 1;
//command ID
string strCmdID = "1";
int nRet= hr_api.HRIF_WayPointRel(0,0, nType, nPointList, dPos_X, dPos_Y, dPos_Z, dPos_Rx, dPos_Ry, dPos_Rz,
dPos_J1, dPos_J2, dPos_J3, dPos_J4, dPos_J5, dPos_J6, nrelMoveType, nAx isMask_1, nAxisMask_2,
```

nAxisMask_3, nAxisMask_4, nAxisMask_5 , nAxisMask_6, dTarget_1, dTarget_2, dTarget_3, dTarget_4, dTarget_5, dTarget_6, sTcpName, sUcsName, dVelocity, dAcc, dRadius, nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);

3.10.4.HRIF_WayPointRel_nJ

3.10.4.1. Description: Relative movement of waypoints.Note: This interface is specifically for the extended-axis version.

✓ input variable

parameter	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nType	type of exercise	int	0/1	0: joint relative movement 1: Linear relative motion
nPointList	Whether to use point list points	int	0/1	0: Do not use the point in the point list 1: Use the point in the point list
pose	Spatial location	Pose		nPointList = 1 : point space position d Pos_ X: X coordinate, unit [mm] d Pos_ Y: Y coordinate, unit [mm] d Pos_ Z: Z coordinate, unit [mm] d Pos_ Rx: Rx coordinates, unit [°] d Pos_ Ry: Ry coordinates, unit [°] d Pos_ Rz: Rz coordinates, unit [°] nPointList = 0 , Both are 0
joints	joint position	JointData		nPointList = 1 : point joint position joints[i]: joint i coordinates, unit [°] nPointList = 0 , Both are 0
nrelMoveType	relative motion type	int	0/1	0: absolute value 1: Overlay value
Axis	Whether to exercise	JointData	0/1	Whether each axis/direction moves 0 : no motion 1: Sports
Target	Movement distance	Pose	-	nType = 0 and nAxisMask = 1 : the absolute distance or superimposed distance of movement in this direction nType = 1 and nAxisMask = 1 : the axis moves the absolute distance or the superimposed distance nAxisMask = 0 : no effect
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name " TCP " can be used

sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name " Base " can be used
dVelocity	speed	double	-	Maximum movement speed, unit [°/s] during joint movement, X, Y, Z during space movement Unit [mm/s], Rx, Ry, Rz Unit [°/s]
dAcc	acceleration	double	-	Maximum acceleration of movement, unit [°/s ^{2}] during joint movement, X, Y, Z during space movement Unit [mm/s ^{2}], Rx, Ry, Rz Unit [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nIsUseJoint	Whether to use joint coordinates	int	0	The default is 0 (other values are invalid), use the joint angle
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection of DI stop is turned on. During the waypoint movement, if the state of DI indexed by the nIOBit bit of the electric box = nIOState, the robot stops moving, otherwise it moves to the target point to complete the movement
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	Detected DI status	int	0/1	Checked DI status, invalid when nIsSeek= 0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to " 1 " , "2" , " 3 " in sequence.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.4.2. example

```
//Motion type
int nType = 0;
//Whether to use point list point
int nPointList = 1;
//Spatial location
Pose pose = new Pose();
pose.dX = 420; pose.dY = 0; pose.dZ = 443;
```

```
pose.dRX = 180; pose.dRY = 0; pose.dRZ = 180;
//joint position
JointsData joints = new JointsData();
joints.joint[0] = 0; joints.joint[1] = 0; joints.joint[2] = 90;
joints.joint[3] = 0; joints.joint[4] = 90; joints.joint[5] = 0; joints.joint[6] = 0;
//relative movement type
int nrelMoveType = 0;
//whether to exercise
JointsData Axis = new JointsData();
Axis.joint[0] = 0; Axis.joint[1] = 0; Axis.joint[2] = 1;
Axis.joint[3] = 0; Axis.joint[4] = 1; Axis.joint[5] = 0; Axis.joint[6] = 0;
//movement distance
Pose Target = new Pose();
Target.dX = 10; Target.dY = 0; Target.dZ = 0;
Target.dRX = 0; Target.dRY = 0; Target.dRZ = 0;
//Tool coordinate name
string sTcpName = "TCP";
//user coordinate name
string sUcsName = "Base";
//speed
double dVelocity = 5;
//acceleration
double dAcc = 360;
// transition radius
double dRadius = 50;
//Whether to use joint coordinates
int nIsUseJoint = 0;
//Whether to detect DI stop
int nIsSeek = 1;
//Detected DI index
int nIOBit = 0;
//Detected DI status
int nIOState = 1;
//command ID
string strCmdID = "1";
int nRet = hr_api.HRIF_WayPointRel_nJ(0, 0, nMoveType, nPointList,pose,joints,nrelMoveType, Axis,Target,
sTcpName, sUcsName, dVelocity, dAcc, dRadius, nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);
```

3.10.5.HRIF_IsMotionDone

3.10.5.1. Description: Determine whether the robot is in motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

output variable	name	type of data	Effective range	content
Done	return value	bool	false/true	false: the movement is not completed and is in motion true: motion complete, in ready state

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.5.2. example

```
// Determine whether the robot is in motion
bool bDone = false;
int nRet= hr_api.HRIF_IsMotionDone(0,0,ref bDone);
```

3.10.6.HRIF_IsBlendingDone

3.10.6.1. Description: Determine whether the movement of the waypoint is completed

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

* Note: This interface only judges whether the movement of the waypoint is completed, and returns true After that, the robot may still be in a moving state; for example, after enabling follow, force control, and relative tracking, the robot is in a moving state. When no waypoint is called, this interface returns true .

✓ output variable

output variable	name	type of data	Effective range	content
Done	return value	bool	false/true	false: the movement is not completed and is in motion true: motion complete, in ready state

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.6.2. example

```
// Determine whether the waypoint movement is complete
bool bDone = false;
int nRet= hr_api. HRIF_IsBlendingDone(0,0, ref bDone);
```

3.10.7.HRIF_WayPointEx

3.10.7.1. Description: Waypoint movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	space target position	double	-	nMoveType= 0 and nIsUseJoint= 1: invalid, nMoveType= 0 and nIsUseJoint= 0: Use this space coordinate as the target position, and the joint coordinates obtained through inverse calculation are the target joint coordinates nMoveType= 1: target space position dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]
dJ1-dJ6	joint target position	double	-	nMoveType= 0 and nIsUseJoint= 1: Use this joint coordinate as the target joint coordinate nMoveType= 0 and nIsUseJoint= 0: this joint coordinate is only used as the reference joint coordinate of the selected solution when calculating the inverse solution nMoveType= 1: invalid dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°] dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6: joint 6 coordinates, unit [°]
dTcp_X- dTcp_Rz	Tool coordinate value	double	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. When nIsUseJoint=1, it is invalid and can be set to 0: dTcp_X: X coordinate, unit [mm]

				dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
dUcs_X-dUcs_Rz	user coordinates	double	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. When nIsUseJoint=1, it is invalid and can be set to 0: dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz: Rz coordinates, unit [°]
dVelocity	speed	double	-	The maximum speed of movement, the unit of joint movement [°/s], the unit of XYZ [mm/s] of space movement, the unit of Rx, Ry, Rz [°/s]
dAcc	acceleration	double	-	The maximum acceleration of movement, the unit of joint movement [°/s ^{2}], the unit of XYZ [mm/s ^{2}] of space movement, the unit of Rx, Ry, Rz [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nMoveType	type of exercise	int	0/1	0: joint movement 1: Space movement
nIsUseJoint	Whether to use joint coordinates	int	0/1	Whether to use the joint angle as the target point, if nMoveType= 0, nIsUseJoint is valid: 0: Do not use joint angles 1: Use joint angles
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0-7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0

strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

* Note: H RIF_ WayPointEx It is necessary to set the specific values of tool coordinates and user coordinates; H RIF_ WayPoint Names of tool coordinates and user coordinates taught by the teach pendant.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.7.2. example

```
// define motion type
int nMoveType = 1;
// Define the spatial target position
double dX = 420; double dY = 0; double dZ = 445;
double dRx = 180; double dRy = 0; double dRz = 180;
// Define the joint target position
double dJ1 = 0; double dJ2 = 0; double dJ3 = 70;
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// Define whether to use joint angles
int nIsUseJoint = 0;
// Define whether to use detection DI to stop
int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
int nIOState = 0;
// Define the waypoint ID
```

```
string strCmdID = "0";
// Execute waypoint movement
int nRet = hr_api.HRIF_WayPointEx(0, 0, dX, dY, dZ, dRx, dRy, dRz, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6, dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz, dUcs_X, dUcs_Y , dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz, dVelocity, dAcc, dRadius, nMoveType, nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);
```

3.10.8.HRIF_WayPointEx_nJ

3.10.8.1. Description: Waypoint movement.Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
pose	space target position	Pose	-	nMoveType= 0 and nIsUseJoint= 1: invalid, nMoveType= 0 and nIsUseJoint= 0: Use this space coordinate as the target position, and the joint coordinates obtained through inverse calculation are the target joint coordinates nMoveType= 1: target space position dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]
joints	joint target position	JointData	-	nMoveType= 0 and nIsUseJoint= 1: Use this joint coordinate as the target joint coordinate nMoveType= 0 and nIsUseJoint= 0: this joint coordinate is only used as the reference joint coordinate of the selected solution when calculating the inverse solution nMoveType= 1: invalid joints[i]: Joint i coordinates, unit [°]
Tcp	Tool coordinate value	Pose	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. When nIsUseJoint=1, it is invalid and can be set to 0: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]

Ucs	user coordinates	Pose	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. When nIsUseJoint=1, it is invalid and can be set to 0: dUcs_X: X coordinate, unit [mm] dUcs_Y: Y coordinate, unit [mm] dUcs_Z: Z coordinate, unit [mm] dUcs_Rx: Rx coordinates, unit [°] dUcs_Ry: Ry coordinates, unit [°] dUcs_Rz: Rz coordinates, unit [°]
dVelocity	speed	double	-	The maximum speed of movement, the unit of joint movement [°/s], the unit of XYZ [mm/s] of space movement, the unit of Rx, Ry, Rz [°/s]
dAcc	acceleration	double	-	The maximum acceleration of movement, the unit of joint movement [°/s ^{2}], the unit of XYZ [mm/s ^{2}] of space movement, the unit of Rx, Ry, Rz [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nMoveType	type of exercise	int	0/1	0: joint movement 1: Space movement
nIsUseJoint	Whether to use joint coordinates	int	0/1	Whether to use the joint angle as the target point, if nMoveType= 0, nIsUseJoint is valid: 0: Do not use joint angles 1: Use joint angles
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0-7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

* Note: H RIF_ WayPointEx It is necessary to set the specific values of tool coordinates and user coordinates; H RIF_ WayPoint Names of tool coordinates and user coordinates taught by the teach pendant.

✓ return value

return value	name	type of data	Effective	content

			range	
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.8.2. example

```
// define motion type
int nMoveType = 1;
// Define the spatial target position
Pose pose = new Pose();
pose.dX = 420; pose.dY = 0; pose.dZ = 443;
pose.dRX = 180; pose.dRY = 0; pose.dRZ = 180;
// Define the joint target position
JointsData joints = new JointsData();
joints.joint[0] = 0; joints.joint[1] = 0; joints.joint[2] = 90;
joints.joint[3] = 0; joints.joint[4] = 90; joints.joint[5] = 0;
// define tool coordinate variables
Pose Tcp = new Pose();
Tcp.dX = 0; Tcp.dY = 0; Tcp.dZ = 0;
Tcp.dRX = 0; Tcp.dRY = 0; Tcp.dRZ = 0;
// define user coordinate variables
Pose Ucs = new Pose();
Ucs.dRX = 0; Ucs.dRY = 0; Ucs.dRZ = 0;
Ucs.dX = 0; Ucs.dY = 0; Ucs.dZ = 0;
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// Define whether to use joint angles
int nIsUseJoint = 0;
// Define whether to use detection DI to stop
int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
int nIOState = 0;
// Define the waypoint ID
string strCmdID = "0";
// Execute waypoint movement
```

int nRet = hr_api.HRIF_WayPointEx_nJ(0, 0,pose,joints,Tcp,Ucs,dVelocity, dAcc, dRadius, nMoveType, nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);

3.10.9.HRIF_WayPoint

3.10.9.1. Description: Waypoint movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	space target position	double	-	nMoveType= 0 and nIsUseJoint= 1: invalid, nMoveType= 0 and nIsUseJoint= 0: Use this space coordinate as the target position, and the joint coordinates obtained through inverse calculation are the target joint coordinates nMoveType= 1: target space position dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]
dJ1-dJ6	joint target position	double	-	nMoveType= 0 and nIsUseJoint= 1: Use this joint coordinate as the target joint coordinate nMoveType= 0 and nIsUseJoint= 0: this joint coordinate is only used as the reference joint coordinate of the selected solution when calculating the inverse solution nMoveType= 1: invalid dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°] dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6: joint 6 coordinates, unit [°]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page . It is invalid when nIsUseJoint=1, and the default name "TCP" can be used

sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used
dVelocity	speed	double	-	The maximum speed of movement, the unit of joint movement [°/s], the unit of XYZ [mm/s] of space movement, the unit of Rx, Ry, Rz [°/s]
dAcc	acceleration	double	-	The maximum acceleration of movement, the unit of joint movement [°/s ^{2}], the unit of XYZ [mm/s ^{2}] of space movement, the unit of Rx, Ry, Rz [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nMoveType	type of exercise	int	0/1	0: joint movement 1: linear motion
nIsUseJoint	Whether to use joint coordinates	int	0/1	Whether to use the joint angle as the target point, if nMoveType= 0, then nisJoint is valid: 0: Do not use joint angles 1: Use joint angles
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

* Note: H RIF_ WayPointEx It is necessary to set the specific values of tool coordinates and user coordinates; H RIF_ WayPoint Names of tool coordinates and user coordinates taught by the teach pendant.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.9.2. example

```
// define motion type
int nMoveType = 0;
```

```
// Define the spatial target position
double dX = 420; double dY = 0; double dZ = 445;
double dRx = 180; double dRy = 0; double dRz = 180;
// Define the joint target position
double dJ1 = 0; double dJ2 = 0; double dJ3 = 80;
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// Define whether to use joint angles
int nIsUseJoint = 0;
// Define whether to use detection DI to stop
int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
int nIOState = 0;
// Define the waypoint ID
string strCmdID = "0";
// Execute waypoint movement
int nRet = hr_api.HRIF_WayPoint(0, 0, dX, dY, dZ, dRx, dRy, dRz, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6, sTcpName, sUcsName, dVelocity, dAcc, dRadius, nMoveType, nIsUseJoint, nIsSeek , nIOBit, nIOState, strCmdID);
```

3.10.10. HRIF_WayPoint_nJ

3.10.10.1. Description: Waypoint movement.Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
pose	space target position	Pose	-	nMoveType= 0 and nIsUseJoint= 1: invalid, nMoveType= 0 and nIsUseJoint= 0: Use this space coordinate as the target position, and the joint coordinates obtained through inverse calculation are the target joint coordinates nMoveType= 1: target space position dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]
joints	joint target position	JointData	-	nMoveType= 0 and nIsUseJoint= 1: Use this joint coordinate as the target joint coordinate nMoveType= 0 and nIsUseJoint= 0: this joint coordinate is only used as the reference joint coordinate of the selected solution when calculating the inverse solution nMoveType= 1: invalid joints[i]: Joint i coordinates, unit [°]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page . It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used
dVelocity	speed	double	-	The maximum speed of movement, the unit of joint movement [°/s], the unit of XYZ [mm/s]

				of space movement, the unit of Rx, Ry, Rz [°/s]
dAcc	acceleration	double	-	The maximum acceleration of movement, the unit of joint movement [°/s ^{2}], the unit of XYZ [mm/s ^{2}] of space movement, the unit of Rx, Ry, Rz [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nMoveType	type of exercise	int	0/1	0: joint movement 1: linear motion
nIsUseJoint	Whether to use joint coordinates	int	0/1	Whether to use the joint angle as the target point, if nMoveType= 0, then nisJoint is valid: 0: Do not use joint angles 1: Use joint angles
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

* Note: H RIF_ WayPointEx It is necessary to set the specific values of tool coordinates and user coordinates; H RIF_ WayPoint Names of tool coordinates and user coordinates taught by the teach pendant.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.10.2. example

```
// define motion type
int nMoveType = 0;
// Define the spatial target position
Pose pose = new Pose();
pose.dX = 420; pose.dY = 0; pose.dZ = 443;
pose.dRX = 180; pose.dRY = 0; pose.dRZ = 180;
// Define the joint target position
JointsData joints = new JointsData();
joints.joint[0] = 0; joints.joint[1] = 0; joints.joint[2] = 90;
```

```
joints.joint[3] = 0; joints.joint[4] = 90; joints.joint[5] = 0;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// Define whether to use joint angles
int nIsUseJoint = 0;
// Define whether to use detection DI to stop
int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
int nIOState = 0;
// Define the waypoint ID
string strCmdID = "0";
// Execute waypoint movement
int nRet = hr_api.HRIF_WayPoint_nJ(0, 0,pose,joints,sTcpName, sUcsName, dVelocity, dAcc, dRadius, nMoveType,
nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);
```

3.10.11. HRIF_WayPoint2

3.10.11.1.Description: Waypoint movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dEndPos_X-dEndPos_Rz	space target location	double	-	nMoveType= 0 and nIsUseJoint= 1: invalid, nMoveType= 0 and nIsUseJoint= 0: Use this space coordinate as the target position, and the joint coordinates obtained through inverse calculation are the target joint coordinates nMoveType= 1: target space position nMoveType= 2: as the target position of the arc dEndPos_X: X coordinate, unit [mm] dEndPos_Y: Y coordinate, unit [mm] dEndPos_Z: Z coordinate, unit [mm] dEndPos_Rx: Rx coordinates, unit [°] dEndPos_Ry: Ry coordinates, unit [°] dEndPos_Rz: Rz coordinates, unit [°]
dJ1-dJ6	joint target position	double	-	nMoveType=0 and nIsUseJoint=1: Use this joint coordinate as the target joint coordinate nMoveType=0 and nIsUseJoint=0: This joint coordinate is only used as the reference joint coordinate of the selected solution when calculating the inverse solution nMoveType=1: invalid nMoveType=2: invalid dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°] dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6 : Joint 6 coordinates, unit [°]
sTcpName	Tool Coordinate	string	-	The name of the tool coordinate system where the target space coordinates are located

	Name			corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used
dVelocity	speed	double	-	The maximum speed of movement, the unit of joint movement [°/s], the unit of XYZ [mm/s] of space movement, the unit of Rx, Ry, Rz [°/s]
dAcc	acceleration	double	-	The maximum acceleration of movement, the unit of joint movement [°/s ^{2}], the unit of XYZ [mm/s ^{2}] of space movement, the unit of Rx, Ry, Rz [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nMoveType	type of exercise	int	0/1/2 _	0: joint movement 1: linear motion 2: arc movement
nIsUseJoint	Whether to use joint coordinates	int	0/1	Whether to use the joint angle as the target point, if nMoveType= 0, nIsUseJoint is valid: 0: Do not use joint angles 1: Use joint angles
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
dAuxPos_X-dAuxPos_Rz	space target location	double	-	nMoveType = 0: invalid nMoveType= 1: invalid nMoveType= 2: As the passing position of the arc dAuxPosX_X : X coordinate, unit [mm] dAuxPosX _Y: Y coordinate, unit [mm] dAuxPosX_Z: Z coordinate, unit [mm]

				dAuxPosX _Rx: Rx coordinates, unit [°] dAuxPosX_Ry: Ry coordinate, unit [°] dAuxPosX_Rz: Rz coordinate, unit [°]
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

* Note: H RIF_ WayPoint2 supports straight line and arc transition without deceleration; H RIF_ WayPoint only supports straight line and straight line transition without deceleration.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.11.2.example

```
// define motion type
int nMoveType = 0;
// Define the spatial target position
double dEndPos_X = 420; double dEndPos_Y = 0; double dEndPos_Z = 445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
// Define the spatial target position
double dAuxPos_X = 420; double dAuxPos_Y = 0; double dAuxPos_Z = 445;
double dAuxPos_Rx = 180; double dAuxPos_Ry = 0; double dAuxPos_Rz = 180;
// Define the joint target position
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// Define whether to use joint angles
int nIsUseJoint = 1;
// Define whether to use detection DI to stop
int nIsSeek = 0;
```

// Define the DI index for detection

int nIOBit = 0;

// Define the DI state of the detection

int nIOState = 0;

// Define the waypoint ID

string strCmdID = "0";

// Execute waypoint movement

int nRet = hr_api.HRIF_WayPoint2(0, 0, dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz, dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_Rx, dAuxPos_Ry, dAuxPos _Rz, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6, sTcpName, sUcsName , dVelocity, dAcc, dRadius, nMoveType, nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);

3.10.12. HRIF_WayPoint2_nJ

3.10.12.1. Description: Waypoint movement.Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
End	space target location	Pose	-	nMoveType= 0 and nIsUseJoint= 1: invalid, nMoveType= 0 and nIsUseJoint= 0: Use this space coordinate as the target position, and the joint coordinates obtained through inverse calculation are the target joint coordinates nMoveType= 1: target space position nMoveType= 2: as the target position of the arc dEndPos_X: X coordinate, unit [mm] dEndPos_Y: Y coordinate, unit [mm] dEndPos_Z: Z coordinate, unit [mm] dEndPos_Rx: Rx coordinates, unit [°] dEndPos_Ry: Ry coordinates, unit [°] dEndPos_Rz: Rz coordinates, unit [°]
joints	joint target position	JointData	-	nMoveType=0 and nIsUseJoint=1: Use this joint coordinate as the target joint coordinate nMoveType=0 and nIsUseJoint=0: This joint coordinate is only used as the reference joint coordinate of the selected solution when calculating the inverse solution nMoveType=1: invalid nMoveType=2: invalid joint[i]: Joint i coordinates, unit [°]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used

dVelocity	speed	double	-	The maximum speed of movement, the unit of joint movement [°/s], the unit of XYZ [mm/s] of space movement, the unit of Rx, Ry, Rz [°/s]
dAcc	acceleration	double	-	The maximum acceleration of movement, the unit of joint movement [°/s ^{2}], the unit of XYZ [mm/s ^{2}] of space movement, the unit of Rx, Ry, Rz [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nMoveType	type of exercise	int	0/1/2 _	0: joint movement 1: linear motion 2: arc movement
nIsUseJoint	Whether to use joint coordinates	int	0/1	Whether to use the joint angle as the target point, if nMoveType= 0, nIsUseJoint is valid: 0: Do not use joint angles 1: Use joint angles
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
Aux	space target location	Pose	-	nMoveType = 0: invalid nMoveType= 1: invalid nMoveType= 2: As the passing position of the arc dAuxPosX_X : X coordinate, unit [mm] dAuxPosX _Y: Y coordinate, unit [mm] dAuxPosX_Z: Z coordinate, unit [mm] dAuxPosX _Rx: Rx coordinates, unit [°] dAuxPosX_Ry: Ry coordinate, unit [°] dAuxPosX_Rz: Rz coordinate, unit [°]
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

* Note: H RIF_ WayPoint2 supports straight line and arc transition without deceleration; H RIF_ WayPoint only supports straight line and straight line transition without deceleration.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.12.2. example

```
// define motion type
int nMoveType = 0;
// Define the spatial target position
Pose End = new Pose();
End.dX = 420; End.dY = 0; End.dZ = 443;
End.dRX = 180; End.dRY = 0; End.dRZ = 180;
// Define the spatial target position
Pose Aux = new Pose();
Aux.dX = 420; Aux.dY = 0; Aux.dZ = 443;
Aux.dRX = 180; Aux.dRY = 0; Aux.dRZ = 180;
// Define the joint target position
JointsData joints = new JointsData();
joints.joint[0] = 0; joints.joint[1] = 0; joints.joint[2] = 90;
joints.joint[3] = 0; joints.joint[4] = 90; joints.joint[5] = 0; joints.joint[6] = 0;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// Define whether to use joint angles
int nIsUseJoint = 1;
// Define whether to use detection DI to stop
int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
int nIOState = 0;
// Define the waypoint ID
string strCmdID = "0";
```

// Execute waypoint movement

int nRet = hr_api.HRIF_WayPoint2_nJ(0, 0,End,Aux,joints,sTcpName, sUcsName, dVelocity, dAcc, dRadius, nMoveType,nIsUseJoint, nIsSeek, nIOBit, nIOState, strCmdID);

3.10.13. HRIF_MoveJ

3.10.13.1. Description: Joint movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	space target location	double	-	nIsUseJoint= 1: invalid nIsUseJoint= 0: Use this space coordinate as the target position, and calculate the joint coordinates through the inverse solution as the target joint coordinates: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
dJ1-dJ6	joint target position	double	-	nIsUseJoint= 1: Use this joint coordinate as the target joint coordinate nIsUseJoint= 0: This joint coordinate is only used as the reference joint coordinate of the selected solution when calculating the inverse solution: dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°] dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6: joint 6 coordinates, unit [°]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant

				page. Invalid when nIsUseJoint=1, the default name "Base" can be used
dVelocity	speed	double	-	Movement maximum speed, unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, unit [°/s ²]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nIsUseJoint	Whether to use joint coordinates	int	0/1	Whether to use the joint angle as the target point, if nMoveType= 0, nIsUseJoint is valid: 0: Do not use joint angles 1: Use joint angles
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.13.2. example

```
// Define the spatial target position
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 0; double dRy = 0; double dRz = 0;
// Define the joint target position
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
```

```
// define transition radius
double dRadius = 50;
// Define whether to use joint angles
int nIsUseJoint = 1;
// Define whether to use detection DI to stop
int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
int nIOState = 0;
// Define the waypoint ID
string strCmdID = "0";
// Execute waypoint movement
int nRet = hr_api.HRIF_MoveJ(0, 0, dX, dY, dZ, dRx, dRy, dRz, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6, sTcpName, sUcsName,
dVelocity, dAcc, dRadius, nIsUseJoint, nIsSeek, nIOBit , nIOState, strCmdID);
```

3.10.14. HRIF_MoveJ_nJ

3.10.14.1. Description: Joint movement.Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
pose	space target location	Pose	-	nIsUseJoint= 1: invalid nIsUseJoint= 0: Use this space coordinate as the target position, and calculate the joint coordinates through the inverse solution as the target joint coordinates: dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinates, unit [°] dRy: Ry coordinates, unit [°] dRz: Rz coordinates, unit [°]
joints	joint target position	JointData	-	nIsUseJoint= 1: Use this joint coordinate as the target joint coordinate nIsUseJoint= 0: This joint coordinate is only used as the reference joint coordinate of the selected solution when calculating the inverse solution: joints[i]: Joint i coordinates, unit [°]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. Invalid when nIsUseJoint=1, the default name "Base" can be used
dVelocity	speed	double	-	Movement maximum speed, unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, unit [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]

nIsUseJoint	Whether to use joint coordinates	int	0/1	Whether to use the joint angle as the target point, if nMoveType= 0, nIsUseJoint is valid: 0: Do not use joint angles 1: Use joint angles
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.14.2. example

```
// Define the spatial target position
Pose pose = new Pose();
pose.dX = 0; pose.dY = 0; pose.dZ = 0;
pose.dRX = 0; pose.dRY = 0; pose.dRZ = 0;
// Define the joint target position
JointsData joints = new JointsData();
joints.joint[0] = 0; joints.joint[1] = 0; joints.joint[2] = 90;
joints.joint[3] = 0; joints.joint[4] = 90; joints.joint[5] = 0;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// Define whether to use joint angles
```

```
int nIsUseJoint = 1;
// Define whether to use detection DI to stop
int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
int nIOState = 0;
// Define the waypoint ID
string strCmdID = "0";
// Execute waypoint movement
int nRet = hr_api.HRIF_MoveJ_nJ(0, 0, pose, joints, sTcpName, sUcsName, dVelocity, dAcc, dRadius, nIsUseJoint,
nIsSeek, nIOBit, nIOState, strCmdID);
```

3.10.15. HRIF_MoveL

3.10.15.1. Description: Linear trajectory movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	space target position	double	-	Target space location: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
dJ1-dJ6	joint reference position	double	-	Joint reference position, it is recommended to use joint coordinate values near the target Cartesian coordinates: dJ1: Joint 1 coordinates, unit [°] dJ2: joint 2 coordinates, unit [°] dJ3: Joint 3 coordinates, unit [°] dJ4: Joint 4 coordinates, unit [°] dJ5: Joint 5 coordinates, unit [°] dJ6: joint 6 coordinates, unit [°]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name "Base" can be used
dVelocity	speed	double	-	Movement maximum speed, X, Y, Z unit [mm/s], Rx, Ry, Rz unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, X, Y, Z unit [mm/s^2], Rx, Ry, Rz unit [$°/s^2$]

dRadius	transition radius	double	-	Transition radius, unit [mm]
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI index	int	0~7	The electrical box corresponds to the DI index, and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.15.2. example

```
// Define the spatial target position
double dX = 420; double dY = 10; double dZ = 445;
double dRx = 180; double dRy = 0; double dRz = 180;
// Define the joint target position
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// Define the transition radius
double dRadius = 50;
// Define whether to use detection DI to stop
 int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
```

```
int nIOState = 0;
// Define the waypoint ID
string strCmdID = "0";
// Execute waypoint movement
int nRet = hr_api.HRIF_MoveL(0, 0, dX, dY, dZ, dRx, dRy, dRz, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6, sTcpName, sUcsName,
dVelocity, dAcc, dRadius, nIsSeek, nIOBit, nIOState , strCmdID);
```

3.10.16. HRIF_MoveL_nJ

3.10.16.1. Description: Linear trajectory movement.Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
pose	space target position	Pose	-	Target space location: dTcp_X: X coordinate, unit [mm] dTcp_Y: Y coordinate, unit [mm] dTcp_Z: Z coordinate, unit [mm] dTcp_Rx: Rx coordinates, unit [°] dTcp_Ry: Ry coordinates, unit [°] dTcp_Rz: Rz coordinates, unit [°]
joints	joint reference position	JointData	-	Joint reference position, it is recommended to use joint coordinate values near the target Cartesian coordinates: joints[i]: Joint i coordinates, unit [°]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name "Base" can be used
dVelocity	speed	double	-	Movement maximum speed, X, Y, Z unit [mm/s], Rx, Ry, Rz unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, X, Y, Z unit [mm/s ^{2}], Rx, Ry, Rz unit [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
nIsSeek	Whether to detect DI stop	int	0/1	If nIsSeek is 1, the detection DI stop is turned on. If the state of DI indexed by the nIOBit bit index of the electric box = nIOState during the waypoint movement, the robot stops moving, otherwise it moves to the target point to complete the movement.
nIOBit	detected DI	int	0~7	The electrical box corresponds to the DI index,

	index			and it is invalid when nIsSeek=0
nIOState	DI status detected	int	0/1	The detected DI state, invalid when nIsSeek=0
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.16.2. example

```
// Define the spatial target position
Pose pose = new Pose();
pose.dX = 0; pose.dY = 0; pose.dZ = 0;
pose.dRX = 0; pose.dRY = 0; pose.dRZ = 0;
// Define the joint target position
JointsData joints = new JointsData();
joints.joint[0] = 0; joints.joint[1] = 0; joints.joint[2] = 90;
joints.joint[3] = 0; joints.joint[4] = 90; joints.joint[5] = 0;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// Define the transition radius
double dRadius = 50;
// Define whether to use detection DI to stop
 int nIsSeek = 0;
// Define the DI index for detection
int nIOBit = 0;
// Define the DI state of the detection
int nIOState = 0;
// Define the waypoint ID
string strCmdID = "0";
// Execute waypoint movement
int nRet = hr_api.HRIF_MoveL_nJ(0, 0, pose, joints,sTcpName, sUcsName, dVelocity, dAcc, dRadius, nIsSeek,
```

nIOBit, nIOState, strCmdID);

3.10.17. HRIF_MoveC

3.10.17.1. Description: Arc track movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dStartPos_X-dStartPos_Rz	Arc starting point position	double	-	Arc starting point position: dStartPos_X: X coordinate, unit [mm] dStartPos_Y: Y coordinate, unit [mm] dStartPos_Z: Z coordinate, unit [mm] dStartPos_Rx: Rx coordinates, unit [°] dStartPos_Ry: Ry coordinate, unit [°] dStartPos_Rz: Rz coordinate, unit [°]
dAuxPos_X-dAuxPos_Rz	Arc passing point position	double	-	Arc passing point position: dAuxPos_X: X coordinate, unit [mm] dAuxPos_Y: Y coordinate, unit [mm] dAuxPos_Z: Z coordinate, unit [mm] dAuxPos_Rx: Rx coordinates, unit [°] dAuxPos_Ry: Ry coordinate, unit [°] dAuxPos_Rz: Rz coordinate, unit [°]
dEndPos_X-dEndPos_Rz	Arc end point position	double	-	The position of the end point of the arc, if the full circle is used, the knot speed point is also a passing point on the circle, and it will stop after the full circle runs: dEndPos_X: X coordinate, unit [mm] dEndPos_Y: Y coordinate, unit [mm] dEndPos_Z: Z coordinate, unit [mm] dEndPos_Rx: Rx coordinates, unit [°] dEndPos_Ry: Ry coordinate, unit [°] dEndPos_Rz: Rz coordinate, unit [°]
nFixedPosure	Whether to fix the posture	int	0/1	Whether to keep the posture unchanged during the entire arc movement: 0: no fixed posture 1: Fixed posture
nMoveCType	Arc type	int	0/1	0 : full circle 1 : Arc

dRadLen	arc length	double	-	It is invalid when using arc movement, and the arc path is determined by three points. When using full circle movement, it indicates the number of circles of the full circle, and the decimal part is invalid.
dVelocity	speed	double	-	Movement maximum speed, X, Y, Z unit [mm/s], Rx, Ry, Rz unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, X, Y, Z unit [mm/s ^{2}], Rx, Ry, Rz unit [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.17.2. example

```
// arc starting point position
double dStartPos_X = 420; double dStartPos_Y = 0; double dStartPos_Z = 445;
double dStartPos_Rx = 180; double dStartPos_Ry = 0; double dStartPos_Rz = 180;
// Arc passing point position
double dAuxPos_X = 420; double dAuxPos_Y = 50; double dAuxPos_Z = 445;
double dAuxPos_Rx = 180; double dAuxPos_Ry = 0; double dAuxPos_Rz = 180;
// Arc end point position
double dEndPos_X = 470; double dEndPos_Y = 0; double dEndPos_Z = 445;
```

double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;

// Whether to fix the posture

int nFixedPosure = 0;

// arc type

int nMoveCType = 0;

// number of full circles

double dRadLen = 1;

// define movement speed

double dVelocity = 50;

// define motion acceleration

double dAcc = 50;

// define transition radius

double dRadius = 50;

// define tool coordinate variables

string sTcpName = "TCP";

// Define user coordinate variables

string sUcsName = "Base";

// Define the waypoint ID

string strCmdID = "0";

// Execute waypoint movement

int nRet= hr_api.HRIF_MoveC(0,0,dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry, dStartPos_Rz, dAuxPos_X, dAuxPos_Y, dAuxPos_Z, dAuxPos_Rx, dAuxPos_Ry, dAuxPos_R z, dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz, nFixedPosure, nMoveCType , dRadLen, dVelocity, dAcc, dRadius, sTcpName , sUcsName, strCmdID);

3.10.18. HRIF_MoveC_nJ

3.10.18.1. Description: Arc track movement.Note: This interface is specifically for the extended-axis version.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
Start	Arc starting point position	Pose	-	Arc starting point position: dStartPos_X: X coordinate, unit [mm] dStartPos_Y: Y coordinate, unit [mm] dStartPos_Z: Z coordinate, unit [mm] dStartPos_Rx: Rx coordinates, unit [°] dStartPos_Ry: Ry coordinate, unit [°] dStartPos_Rz: Rz coordinate, unit [°]
Aux	Arc passing point position	Pose	-	Arc passing point position: dAuxPos_X: X coordinate, unit [mm] dAuxPos_Y: Y coordinate, unit [mm] dAuxPos_Z: Z coordinate, unit [mm] dAuxPos_Rx: Rx coordinates, unit [°] dAuxPos_Ry: Ry coordinate, unit [°] dAuxPos_Rz: Rz coordinate, unit [°]
End	Arc end point position	Pose	-	The position of the end point of the arc, if the full circle is used, the knot speed point is also a passing point on the circle, and it will stop after the full circle runs: dEndPos_X: X coordinate, unit [mm] dEndPos_Y: Y coordinate, unit [mm] dEndPos_Z: Z coordinate, unit [mm] dEndPos_Rx: Rx coordinates, unit [°] dEndPos_Ry: Ry coordinate, unit [°] dEndPos_Rz: Rz coordinate, unit [°]
nFixedPosure	Whether to fix the posture	int	0/1	Whether to keep the posture unchanged during the entire arc movement: 0: no fixed posture 1: Fixed posture
nMoveCType	Arc type	int	0/1	0 : full circle 1 : Arc

dRadLen	arc length	double	-	It is invalid when using arc movement, and the arc path is determined by three points. When using full circle movement, it indicates the number of circles of the full circle, and the decimal part is invalid.
dVelocity	speed	double	-	Movement maximum speed, X, Y, Z unit [mm/s], Rx, Ry, Rz unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, X, Y, Z unit [mm/s ^{2}], Rx, Ry, Rz unit [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence
joints	Reference joint angles at the end point	JointsData		Reference joint angles at the end point, used to determine whether the linked axes need to move joints. joint[i]: Angle of the i-th axis, unit [°]
nIsUseCurRefACS	Whether to use the reference joint angles	int	0/1	0: Use the reference joint angles at the end point 1: The extended axis does not move

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.18.2. example

```
// arc starting point position
Pose Start=new Pose();
```

```
Start.dX = 420; Start.dY = 0; Start.dZ = 443;
Start.dRX = 180; Start.dRY = 0; Start.dRZ = 180;
// Arc passing point position
Pose Aux=new Pose();
Aux.dX = 420; Aux.dY = 50; Aux.dZ = 443;
Aux.dRX = 180; Aux.dRY = 0; Aux.dRZ = 180;
// Arc end point position
Pose End=new Pose();
End.dX = 420; End.dY = 0; End.dZ = 493;
End.dRX = 180; End.dRY = 0; End.dRZ = 180;
// Whether to fix the posture
int nFixedPosure = 0;
// arc type
int nMoveCType = 0;
// number of full circles
double dRadLen = 1;
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// Define the waypoint ID
string strCmdID = "0";
// Define reference joint angles at the end point
JointsData joints = new JointsData();
for(int i=0; i < 10; i++)
{
    joints.joint[i] = 0;
}
// Define whether to use the reference joint angles
int nIsUseCurRefACS = 0;
// Execute waypoint movement
int nRet = hr_api.HRIF_MoveC_nJ(0, 0,Start,Aux,End,nFixedPosure, nMoveCType, dRadLen, dVelocity, dAcc,
dRadius, sTcpName, sUcsName, strCmdID,joints,nIsUseCurRefACS);
```

3.10.19. HRIF_MoveZ

3.10.19.1. Description: Z-shaped trajectory movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dStartPos_X-dStartPos_Rz	Z-shaped starting point position	double	-	Z-shaped starting point position: dStartPos_X: X coordinate, unit [mm] dStartPos_Y: Y coordinate, unit [mm] dStartPos_Z: Z coordinate, unit [mm] dStartPos_Rx: Rx coordinates, unit [°] dStartPos_Ry: Ry coordinate, unit [°] dStartPos_Rz: Rz coordinate, unit [°]
dEndPos_X-dEndPos_Rz	Z-type end point position	double	-	Z-type end point position: dEndPos_X: X coordinate, unit [mm] dEndPos_Y: Y coordinate, unit [mm] dEndPos_Z: Z coordinate, unit [mm] dEndPos_Rx: Rx coordinates, unit [°] dEndPos_Ry: Ry coordinate, unit [°] dEndPos_Rz: Rz coordinate, unit [°]
dPlanePos_X-dPlanePos_Rz	The trajectory determines the position of the plane point	double	-	Z-shaped trajectory plane point position: dPlanePosPos_X: X coordinate, unit [mm] dPlanePosPos_Y: Y coordinate, unit [mm] dPlanePosPos_Z: Z coordinate, unit [mm] dPlanePosPos_Rx: Rx coordinates, unit [°] dPlanePosPos_Ry: Ry coordinates, unit [°] dPlanePosPos_Rz: Rz coordinates, unit [°]
dVelocity	speed	double	-	Movement maximum speed, X, Y, Z unit [mm/s], Rx, Ry, Rz unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, X, Y, Z unit [mm/s ^{2}], Rx, Ry, Rz unit [°/s ^{2}]
dW ID th	width	double	-	track width
d Density	density	double	-	Track Density, automatically calculates density from velocity when density is not

				used
nEnableDensity	whether to use density	int	-	Whether to use density: 0: not used 1: Use
nEnablePlane	Whether to use flat points	int	-	Whether to use plane points, if not used, determine the XYZ plane according to the selected user coordinates: 0: not used 1: Use
nEnableWaiTime	Whether to enable turning point waiting time	int	-	Whether to enable turning point waiting time: 0: not used 1: use
nPosiTime	Positive turning point waiting time ms	int	-	Waiting time for forward turning point , unit [ms]
nNegaTime	Negative turning point waiting time ms	int	-	Negative turning point waiting time , unit [ms]
dRadius	transition radius	double	-	Transition radius, unit [mm]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.19.2. example

// starting point position

double dStartPos_X = 420; double dStartPos_Y = 0; double dStartPos_Z = 445;

```
double dStartPos_Rx = 180; double dStartPos_Ry = 0; double dStartPos_Rz = 180;
// end point position
double dEndPos_X = 420; double dEndPos_Y = 100; double dEndPos_Z = 445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
// Determine the position of the trajectory plane point
double dPlanePos_X = 470; double dPlanePos_Y = 50; double dPlanePos_Z = 445;
double dPlanePos_Rx = 180; double dPlanePos_Ry = 0; double dPlanePos_Rz = 180;
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 2500;
// width
double dWidth = 50;
// density
double dDensity = 10;
// use density
int nEnableDensity = 1;
// use flat points
int nEnablePlane = 1;
// Whether to wait at the turning point - do not wait
int nEnableWaiTime = 0;
// Waiting time for forward turning point
int nPosiTime = 0;
// Negative turning point waiting time
int nNegaTime = 0;
// Define the transition radius
double dRadius = 5;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// Define the waypoint ID
string strCmdID = "0";
// Execute waypoint movement
int nRet= hr_api.HRIF_MoveZ(0,0, dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry, dStartPos_Rz, dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_R z, dPlanePos_X, dPlanePos_Y, dPlanePos_Z, dPlanePos_Rx, dPlanePos_Ry, dPlanePos_Rz, dVelocity, dAcc , dWidth, dDensity, nEnableDensity, nEnablePlane, nEnableWaiTime, nPosiTime, nNegaTime, dRadius, sTcpName, sUcsName, strCmdID);
```

3.10.20. HRIF_MoveE

3.10.20.1. Description: Elliptical trajectory movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
d P1 _X-d P1 _Rz	Teach position 1	double	-	Teaching position 1: dP1_X: X coordinate, unit [mm] dP1_Y: Y coordinate, unit [mm] dP1_Z: Z coordinate, unit [mm] d P1 _Rx: Rx coordinates, unit [°] d P1 _Ry: Ry coordinates, unit [°] d P1 _Rz: Rz coordinates, unit [°]
d P2 _X-d P2 _Rz	Teach position 2	double	-	Teaching position 2 : dP 2 _X: X coordinate, unit [mm] dP 2 _Y: Y coordinate, unit [mm] dP 2 _Z: Z coordinate, unit [mm] d P2 _Rx: Rx coordinates, unit [°] d P2 _Ry: Ry coordinates, unit [°] d P2 _Rz: Rz coordinates, unit [°]
d P3 _X-d P1 _Rz	Teach position 3	double	-	Teaching position 3 : dP 3 _X: X coordinate, unit [mm] dP 3 _Y: Y coordinate, unit [mm] dP 3 _Z: Z coordinate, unit [mm] d P3 _Rx: Rx coordinates, unit [°] d P3 _Ry: Ry coordinates, unit [°] d P3 _Rz: Rz coordinates, unit [°]
d P4 _X-d P4 _Rz	Teach position 4	double	-	Teaching position 4 : dP 4 _X: X coordinate, unit [mm] dP 4 _Y: Y coordinate, unit [mm] dP 4 _Z: Z coordinate, unit [mm] d P4 _Rx: Rx coordinates, unit [°] d P4 _Ry: Ry coordinates, unit [°] d P4 _Rz: Rz coordinates, unit [°]
d P5 _X-d P5	Teaching	double	-	Teaching position 5 :

_Rz	position 5			dP 5 _X: X coordinate, unit [mm] dP 5 _Y: Y coordinate, unit [mm] dP 5 _Z: Z coordinate, unit [mm] d P5 _Rx: Rx coordinates, unit [°] d P5 _Ry: Ry coordinates, unit [°] d P5 _Rz: Rz coordinates, unit [°]
nOrientMode	sports mode	int	0 /1	0: Ellipse arc 1: Entire ellipse
n MoveType	type of exercise	int	0 /1	0: Do not use fixed posture 1: Use a fixed posture
dArcLength	radian	double	-	Radian, unit [°]
dVelocity	speed	double	-	Movement maximum speed, X, Y, Z unit [mm/s], Rx, Ry, Rz unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, X, Y, Z unit [mm/s ^{2}], Rx, Ry, Rz unit [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

* Note

HR6.5 series and later versions (including HR6.5.x, HR6.6.x, HRC8.x, ...) remove this command and no longer support it.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.20.2. example

```
// teaching point 1
double dP1_X = 420; double dP1_Y = 0; double dP1_Z = 445;
double dP1_Rx = 180; double dP1_Ry = 0; double dP1_Rz = 180;
// teaching point 2
```

```
double dP2_X = 460; double dP2_Y = 0; double dP2_Z = 445;
double dP2_Rx = 180; double dP2_Ry = 0; double dP2_Rz = 180;
// teaching point 3
double dP3_X = 480; double dP3_Y = 10; double dP3_Z = 445;
double dP3_Rx = 180; double dP3_Ry = 0; double dP3_Rz = 180;
// teaching point 4
double dP4_X = 460; double dP4_Y = 20; double dP4_Z = 445;
double dP4_Rx = 180; double dP4_Ry = 0; double dP4_Rz = 180;
// teaching point 5
double dP5_X = 420; double dP5_Y = 20; double dP5_Z = 445;
double dP5_Rx = 180; double dP5_Ry = 0; double dP5_Rz = 180;
// sport mode
int nOrientMode = 0;
// movement type
int nMoveType = 1;
// arc length
int dArcLength = 360;
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 2500;
// define transition radius
double dRadius = 5;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// Define the waypoint ID
string strCmdID = "0";
// perform ellipse motion
int nRet = hr_api.HRIF_MoveE(0, 0, dP1_X, dP1_Y, dP1_Z, dP1_Rx, dP1_Ry, dP1_Rz, dP2_X, dP2_Y, dP2_Z,
dP2_Rx, dP2_Ry, dP2_Rz, dP3_X, dP3_Y, dP3_Z, dP3 _Rx, dP3_Ry, dP3_Rz, dP4_X, dP4_Y , DP4_Z, DP4_RX,
DP4_ry, DP4_RZ, DP5_X, DP5_Y, DP5_RX, DP5_ry, DP5_RZ, NMoveType, DVELOCITY, DRADIUS, Stpn AME,
Sucsname, Strcmdid);
```

3.10.21. HRIF_MoveS

3.10.21.1. Description: Archimedes helical motion, the initial radius is fixed at 1mm.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dSpiralIncrement	incremental radius	double	> 0	Incremental radius per circle of helical motion, unit [mm]
dSpiralDiameter	end radius	double	>1	End radius of spiral movement, unit [mm]
dVelocity	speed	double	-	Movement maximum speed, unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, unit [°/s ^{2}]
dRadius	transition radius	double	-	Transition radius, unit [mm]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page. It is invalid when nIsUseJoint=1, and the default name "Base" can be used
strCmdID	command ID	string	-	Current waypoint ID, can be customized, or set to "1", "2", "3" in sequence

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.21.2. example

```
// define incremental radius
double dSpiralIncrement = 1;
// define end radius
double dSpiralDiameter = 5;
// Define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
```

```
string sUcsName = "Base";
// define movement speed
double dVelocity = 50;
// define motion acceleration
double dAcc = 50;
// define transition radius
double dRadius = 50;
// Define the waypoint ID
string strCmdID = "0";
// Execute spiral trajectory movement
int nRet = hr_api.HRIF_MoveS(0, 0, dSpiralIncrement, dSpiralDiameter, dVelocity, dAcc, dRadius, sTcpName, sUcsName, strCmdID);
```

3.10.22. HRIF_MoveLinearWeave

3.10.22.1. Description: Archimedes helical motion, the initial radius is fixed at 1mm.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dStartPos_X-dStartPos_Rz	Start Position	double	> 0	Linear weave welding start position: dStartPos_X: X coordinate, unit [mm] dStartPos_Y: Y coordinate, unit [mm] dStartPos_Z: Z coordinate, unit [mm] dStartPos_Rx: Rx coordinate, unit [°] dStartPos_Ry: Ry coordinate, unit [°] dStartPos_Rz: Rz coordinate, unit [°]
dEndPos_X-dEndPos_Rz	End Position	double	>1	End radius of spiral movement, unit [mm]
dVelocity	speed	double	-	Movement maximum speed, unit [°/s]
dAcc	acceleration	double	-	Movement maximum acceleration, unit [°/s^2]
dRadius	transition radius	double	-	Transition radius, unit [mm]
intervalDistance	Amplitude	double	-	Weave welding amplitude, unit [mm]
weaveFrameType	Interval Distance	int	-	Method to determine weave plane and direction: 0: The +X direction of the weave plane is from StartPos to EndPos; the +Z direction (normal direction of the weave plane) is the same as the +Z of the tool coordinate system; the +Y direction is determined by the right-hand rule. The initial weave motion direction is on the same side as the +Y direction. 1: The +X direction of the weave plane is from StartPos to EndPos; the Z direction is parallel to the Z direction of the tool coordinate system; the +Y direction is on the same side as the +Y of the tool coordinate system. The initial weave motion direction is on the same side as the +Y direction.

elevation	Elevation Angle	double	-	Weave welding elevation angle (rotation angle of the weld seam plane around the weld advancement direction), unit [°]
azimuth	Azimuth Angle	double	-	Weave welding azimuth angle (rotation angle around the normal vector of the weave plane), unit [°]
centreRise	Central Rise	double	-	Weave welding central rise (torch rise at the weave center), unit [mm]
nEnableWaiTime	Turnaround Wait Time Enable	int	-	Turnaround wait time enable: 0: Not used 1: Used
nPosiTime	Positive Turnaround Wait Time	int	-	Positive turnaround wait time, unit [ms]
nNegaTime	Negative Turnaround Wait Time	int	-	Negative turnaround wait time, unit [ms]
sTcpName	Tool Coordinate Name	string	-	Name of the tool coordinate system where the target spatial coordinates are located (corresponding to the name on the teach pendant; default name "TCP" is available)
sUcsName	User Coordinate Name	string	-	Name of the user coordinate system where the target spatial coordinates are located (corresponding to the name on the teach pendant; default name "Base" is available)
strCmdID	Command ID	string	-	Current waypoint ID (customizable, or sequentially set as "1", "2", "3")

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.22.2. example

```
// Start position
double dStartPos_X = 420;    double dStartPos_Y = 0;    double dStartPos_Z = 445;
double dStartPos_Rx = 180;    double dStartPos_Ry = 0;    double dStartPos_Rz = 180;
// End position
double dEndPos_X = 520; double dEndPos_Y = 100; double dEndPos_Z = 445;
double dEndPos_Rx = 180; double dEndPos_Ry = 0; double dEndPos_Rz = 180;
// Velocity
double dVelocity = 10;
// Acceleration
double dAcc = 2500;
```

```
// Blending radius
double dRadius = 5;
// Amplitude
double dAmplitude = 30;
// Interval distance
double dIntervalDistanc = 10;
// Weave direction
int nWeaveFrameType = 0;
// Elevation angle
double dElevation = 10;
// Azimuth angle
double dAzimuth = 0;
// Central rise
double dCentreRise = 0;
// Turnaround wait time enable
int nEnableWaiTime = 0;
// Positive turnaround wait time
int nPosiTime = 0;
// Negative turnaround wait time
int nNegaTime = 0;
// Tool coordinate name
string sTcpName = "TCP";
// Coordinate system
string sUcsName = "Base";
// Command ID
string sCmdID = "0";
// Returns 0 for successful call
int nRet = hr_api.HRIF_MoveLinearWeave(0, 0, dStartPos_X, dStartPos_Y, dStartPos_Z, dStartPos_Rx, dStartPos_Ry, dStartPos_Rz, dEndPos_X, dEndPos_Y, dEndPos_Z, dEndPos_Rx, dEndPos_Ry, dEndPos_Rz, dVelocity, dAcc, dRadius, dAmplitude, dIntervalDistanc, nWeaveFrameType, dElevation, dAzimuth, dCentreRise, nEnableWaiTime, nPosiTime, nNegaTime, sTcpName, sUcsName, sCmdID);
```

3.10.23. MoveCircularWeave

3.10.23.1. Description: Circular weave welding motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dStart_X-dStart_Rz	Start Position	double	> 0	Linear weave welding start position: dStartPos_X: X coordinate, unit [mm] dStartPos_Y: Y coordinate, unit [mm] dStartPos_Z: Z coordinate, unit [mm] dStartPos_Rx: Rx coordinate, unit [°] dStartPos_Ry: Ry coordinate, unit [°] dStartPos_Rz: Rz coordinate, unit [°]
dAux_X-dAux_Rz	End Position	double	>1	End radius of spiral movement, unit [mm]
dEnd_X-dEnd_Rz	speed	double	-	Movement maximum speed, unit [°/s]
dVelocity	acceleration	double	-	Movement maximum acceleration, unit [°/s^2]
dAcc	transition radius	double	-	Transition radius, unit [mm]
dRadius	Fixed Orientation Enable	int	0/1	Fixed orientation enable: 0: Do not use fixed orientation 1: Use fixed orientation
nOrientMode	Trajectory Type	int	0/1	Trajectory type: 0: Full circle trajectory 1: Circular arc trajectory
nMoveWhole	Number of Cycles	int	-	Invalid for circular arc motion (arc path and angle are determined by calculating three points); represents the number of full circles for full circle motion
nMoveWholeLen	Amplitude	float	-	Weave welding amplitude, unit [mm]
dAmplitude	Amplitude	double	-	Weave welding amplitude, unit [mm]
dInterValDistance	Interval Distance	int	-	Method to determine weave plane and direction: 0: The +X direction of the weave plane is from StartPos to EndPos; the +Z direction (normal direction of the weave plane) is the same as the +Z of the tool coordinate

				system; the +Y direction is determined by the right-hand rule. The initial weave motion direction is on the same side as the +Y direction. 1: The +X direction of the weave plane is from StartPos to EndPos; the Z direction is parallel to the Z direction of the tool coordinate system; the +Y direction is on the same side as the +Y of the tool coordinate system. The initial weave motion direction is on the same side as the +Y direction.
nWeaveFrameType	Elevation Angle	double	-	Weave welding elevation angle (rotation angle of the weld seam plane around the weld advancement direction), unit [°]
dElevation	Azimuth Angle	double	-	Weave welding azimuth angle (rotation angle around the normal vector of the weave plane), unit [°]
dAzimuth	Central Rise	double	-	Weave welding central rise (torch rise at the weave center), unit [mm]
dCentreRise	Turnaround Wait Time Enable	int	-	Turnaround wait time enable: 0: Not used 1: Used
nEnableWaitTime	Positive Turnaround Wait Time	int	-	Positive turnaround wait time, unit [ms]
nPosiTime	Negative Turnaround Wait Time	int	-	Negative turnaround wait time, unit [ms]
nNegaTime	Tool Coordinate Name	string	-	Name of the tool coordinate system where the target spatial coordinates are located (corresponding to the name on the teach pendant; default name "TCP" is available)
sTcpName	User Coordinate Name	string	-	Name of the user coordinate system where the target spatial coordinates are located (corresponding to the name on the teach pendant; default name "Base" is available)
sUcsName	Command ID	string	-	Current waypoint ID (customizable, or sequentially set as "1", "2", "3")

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.23.2. example

```
// define incremental radius
double dSpiralIncrement = 1;
// Taught point 1
double dStart_X = 420; double dStart_Y = 0; double dStart_Z = 445;
double dStart_Rx = 180; double dStart_Ry = 0; double dStart_Rz = 180;
// Taught point 2
double dAux_X = 460; double dAux_Y = 0; double dAux_Z = 445;
double dAux_Rx = 180; double dAux_Ry = 0; double dAux_Rz = 180;
// Taught point 3
double dEnd_X = 480; double dEnd_Y = 10; double dEnd_Z = 445;
double dEnd_Rx = 180; double dEnd_Ry = 0; double dEnd_Rz = 180;
// Define motion velocity
double dVelocity = 50;
// Define motion acceleration
double dAcc = 500;
// Define blending radius
double dRadius = 50;
// Define fixed orientation
int nOrientMode = 0;
// Define trajectory type
int nMoveWhole = 0;
// Define number of cycles
int nMoveWholeLen = 10;
// Define amplitude
double dAmplitude = 10;
// Define interval distance
double dInterValDistance = 10;
// Define selection method
int nWeaveFrameType = 0;
// Define elevation angle
double dElevation = 10;
// Define azimuth angle
double dAzimuth = 10;
// Define central rise
double dCentreRise = 5;
// Define turnaround wait time enable
int nEnableWaitTime = 0;
// Define positive turnaround wait time
```

```
int nPosiTime = 10;
// Define negative turnaround wait time
int nNegaTime = 50;
// Define tool coordinate variable
string sTcpName = "TCP";
// Define user coordinate variable
string sUcsName = "Base";
// Define waypoint ID
string sCmdID = "0";
// Execute circular weave welding motion
int nRet = hr_api.HRIF_MoveCircularWeave(0, 0, dStart_X, dStart_Y, dStart_Z, dStart_Rx, dStart_Ry, dStart_Rz, dAux_X, dAux_Y, dAux_Z, dAux_Rx, dAux_Ry, dAux_Rz, dEnd_X, dEnd_Y, dEnd_Z, dEnd_Rx, dEnd_Ry, dEnd_Rz, dVelocity, dAcc, dRadius, nOrientMode, nMoveWhole, nMoveWholeLen, dAmplitude, dInterValDistance, nWeaveFrameType, dElevation, dAzimuth, dCentreRise, nEnableWaitTime, nPosiTime, nNegaTime, sTcpName, sUcsName, sCmdID);
```

3.10.24. HRIF_ShortJogJ

3.10.24.1. Description: Short jog motion of joints with a movement distance of 2° and a maximum speed not exceeding 10°/s.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nAxisID	Axis ID	int	0~5	Target axis ID for motion, corresponding to joints J1-J6
nDirection	Direction	int	0/1	0: Negative direction 1: Positive direction

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.24.2. example

```
// Coordinate system axis ID
int nAxisID = 0;
// Motion direction
int nDirection = 0;
// Short jog motion of joints
int nRet= hr_api.HRIF_ShortJogJ(0,0, nAxisID , nDirection);
```

3.10.25. HRIF_ShortJogL

3.10.25.1. Description: Short jog motion in Cartesian coordinates with a movement distance of 2mm and a maximum speed not exceeding 50mm/s.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nAxisID	Axis ID	int	0~5	Target axis ID for motion, corresponding to Cartesian coordinates X-Rz
nDirection	Direction	int	0/1	0: Negative direction 1: Positive direction

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.25.2. example

```
// Coordinate system axis ID
int nAxisID = 0;
// Motion direction
int nDirection = 0;
// Short jog motion in Cartesian coordinates
int nRet = hr_api.HRIF_ShortJogL(0, 0, nAxisID, nDirection);
```

3.10.26. HRIF_LongJogJ

3.10.26.1. Description: Long jog motion of joints with a maximum movement speed not exceeding 10°/s..

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nAxisID	Axis ID	int	0~5	Target axis ID for motion, corresponding to joints J1-J6
nDirection	Direction	int	0/1	0: Negative direction 1: Positive direction
nState	State	int	0/1	0: Disable 1: Enable

* Note: Must be used in conjunction with HRIF_LongMoveEvent.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.26.2. example

```
// Coordinate system axis ID
int nAxisID = 0;
// Motion direction
int nDirection = 0;
// Set state
int nState = 1;
// Long jog motion of joints
int nRet = hr_api.HRIF_LongJogJ(0, 0, nAxisID, nDirection, nState);
```

3.10.27. HRIF_LongJogL

3.10.27.1. Description: Long jog motion in Cartesian coordinates with a maximum movement speed not exceeding 30mm/s.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nAxisID	Axis ID	int	0~5	Target axis ID for motion, corresponding to Cartesian coordinates X-Rz
nDirection	Direction	int	0/1	0: Negative direction 1: Positive direction
nState	State	int	0/1	0: Disable 1: Enable

* Note: Must be used in conjunction with HRIF_LongMoveEvent.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.27.2. example

```
// Coordinate system axis ID
int nAxisID = 0;
// Motion direction
int nDirection = 0;
// Set state
int nState = 1;
// Long jog motion in Cartesian coordinates
int nRet= hr_api.HRIF_LongJogL(0,0, nAxisID , nDirection, nState);
```

3.10.28. HRIF_LongMoveEvent

3.10.28.1. Description: Long jog motion continuation command. After starting long jog motion, this command must be sent at a time interval of 200 milliseconds or shorter; otherwise, the long jog motion will stop.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.10.28.2. example

```
// Long jog motion continuation command
int nRet= hr_api.HRIF_LongMoveEvent(0,0);
```

3.11. Continuous trajectory motion control instructions

3.11.1. HRIF_InitPath

3.11.1.1. Description: Initialize trajectory.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nRawDataType	Original Pose Type	int	0/1	0: Joint Pose; 1: Cartesian Pose
sPathName	track name	string	-	Name of the trajectory, consisting of underscores, letters or numbers, with a maximum length of 29 characters.
dSpeedRatio	Trajectory speed ratio	double	0~1	Trajectory speed ratio
dRadius	transition radius	double	>= 0	Transition radius, unit [mm]
dVelocity	Trajectory speed	double	-	Trajectory motion velocity, unit [mm/s]. This parameter is required for PathL calculation. Mandatory when nRawDataType = 1; not required when nRawDataType = 0 (the system automatically uses the maximum value for PathL calculation).
dAcc	Trajectory motion acceleration	double	-	Trajectory motion acceleration, unit [mm/s²]. This parameter is required for PathL calculation. Mandatory when nRawDataType = 1; not required when nRawDataType = 0 (the system automatically uses the maximum value for PathL calculation).
dJerk	trajectory jerk	double	-	Trajectory motion jerk, unit [mm/s³]. This parameter is required for PathL calculation.Mandatory when nRawDataType = 1; not required when nRawDataType = 0 (the system automatically uses the maximum value for PathL calculation).
sUcsName	User Coordinate Name	string	-	Name of the user coordinate system where the target spatial coordinates are located, corresponding to the name on the teach pendant

				page - the default name "Base" can be used. Name of the user coordinate system where the target spatial coordinates are located, corresponding to the name on the teach pendant page - the default name "Base" can be used.
sTcpName	Tool Coordinate Name	string	-	Name of the tool coordinate system where the target spatial coordinates are located, corresponding to the name on the teach pendant page - the default name "TCP" can be used. This parameter is required for PathL calculation. Mandatory when nRawDataType = 1; not required when nRawDataType = 0 (the system automatically uses "TCP" for PathL calculation).

* Note:

When this command is executed, the system first checks whether a trajectory with the same name already exists. If it exists, the system deletes it first.

After the trajectory is successfully created, its status is set to "Teaching in Progress".

This command can replace HRIF_StartPushMovePathJ and HRIF_InitMovePathL.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.1.2. example

```
// Original pose type
int nRawDataType = 1;
// Trajectory name
string trajectName = "Path_01";
// Trajectory motion speed ratio
double dSpeedRatio = 0.3;
// Blending radius
double dRadius = 20;
// Trajectory motion velocity
double dVelocity = 100;
// Trajectory motion acceleration
double dAcc = 500;
// Trajectory motion jerk
double dJerk = 10000;
// User coordinate system name
string sUcsName = "Base";
```

```
// Tool coordinate system name
string sTcpName = "TCP";
// Initialize linear motion trajectory
int nRet = hr_api.HRIF_InitPath(0, 0, nRawDataType, trajectName, dSpeedRatio, dRadius, dVelocity, dAcc, dJerk,
sUcsName, sTcpName);
```

3.11.2. HRIF_PushMovePathL

3.11.2.1. Description: Batch push raw pose data to the trajectory (can be called multiple times).

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sPathName	Trajectory Name	string	-	Trajectory name, applicable to MovePathJ and MovePathL
sPoints	Pose Data	double	-	Pose data, with each data item separated by a comma. Each pose contains 6 coordinate values, so the number of values in this field must be a multiple of 6. The system automatically calculates the number of poses.

* Note:

This command can be called repeatedly.

This command is also applicable to trajectories with statuses of "Teaching Completed", "Calculation Completed", "Calculation Failed", or "Not Taught".

The maximum number of raw poses allowed is 10000. If the sum of the poses to be pushed and the existing poses exceeds 10000, none of the poses in the current batch will be added to the pose queue.

After executing this command, the trajectory status changes to "Teaching in Progress".

This command can replace HRIF_PushMovePaths.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.2.2. example

```
// Trajectory name
string trajectName = "Path_01";
// Pose data
string sPoints = "420,0,445,180,0,180, 430,10,445,180,0,180, 440,50,445,180,0,180,
                  520,100,445,180,0,180, 450,50,445,180,0,180, 430,200,445,180,0,180";
// Batch push raw poses to the trajectory
int nRet = hr_api.HRIF_PushPathPoints(0, 0, trajectName, sPoints);
```

3.11.3. HRIF_DelPathJ

3.11.3.1. Description: Delete the specified track.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sPathName	Trajectory Name	string	-	Trajectory name, applicable to MovePathJ and MovePathL

* Note:

This command can replace HRIF_DelMovePathJ.

Trajectories that are calculating target poses or undergoing post-import processing cannot be deleted.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.3.2. example

```
// Trajectory name
string sPathName = "drag_03";
// Delete trajectory
int nRet = hr_api.HRIF_DelPath(0, 0, sPathName);
```

3.11.4. HRIF_StartPushMovePathJ

3.11.4.1. Description: Initialize joint continuous trajectory motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name
dSpeedRatio	Trajectory speed ratio	double	0~1	Trajectory speed ratio
dRadius	transition radius	double	>= 0	Transition radius, unit [mm]

✓ return value

return value	**name**	**type of data**	**Effective range**	**content**
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.4.2. example

```
// track name
string sTrackName = "drag_1";
// speed
double dSpeedRatio = 0.5;
// transition radius
double dRadius = 2;
// Initialize joint continuous trajectory motion
int nRet= hr_api.HRIF_StartPushMovePathJ(0,0, sTrackName, dSpeedRatio, dRadius);
```

3.11.5. HRIF_PushMovePathJ

3.11.5.1. Description: Issue the movement track point.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name
dJ1-dJ6	Joint point	double	-	Target joint position: dJ1: joint 1 position, unit [°] dJ2: joint 2 position, unit [°] dJ3: position of joint 3, unit [°] dJ4: position of joint 4, unit [°] dJ5: joint 5 position, unit [°] dJ6: position of joint 6, unit [°]

* Note: After calling H RIF_ StartPushMovePath, this function can be called multiple times. Generally, the number of points needs to be greater than 4.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.5.2. example

```
// track name
string sTrackName = "drag_1";
// Target joint position
double dJ1 = 0; double dJ2 = 0; double dJ3 = 0;
double dJ4 = 0; double dJ5 = 0; double dJ6 = 0;
// send joint points
int nRet= hr_api.HRIF_PushMovePathJ(0,0, sTrackName, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6);
```

3.11.6. HRIF_EndPushMovePathJ

3.11.6.1. Description: The trajectory distribution is completed and trajectory calculation starts.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.6.2. example

```
// track name
string sTrackName = "drag_1";
// After the delivery is complete, start to calculate the trajectory
int nRet= hr_api.HRIF_EndPushMovePathJ(0,0, sTrackName);
```

3.11.7. HRIF_MovePathJ

3.11.7.1. Description: Motion specified trajectory.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.7.2. example

```
// track name
string sTrackName = "drag_1";
// motion track
int nRet= hr_api.HRIF_MovePathJ(0,0, sTrackName);
```

3.11.8. HRIF_ReadMovePathJState

3.11.8.1. Description: Read the current track state.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name

✓ output variable

output variable	name	type of data	Effective range	content
nState	track state	int	0~5	Current track status: 0: Path teaching has not started, switch to this state after calling HRIF_StartPushMovePath J 1: During trajectory teaching, call HRIF_PushMovePathJ switch to this state after 2: During trajectory calculation, switch to this state after calling HRIF_EndPushMovePath J 3: The trajectory calculation is completed. After the calculation is completed, switch to this state and start to move. 4: End teaching 5: The calculation is wrong, you need to check whether the point is reasonable, and then teach again after correction.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.8.2. example

```
// track name
string sTrackName = "drag_1";
// read the current track state
int nState = 0;
// read track status
int nRet= hr_api.HRIF_ReadMovePathJState(0,0, sTrackName, ref nState);
```

3.11.9. HRIF_DelMovePathJ

3.11.9.1. Description: Delete the specified track.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.9.2. example

```
// track name
string sTrackName = "drag_1";
// delete track
int nRet= hr_api.HRIF_DelMovePathJ(0,0, sTrackName);
```

3.11.10. HRIF_ReadTrackProcess

3.11.10.1.Description: Read the current trajectory movement progress.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

* Note: This interface is only for HRIF_MovePathL Valid, invalid for HRIF_MovePathJ.

✓ output variable

output variable	name	type of data	Effective range	content
dProcess	Track running progress	double	0~1	The progress of the current trajectory movement, the current movement progress > 0.999999 means the movement is completed
nIndex	point index	int	integer value >0	The point index to which the current trajectory moves

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.10.2.example

```
// track running progress
double dProcess = 0;
// point index
int nIndex = 0;
// read track status
int nRet= hr_api.HRIF_ReadTrackProcess(0,0, ref dProcess, ref nIndex);
```

3.11.11. HRIF_InitMovePathL

3.11.11.1.Description: Initialize the spatial trajectory movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	Track name, the track name of the current space track movement can be defined arbitrarily, the same track operation needs to execute HRIF_InitMovePathL, HRIF_PushMovePaths, HRIF_MovePathL, and call HRIF_MovePathL After calculating the trajectory, it will start to move directly, and the calculation time is 2-4s Left or right, determined according to the actual track size
dVelocity	Trajectory speed	double	-	Trajectory movement speed, unit [mm/s]
dAcc	Trajectory motion acceleration	double	-	Trajectory motion acceleration, unit [mm/s ^{2}]
dJerk	trajectory jerk	double	-	Track motion jerk, unit [mm/s ^{3}]
sTcpName	Tool Coordinate Name	string	-	The name of the tool coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name "TCP" can be used
sUcsName	User Coordinate Name	string	-	The name of the user coordinate system where the target space coordinates are located corresponds to the name of the teach pendant page, and the default name "Base" can be used

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.11.2.example

```
// track name
string sTrackName = "drag_1";
// define movement speed
double dVelocity = 10;
// define motion acceleration
```

```
double dAcc = 2500;
// Define motion jerk
double dJerk = 100;
// define tool coordinate variables
string sTcpName = "TCP";
// Define user coordinate variables
string sUcsName = "Base";
// Initialize joint continuous trajectory motion
int nRet= hr_api.HRIF_InitMovePathL(0,0, sTrackName, dVelocity, dAcc, dJerk, sUcsName, sTcpName);
```

3.11.12. HRIF_PushMovePathL

3.11.12.1.Description: Issue the movement track point.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name
dX -dRz	Space point	double	-	Target space location: dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinate, unit [°] dRy: R y coordinate, unit [°] dRz: Rz coordinate, unit [°]

* Note: After calling HRIF_InitPushMovePathL , this function can be called multiple times. Generally, the number of points needs to exceed 4.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.12.2.example

```
// track name
string sTrackName = "drag_1";
// Define the spatial target position
double dX = 420; double dY = 0; double dZ = 445;
double dRx = 180; double dRy = 0; double dRz = 180;
// Issue the spatial target point
int nRet= hr_api.HRIF_PushMovePathL(0,0, sTrackName, dX, dY, dZ, dRx, dRy, dRz);
```

3.11.13. HRIF_PushMovePaths

3.11.13.1.Description: Send track points in batches, and multiple point data can be sent once called.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name
nMoveType	Point type	int	0/1	0: Issue the joint point and share it with H RIF_ MovePathJ 1: Issue the space point, shared with H RIF_ MovePath L
nPointsSize	Number of points	int	-	The number of points must be consistent with the number in sPoints
sPoints	point data	string	-	Point data, each data is separated by a comma

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.13.2.example

```
// track name
string sTrackName = "drag_1";
// Movement type - MovePathL
int nMoveType = 1;
// number of points
int nPointsSize = 6;
string  sPoints="420,5,445,180,1,180,422,11,445,180,0,180,423,22,445,180,0,180,424,28,445,180,0,180,427,30,445,180,0,180,420,55,44  5,180,0,180";
// Issue the space point
int nRet= hr_api.HRIF_PushMovePaths(0,0, sTrackName, nMoveType, nPointsSize, sPoints);
// track name
string sTrackName = "drag_2";
// Movement type - MovePathJ
int nMoveType = 1 ;
// number of points
int nPointsSize = 9;
```

```
string sPoints = "420,4,445,180,0,180,420,7,445,180,0,180,420,23,445,180,1,180,420,30,445,180,0,180,420,37,445,
180,0,180,420,53,445 ,180,0,180,420,61,450,180,0,180,425,70,445,180,0,180,420,74,445,180,0,180";
// Issue the space point
int nRet= hr_api.HRIF_PushMovePaths(0,0, sTrackName, nMoveType, nPointsSize, sPoints);
```

3.11.14. HRIF_MovePathL

3.11.14.1.Description: Execute space coordinate trajectory movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name

* Note: call HRIF_MovePathL After calculating the trajectory, it will start to move directly, and the calculation time is 2-4s Left or right, determined according to the actual track size.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.14.2.example

```
// track name
string sTrackName = "drag_1";
// Start space continuous trajectory movement
int nRet= hr_api.HRIF_MovePathL(0,0, sTrackName);
```

3.11.15. HRIF_SetMovePathOverride

3.11.15.1.Description: Set MovePath speed ratio , MovePath The setting is valid during exercise.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
MovePathOverride _ _	set speed ratio	d double	1~100	The speed ratio that needs to be set

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.15.2.example

```
//speed
double dOverride = 0.1;
//set speed ratio
int nRet= hr_api.HRIF_SetMovePathOverride(0,0, dOverride);
```

3.11.16. HRIF_EndPushPathPoints

3.11.16.1.Description: End pushing poses to the trajectory and start trajectory calculation.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sPathName	Trajectory Name	string	-	Trajectory name, applicable to MovePathJ and MovePathL

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.16.2.example

```
// Trajectory name
string sPathName = "drag_01";
// Complete pushing and start trajectory calculation
int nRet = hr_api.HRIF_EndPushPathPoints(0, 0, sPathName);
```

3.11.17. HRIF_MovePathJOL

3.11.17.1.Description: Execute online planned joint motion, effective for joint trajectory planning of small path segments.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dVel	Joint Velocity	double	-	Joint motion velocity, unit [°/s].
dAcc	Joint Acceleration	double	-	Joint motion acceleration, unit [°/s²].
dTol	Blending Parameter	double	-	Blending parameter, unit [°].
sPoints	Pose Data	string	-	Pose data, with each data item separated by a comma. Each pose contains 6 coordinate values, so the number of values in this field must be a multiple of 6. The system automatically calculates the number of poses.
JOLPointsData	JOL Motion Poses	List<MovePathJOLPointsData>	-	MovePathJOL motion pose data. MovePathJOLPointsData is a structure that contains various data to be set for MovePathJOL motion poses.

✓ input variable

input variable	name	type of data	Effective range	content
dJ1	Joint Target Position	double	-	Joint 1 coordinate, unit [°]
dJ2	Joint Target Position	double	-	Joint 2 coordinate, unit [°]
dJ3	Joint Target Position	double	-	Joint 3 coordinate, unit [°]
dJ4	Joint Target Position	double	-	Joint 4 coordinate, unit [°]
dJ5	Joint Target Position	double	-	Joint 5 coordinate, unit [°]
dJ6	Joint Target Position	double	-	Joint 6 coordinate, unit [°]
nIsSetIO	IO Setting Flag	int	0/1	1: Set IO after the motion of this pose is completed;
nEndDOMask	EndDO Modification Bit Mask	int	-	EndDO0~EndDO7 correspond to bit0~bit7 of nEndDOMask. If a bit is 1, the value of the corresponding EndDO needs to be modified; otherwise, no modification is required.

nEndDOVal	EndDO Modification Bit Mask	int	-	EndDO0~EndDO7 correspond to bit0~bit7 of nEndDOVal.
nBoxDOMask	Target State of Modified EndDOs	int	-	EndDO0~EndDO7 correspond to bit0~bit7 of nEndDOVal. 0: Low level 1: High level
nBoxDOVal	BoxDO Modification Bit Mask	int	-	BoxDO0~BoxDO7 correspond to bit0~bit7 of nBoxDOMask. If a bit is 1, the value of the corresponding BoxDO needs to be modified; otherwise, no modification is required.
nBoxCOMask	Target State of Modified BoxDOs	int	-	BoxDO0~BoxDO7 correspond to bit0~bit7 of nBoxDOVal. 0: Low level 1: High level
nBoxCOVal	BoxCO Modification Bit Mask	int	-	BoxCO0~BoxCO7 correspond to bit0~bit7 of nBoxCOMask. If a bit is 1, the value of the corresponding BoxCO needs to be modified; otherwise, no modification is required.
nBoxAOCH0_Mask	BoxAOCH1 Modification Flag	int	0/1	0: No modification required 1: Modification required
nBoxAOCH0_Mode	Mode	int	1/2	1: Voltage mode 2: Current mode
nBoxAOCH1_Mask	BoxAOCH1 Modification Flag	int	0/1	0: No modification required 1: Modification required
nBoxAOCH1_Mode	Mode	int	1/2	1: Voltage mode 2: Current mode
dbBoxAOCH0_Val	Corresponding Analog Value	double	0~20	Current mode: 4-20mA Voltage mode: 0-10V
dbBoxAOCH1_Val	Corresponding Analog Value	double	0~20	Current mode: 4-20mA Voltage mode: 0-10V

* Note: Among the above parameters, the complete data of one point includes dJ1 to dbBoxAOCh1_Val in sequence, all of which are mandatory. A single issued command must contain data for at least 2 points and a maximum of 5000 points.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.17.2.example

```
double dVel = 100;
double dAcc = 100;
double dTol = 10;
```

```
// Define a List container to store objects of custom structure type MovePathJOLPointsData
List<MovePathJOLPointsData> JOLPointsDatas = new();
// Define joint movement distance
int JointMoveDistance = 2;
for (int i = 0; i < 100; i++) {
    // Define MovePathJOLPointsData object to store various data required for MovePathJOL motion poses
    MovePathJOLPointsData JOLPointsData;
    // Joint 1 coordinate position
    JOLPointsData.dbJ1 = 0 + i * JointMoveDistance;
    // Joint 2 coordinate position
    JOLPointsData.dbJ2 = 0;
    // Joint 3 coordinate position
    JOLPointsData.dbJ3 = 90;
    // Joint 4 coordinate position
    JOLPointsData.dbJ4 = 0;
    // Joint 5 coordinate position
    JOLPointsData.dbJ5 = 90;
    // Joint 6 coordinate position
    JOLPointsData.dbJ6 = 0;
    // IO setting flag
    JOLPointsData.nIsSetIO = 1;
    // EndDO modification bit mask
    JOLPointsData.nEndDOMask = 7;
    // Target state of modified EndDOs
    JOLPointsData.nEndDOVal = 2;
    // BoxDO modification bit mask
    JOLPointsData.nBoxDOMask = 86;
    // Target state of modified BoxDOs
    JOLPointsData.nBoxDOVal = 255;
    // BoxCO modification bit mask
    JOLPointsData.nBoxCOMask = 255;
    // Target state of modified BoxCOs
    JOLPointsData.nBoxCOVal = 169;
    // BoxAOCH0 modification flag
    JOLPointsData.nBoxAOCH0_Mask = 1;
    // Mode
    JOLPointsData.nBoxAOCH0_Mode = 2;
    // BoxAOCH1 modification flag
    JOLPointsData.nBoxAOCH1_Mask = 1;
```

```
    // Mode
    JOLPointsData.nBoxAOCH1_Mode = 1;
    // Corresponding analog value
    JOLPointsData.dbBoxAOCH0_Val = 6.66;
    // Corresponding analog value
    JOLPointsData.dbBoxAOCH1_Val = 9.99;
    // Store single pose information into JOLPointsDatas container
    JOLPointsDatas.Add(JOLPointsData);
}
// Execute online planned joint motion
int nRet = hr_api.HRIF_MovePathJOL(0, 0, dVel, dAcc, dTol, JOLPointsDatas);
```

3.11.18. HRIF_GetMovePathJOLIndex

3.11.18.1.Description: Get the current index and total number of poses of the online joint trajectory motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

input variable	name	type of data	Effective range	content
nCurIndex	Current Index	int	-	The index of the currently running pose
nBufIndex	Total Number of Poses	int	-	Total number of poses

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.18.2.example

```
int nCurIndex = 0;
int nBufIndex = 0;
// Get the current index and total number of poses of the online joint trajectory motion
int nRet = hr_api.HRIF_GetMovePathJOLIndex(0, 0, ref nCurIndex, ref nBufIndex);
```

3.11.19. HRIF_ReadPathState

3.11.19.1.Description: Execute space coordinate trajectory movement.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	track name	string	-	track name

✓ output variable

input variable	name	type of data	Effective range	content
nStateJ	MovePathJ State	int	0/1/2/3/4/5/9/10	0: The trajectory has not been taught yet. After a trajectory in teaching state ends teaching, if there are no raw poses, it enters this state; 1: The trajectory is being taught. This state is entered after InitPath and PushPathPoints are executed; 2: The trajectory is calculating target poses. The system automatically enters this state after detecting the "Teaching Completed" state. The duration of this state depends on the length of the trajectory and speed ratio, and may last more than 10 minutes. Trajectories in this state cannot be deleted; 3: The trajectory target pose calculation is completed. The system automatically enters this state after successfully calculating the target trajectory; 4: The trajectory teaching is ended. This state is entered after EndPushPathPoints is executed, and also after the post-processing of trajectory file import is completed; 5: The trajectory target pose calculation failed. The system automatically enters this state after an error occurs in target trajectory calculation; 9: The trajectory has been imported. This state is entered after a trajectory is imported from the front end; 10: The trajectory is undergoing post-import processing. The system automatically enters this state after detecting the "Imported" state.

				Trajectories in this state cannot be deleted.
nErrorCodeJ	MovePathJ Error Code	int	-	Refer to the total error code table. This parameter is valid when nStateJ = 5.
nStateL	MovePathL State	int	0/1/2/3/4/5/9/10	Same as nStateJ. Among them, the duration of state 2 depends on the length of the trajectory and speed ratio, and may last tens of seconds.
nErrorCodeL	MovePathL Error Code	int	-	Refer to the total error code table. This parameter is valid when nStateL = 5.

* Note: call HRIF_MovePathL After calculating the trajectory, it will start to move directly, and the calculation time is 2-4s Left or right, determined according to the actual track size.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.19.2.example

```
// Trajectory name
string sPathName = "drag_01";
// MovePathJ state
int nStateJ = 0;
// MovePathJ error code
int nErrorCodeJ = 0;
// MovePathL state
int nStateL = 0;
// MovePathL error code
int nErrorCodeL = 0;
// Read current trajectory state
int nRet = hr_api.HRIF_ReadPathState(0, 0, sPathName, ref nStateJ, ref nErrorCodeJ, ref nStateL, ref nErrorCodeL);
```

3.11.20. HRIF_ReadPathInfo

3.11.20.1.Description: Read information of the trajectory with the specified name.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sPathName	Trajectory Name	string	-	Trajectory name, applicable to MovePathJ and MovePathL

✓ output variable

input variable	name	type of data	Effective range	content
result	Trajectory Information	string	-	Information of the specified trajectory read. The returned result parameters are explained in sequence as follows: nRawDataType: Original pose type (0: Joint Pose; 1: Cartesian Pose) stateJ: MovePathJ state, 8 states: Not Taught (0), Teaching in Progress (1), Teaching Completed (4), Calculating (2), Calculation Failed (5), Calculation Completed (3), Imported (9), Post-Import Processing (10) ErrorCodeJ: MovePathJ error code stateL: MovePathL state (same as stateJ) ErrorCodeL: MovePathL error code dOverride: Trajectory motion speed ratio dRadius: Blending radius, unit: mm dLinearVel: Trajectory motion velocity, unit: mm/s dLinearAcc: Trajectory motion acceleration, unit: mm/s² dLinearJerk: Trajectory motion jerk, unit: mm/s³ dUcs_X-dUcs_Rz: User coordinates dTcp_X-dTcp_Rz: Tool coordinates nRawPointCount: Number of raw poses dHeadPoint_: Coordinates of the first raw pose (J1/J2/J3/J4/J5/J6 or X/Y/Z/RX/RY/RZ)

✓ return value

return value	name	type of data	Effective range	content

nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.20.2.example

```
// Trajectory name
string sPathName = "Path_01";
// Trajectory information
string result = "";
// Read information of the specified trajectory
int nRet = hr_api.HRIF_ReadPathInfo(0, 0, sPathName, ref result);
```

3.11.21. HRIF_ReadPathList

3.11.21.1.Description: Read the trajectory list.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0

✓ output variable

input variable	name	type of data	Effective range	content
result	Trajectory List	string	-	List of all trajectory names read

* Note:The returned result is string-type data, listing all trajectory names in the system one by one.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.21.2.example

```
// Trajectory list
string result = "";
// Read the trajectory list
int nRet = hr_api.HRIF_ReadPathList(0, 0, ref result);
```

3.11.22. HRIF_UpdateMovePathJName

3.11.22.1.Description: Update the name of the specified track.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sTrackName	original track name	string	-	original track name
sTrackNewName	updated track name	string	-	updated track name

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.22.2.example

```
// track name
string sTrackName = "drag_1";
// updated track name
string sTrackNewName = "drag_2";
// rename track name
int nRet= hr_api.HRIF_UpdateMovePathJName(0,0, sTrackName, sTrackNewName);
```

3.11.23. HRIF_UpdatePathName

3.11.23.1.Description: Update the name of the specified trajectory.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sPathName	Original Trajectory Name	string	-	Original name of the trajectory
sPathNewName	Updated Trajectory Name	int	0/1	Updated name of the trajectory

* Note:

For HR6.2.17 and earlier versions, only the HRIF_UpdateMovePathJName command can be used to update the name of the specified trajectory.

For HR6.3.3 and later versions, it is recommended to use this command. For HR6.5 and later versions, the HRIF_UpdateMovePathJName command will no longer be available.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.23.2.example

```
// Original trajectory name
string sPathName = "drag_01";
// Updated trajectory name
string sTrackNewName = "drag_02";
// Rename the trajectory
int nRet = hr_api.HRIF_UpdatePathName(0, 0, sPathName, sPathNewName);
```

3.11.24. HRIF_SetPathRefJoints

3.11.24.1.Description: Update the name of the specified track.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
sPathName	Trajectory Name	string	-	Name of the trajectory
dJ1-dJ6	Reference Joint Coordinates	double	-	Reference joint positions that the trajectory attempts to approach

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.11.24.2.example

```
// Trajectory name
string sPathName = "drag_01";
// Define reference joint positions
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
// Set reference joints for the trajectory
int nRet = hr_api.HRIF_SetPathRefJoints(0, 0, sPathName, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6);
```

3.12. Servo Motion Control Instructions

3.12.1.HRIF_StartServo

3.12.1.1. Description: When starting robot online control (ServoJ or ServoP), set position fixed update period or stiffness gain factor, and lookahead time.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default = 0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dServoTimeOrStifiness	Update Period/Stiffness Factor	double	>0	Fixed update period time, unit [s] /Stiffness gain factor
dLookaheadTime	look-ahead time	double	>0	Look-ahead time, unit [s]

* Note:

1. HRIF_StartServo/HRIF_PushServoJ/HRIF_PushServoP are a set of interfaces and are not shared with other Servo commands.
2. In versions prior to HR6.5.19, the dServoTimeOrStifiness parameter only indicates the update cycle.
3. In HR6.5.19 and later versions, the dServoTimeOrStifiness parameter has two meanings: update cycle and stiffness gain coefficient. Its specific meaning is determined by its value: when the value is in the range of 0~1, it represents the fixed update time cycle; when the value is 1 or greater, it represents the stiffness gain coefficient. It is recommended to operate with the stiffness gain coefficient as the parameter.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.12.1.2. example

```
// period
double dServoTimeOrStifiness= 0.02;
// look-ahead time
double dLookaheadTime =0. 2;
// Start robot online control
int nRet= hr_api.HRIF_StartServo(0,0, dServoTimeOrStifiness, dLookaheadTime);
// Stiffness gain factor
double dServoTimeOrStifiness= 8000;
// look-ahead time
```

```
double dLookaheadTime = 0.05;
// Start robot online control
int nRet= hr_api.HRIF_StartServo(0,0, dServoTimeOrStifiness, dLookaheadTime);
```

3.12.2.HRIF_PushServoJ

3.12.2.1. Description: Online joint position command control, the joint position is sent at a fixed update time set by StartServo, and the robot will track the joint position command in real time.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dJ1-dJ6	Joint point	double	-	Target joint position: dJ1: joint 1 position, unit [°] dJ2: Joint 2 position, unit [°] dJ3: Joint 3 position, unit [°] dJ4: Joint 4 position, unit [°] dJ5: joint 5 position, unit [°] dJ6: position of joint 6, unit [°]

* Note: HRIF_StartServo/HRIF_PushServoJ/HRIF_PushServoP form a set of interfaces and are not shared with other Servo commands. For HR6.5.19 and later versions, if the dServoTimeOrStifiness parameter is the stiffness gain coefficient, the time interval between two consecutive pose issuances should not exceed 100 controller cycles in principle. Otherwise, it will affect motion accuracy and even cause the servo to stop. The specific time depends on the controller.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.12.2.2. example

```
// Target joint position
double dJ1 = 0; double dJ2 = 0; double dJ3 = 90;
double dJ4 = 0; double dJ5 = 90; double dJ6 = 0;
// Online joint position command control
int nRet= hr_api.HRIF_PushServoJ(0,0, dJ1, dJ2, dJ3, dJ4, dJ5, dJ6);
```

3.12.3.HRIF_PushServoP

3.12.3.1. Description: Control the TCP position command at the end of the line, send the TCP position with the fixed update time set by StartServo, and the robot will track the target TCP position in real time and inversely calculate the converted joint position command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	Space point	double	-	Target space location: dX: X coordinate, unit [mm] dY: Y coordinate, unit [mm] dZ: Z coordinate, unit [mm] dRx: Rx coordinate, unit [°] dRy: Ry coordinate, unit [°] dRz: Rz coordinate, unit [°]
dTcp_X-dTcp_Rz	tool coordinates	double	-	The tool coordinates corresponding to the target position: dTcp_X : X coordinate, unit [mm] dTcp_Y : Y coordinate, unit [mm] dTcp_Z : Z coordinate, unit [mm] dTcp_Rx : Rx coordinates, unit [°] dTcp_Ry : Ry coordinate, unit [°] dTcp_Rz : R z coordinate, unit [°]
dUcs_X-dUcs_Rz	user coordinates	double	-	The user coordinates corresponding to the target position: dUcs_X : X coordinate, unit [mm] dUcs_Y : Y coordinate, unit [mm] dUcs_Z : Z coordinate, unit [mm] dUcs_Rx : Rx coordinate, unit [°] dUcs_Ry : Ry coordinate, unit [°] dUcs_Rz : Rz coordinate, unit [°]

* Note: HRIF_StartServo/HRIF_PushServoJ/HRIF_PushServoP form a set of interfaces and are not shared with other Servo commands. For HR6.5.19 and later versions, if the dServoTimeOrStifiness parameter is the stiffness gain coefficient, the time interval between two consecutive pose issuances should not exceed 100 controller cycles in principle. Otherwise, it will affect motion accuracy and even cause the servo to stop. The specific time depends on the controller.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.12.3.2. example

```
// Spatial target position
double dX = 420; double dY = 0; double dZ = 445;
double dRx = 180; double dRy = 0; double dRz = 180;
// define tool coordinate variables
double dTcp_X = 0; double dTcp_Y = 0; double dTcp_Z = 0;
double dTcp_Rx = 0; double dTcp_Ry = 0; double dTcp_Rz = 0;
// Define user coordinate variables
double dUcs_X = 0; double dUcs_Y = 0; double dUcs_Z = 0;
double dUcs_Rx = 0; double dUcs_Ry = 0; double dUcs_Rz = 0;
// online space position command control
int nRet= hr_api.HRIF_PushServoP(0,0, dX, dY, dZ, dRx, dRy, dRz, dTcp_X, dTcp_Y, dTcp_Z, dTcp_Rx, dTcp_Ry, dTcp_Rz, dUcs_X, dUcs_Y, dUcs_Z, dUcs_Rx, dUcs_Ry, dUcs_Rz);
```

3.12.4.HRIF_SpeedJ

3.12.4.1. Description: Online joint motion speed servo control, which moves for the specified duration at the speed and acceleration of each joint specified in this command.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dJ1-dJ6	Joint point	double	-	Updated command speed of each joint, unit: °/s
acc	Acceleration	double	-	Set joint acceleration, unit: °/s²
runtime	Runtime	double	0.001~86400	Maximum motion execution duration. If the command runs beyond this time, the motion will stop, unit: s

* Note:

1. The value range of joint speed varies with different machine models.

2. The motion initiated by this command can be terminated in advance using the GrpStop command, and can be paused/resumed using the GrpInterrupt/GrpContinue commands.

3. If a new successfully executed SpeedJ command is received before the maximum execution duration of the current command expires, the motion of the previous SpeedJ command will be terminated in advance, and the motion will continue with the parameters of the new SpeedJ command.

4. Before the motion triggered by this command ends, executing other waypoint motion commands will return a failure, and executing the SpeedL command will cause the robot to enter an error state.

5. The motion speed triggered by this command is not affected by the SetOverride command.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.12.4.2. example

```
// Command joint speed
double J1 = 5; double J2 = 3; double J3 = 3;
double J4 = 5; double J5 = 5; double J6 = 5;
// Define joint acceleration
double acc = 20;
// Define runtime
double runtime = 2;
// Issue joint command speed instruction
int nRet = hr_api.HRIF_SpeedJ(0, 0, J1, J2, J3, J4, J5, J6, acc, runtime);
```

3.12.5.HRIF_SpeedL

3.12.5.1. Description: Issue joint command speed instructions in SpeedL mode.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX -dRz	Space point	double	-	Updated command speed of coordinates: X, unit: mm/s Y, unit: mm/s Z, unit: mm/s RX, unit: °/s RY, unit: °/s RZ, unit: °/s
LinearAcc	X/Y/Z Acceleration	double	-	X/Y/Z acceleration, unit: mm/s²
AngularAcc	RX/RY/RZ Acceleration	double	-	RX/RY/RZ acceleration, unit: °/s²
runtime	Runtime	double	0.001~86400	Maximum motion execution duration. If the command runs beyond this time, the motion will stop, unit: s

* Note:

1. The value range of speed varies with different machine models.

2. The motion initiated by this command can be terminated in advance using the GrpStop command, and can be paused/resumed using the GrpInterrupt/GrpContinue commands.

3. If a new successfully executed SpeedL command is received before the maximum execution duration of the current command expires, the motion of the previous SpeedL command will be terminated in advance, and the motion will continue with the parameters of the new SpeedL command.

4. Before the motion triggered by this command ends, executing other waypoint motion commands will return a failure, and executing the SpeedJ command will cause the robot to enter an error state.

5. The motion speed triggered by this command is not affected by the SetOverride command.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.12.5.2. example

```
// Command space speed
double dVel_X = 20; double dVel_Y = 20; double dVel_Z = 20;
```

```
double dVel_RX = 2; double dVel_RY = 2; double dVel_RZ = 2;
// Define X/Y/Z acceleration
double LinearAcc = 100;
// Define RX/RY/RZ acceleration
double AngularAcc = 200;
// Define runtime
double runtime = 2;
// Issue joint command speed instruction
int nRet = hr_api.HRIF_SpeedL(0, 0, dVel_X, dVel_Y, dVel_Z, dVel_RX, dVel_RY, dVel_RZ, LinearAcc, AngularAcc, runtime);
```

3.13. Relative tracking motion control commands

3.13.1.HRIF_SetMoveTraceParams

3.13.1.1. Description: Set relative tracking motion control parameters.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Robot ID number, default=0
nState	track status	int	0/1	0: Disable relative tracking motion 1: Turn on relative tracking motion
dDistance	Relative distance to keep track of movement	double	>0	Relative distance to keep track of movement
dAwayVelocity	The away seek speed relative to the tracked motion	double	>0	The away seek speed relative to the tracked motion
dBackVelocity	Approach seek speed relative to tracked motion	double	>0	Approach seek speed relative to tracked motion

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.13.1.2. example

```
// Set the tracking state and the relative distance to keep
int nState = 1;
double dDistance = 100;
// seek speed relative to tracked motion
double dAwayVelocity = 50;
double dBackVelocity = 50;
// Set relative tracking motion control parameters and enable relative tracking motion
int nRet= hr_api.HRIF_SetMoveTraceParams(0,0, nState, dDistance,dAwayVelocity,dBackVelocity);
```

3.13.2.HRIF_SetMoveTraceInitParams

3.13.2.1. Description: Set relative tracking motion initialization parameters.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dK	Sensor Calculation Parameters	double	-	Calculation formula y = dK * x + dB
dB	Sensor Calculation Parameters	double	-	Calculation formula y = dK * x + dB
dMaxLimit	Maximum detection distance of laser sensor	double	-	Maximum detection distance of laser sensor
dMinLimit	Minimum detection distance of laser sensor	double	-	Minimum detection distance of laser sensor

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.13.2.2. example

```
// sensor calculation parameters
double dK= -32;
// sensor calculation parameters
double dB= 280;
// The maximum detection distance of the laser sensor
double dMaxLimit = 130;
// The minimum detection distance of the laser sensor
double dMinLimit = 65;
// Set the tracking state initialization parameters
int nRet= hr_api.HRIF_SetMoveTraceInitParams(0,0, dK, dB, dMaxLimit, dMinLimit);
```

3.13.3.HRIF_SetMoveTraceUcs

3.13.3.1. Description: Sets the tracking seek direction relative to the tracking motion.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
dX - dRz	track search direction	double	>-180,<180	Target space location: dX: invalid, can be set to 0 dY: invalid, can be set to 0 dZ: invalid, can be set to 0 dRx: Rx direction, unit [°] dRy: Ry direction, unit [°] dRz: Rz direction, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.13.3.2. example

```
// set track direction
double dX = 0; double dY = 0; double dZ = 0;
double dRx = 180; double dRy = 0; double dRz = 0;
// set track direction
int nRet= hr_api.HRIF_SetMoveTraceUcs(0,0, dX, dY, dZ, dRx, dRy, dRz);
```

3.13.4.HRIF_SetTrackingState

3.13.4.1. Description: Set the conveyor belt tracking motion state.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nState	track status	int	0/1	0: Turn off conveyor tracking motion 1: Turn on the conveyor belt tracking movement

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.13.4.2. example

```
// Set conveyor tracking to on
int nState = 1;
// enable conveyor tracking
int nRet= hr_api.HRIF_SetTrackingState(0,0, nState);
```

3.14. Position follow motion instruction

3.14.1.HRIF_SetPoseTrackingMaxMotionLimit

3.14.1.1. Description: Set the maximum following speed of position following

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	int	0~5	Electric box ID number, default=0
wxya	robot id	int	0 ~ 5	Machine ID number, default=0
dMaxLineVel	straight line speed	double	> 0	The position follows the line and the maximum following speed, unit [mm / s]
dMaxOriVel	Attitude maximum speed	double	> 0	The maximum following speed of position following attitude, unit [°/ s]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.14.1.2. example

```
// Maximum speed in a straight line
double MaxLineVel = 100;
//Attitude maximum speed
double MaxOriVel = 90;
// call interface
int nRet = hr_api.HRIF_SetPoseTrackingMaxMotionLimit(0, 0, MaxLineVel, MaxOriVel);
```

3.14.2.HRIF_SetPoseTrackingPIDParams

3.14.2.1. Description: Set PID parameters

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	int	0~5	Electric box ID number, default=0
wxya	robot id	int	0 ~ 5	Machine ID number, default=0
dPosPID1	Position follows PID	double	-	position follows PID, default = 5
dPosPID2	Position follows PID	double	-	position follows PID, default = 0.1
dPosPID3	Position follows PID	double	-	position follows PID, default = 0
dOriPID1	Attitude follows PID	double	-	Attitude follows PID, default = 5
dOriPID2	Attitude follows PID	double	-	Pose follows PID, default = 0.1
dOriPID3	Attitude follows PID	double	-	Attitude follows PID, default = 0

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.14.2.2. example

```
//position follows PID 1
double dPosPID1 = 5;
//position follows PID 2
double dPosPID2 = 0.1;
//position follows PID 3
double dPosPID3 = 0;
//Bit posture follows PID 1
double dOriPID1 = 5;
//Bit posture follows PID 2
double dOriPID2 = 0.1;
// Bit posture follows PID 3
double dOriPID3 = 0;
```

// call interface

int nRet = hr_api.HRIF_SetPoseTrackingPIDParams(0, 0, dPosPID1, dPosPID2, dPosPID3, dOriPID1, dOriPID2, dOriPID3);

3.14.3.HRIF_SetPoseTrackingTargetPos

3.14.3.1. Description: Set the target position for position following

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	int	0~5	Electric box ID number, default=0
nRbtID	robot id	int	0 ~ 5	Machine ID number, default=0
dX	The distance to keep in the X direction	double	-	The distance maintained in the X direction, unit [mm]
dY	The distance to keep in the Y direction	double	-	The distance maintained in the Y direction, unit [mm]
dZ	The distance to keep in the Z direction	double	-	The distance maintained in the Z direction, unit [mm]
dRx	The distance maintained in the Rx direction	double	-	The distance maintained in the Rx direction, unit [°]
dRy	The distance maintained in the Ry direction	double	-	The distance maintained in the y direction, unit [°]
dRz	The distance maintained in the Rz direction	double	-	The distance maintained in the z direction, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.14.3.2. example

```
//The distance maintained in the x direction
double dX = 0;
//The distance maintained in the y direction
double dY = 0;
```

```
// The distance maintained in the z direction
double dZ = 100;
// The distance maintained in the Rx direction
double dRx = 0;
// The distance maintained in the Ry direction
double dRy = 0;
// The distance maintained in the Rz direction
double dRz = 0;
// call interface
int nRet = hr_api.HRIF_SetPoseTrackingTargetPos(0, 0, dX, dY, dZ, dRx, dRy, dRz);
```

3.14.4.HRIF_SetPoseTrackingState

3.14.4.1. Description: Set the position follow state

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	int	0~5	Electric box ID number, default=0
nRbtID	robot id	int	0 ~ 5	Machine ID number, default=0
nState	position following state	int	0/1	0 : Turn off position follow 1: Turn on position follow

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.14.4.2. example

```
//Position follow state
int nState = 1;
// call interface
nRet = hr_api.HRIF_SetPoseTrackingState(0, 0, nState);
```

3.14.5.HRIF_SetUpdateTrackingPose

3.14.5.1. Description: Set real-time update sensor location information

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	int	0~5	Electric box ID number, default=0
nRbtID	robot id	int	0 ~ 5	Machine ID number, default=0
dX	The detected distance in the X direction	double	-	The detected distance in the X direction, unit [mm]
dY	detected distance in the Y direction	double	-	The detected distance in the Y direction, unit [mm]
dZ	detected distance in the Z direction	double	-	The detected distance in the Z direction, unit [mm]
dRx	Detected distance in Rx direction	double	-	The detected distance in the Rx direction, unit [°]
dRy	Detected distance in Ry direction	double	-	The detected distance in the Ry direction, unit [°]
dRz	Detected distance in Rz direction	double	-	The detected distance in the Rz direction, unit [°]

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	integer value >0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.14.5.2. example

```
//The detected distance in the x direction
double dX = 0;
//The detected distance in the y direction
double dY = 0;
// The detected distance in the z direction
double dZ = 100;
//The distance detected in the Rx direction
```

```
double dRx = 0;
//Detected distance in the R y direction
double dRy = 0;
//The detected distance in the R z direction
double dRz = 0;
// call interface
int nRet = hr_api.HRIF_SetUpdateTrackingPose(0, 0, dX, dY, dZ, dRx, dRy, dRz);
```

3.15. other instructions

3.15.1.HRIF_HRAppCmd

3.15.1.1. Description: Execute the plug-in App Order.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
HRAppName	plugin name	string	-	plugin name
HRAppCmdName	command name	string	0/1	command name
sParams	parameter list	string	-	parameter list

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.15.1.2. example

```
//plugin name
string HRAppName = "MT_Kinco";
// set command name
string HRAppCmdName = "MotorPowerOn";
// Setting parameters
string sParams = "1,100,10";
// Execute the plugin command
int nRet = hr_api.HRIF_HRAppCmd(0, HRAppName, HRAppCmdName, sParams);
```

3.15.2.HRIF_HRAppData

3.15.2.1. Description: Execute the plug-in App Order and read the data returned.

✓ 输入变量

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
HRAppName	plugin name	string	-	plugin name
HRAppCmdName	command name	string	0/1	command name
sParams	parameter list	string	-	parameter list

✓ output variable

input variable	name	type of data	Effective range	content
Data	the data returned	List<string>	-	data returned by the plugin

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

1.1.1.1. 示例

```
//plugin name
string HRAppName = "MT_Kinco";
// set command name
string HRAppCmdName = "MotorPowerOn";
// Setting parameters
string sParams = "1,100,10";
// Initialize list to store returned data
List<string> Data = new List<string>();
// Execute the plugin command
int nRet = hr_api.HRIF_HRAppData(0, HRAppName, HRAppCmdName, sParams,ref Data);
```

3.15.3.HRIF_WriteEndHoldingRegisters

3.15.3.1. Description: Write to the Modbus slave registers of the end connection.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nSlaveID	Slave ID	int	1~255	Slave ID
nFunction	function code	int	>0	function code: 0x01-read coil register 0x02 - Read Line Discrete Input Register 0x03-read holding register 0x04 - read input register 0x05 - write single coil register 0x06 - Write a single holding register 0x0f - write multiple coil registers 0x10 - write multiple holding registers
nRegAddr	register address	int	>0	Register start address
nRegCount	number of registers	int	>0	number of registers
vecData	register data	vector<int>	-	register data

* Note: The end is EtherCAT Bus version IO valid.

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.15.3.2. example

```
// Set slave ID=1
int nSlaveID = 1;
// set function code
int nFunction = 16;
// Set the register start address
int nRegAddr = 5000;
// Set the number of registers
int nRegCount = 5;
// set register data
```

```
List<int> vecData = new List<int>();
vecData. Add(88);
vecData. Add(89);
vecData. Add(90);
vecData. Add(91);
vecData. Add(92);
// write register data
int nRet = hr_api.HRIF_WriteEndHoldingRegisters(0, 0, nSlaveID, nFunction, nRegAddr, nRegCount, vecData);
```

3.15.4.HRIF_ReadEndHoldingRegisters

3.15.4.1. Description: Read the Modbus slave registers of the end connection.

✓ input variable

input variable	name	type of data	Effective range	content
boxID	electrical box ID	unsigned int	0~5	Electric box ID number, default=0
rbtID	Robot ID	int	0~5	Machine ID number, default=0
nSlaveID	Slave ID	int	1~255	Slave ID
nFunction	function code	int	>0	function code: 0x01-read coil register 0x02 - Read Line Discrete Input Register 0x03-read holding register 0x04 - read input register 0x05 - write single coil register 0x06 - Write a single holding register 0x0f - write multiple coil registers 0x10 - write multiple holding registers
nRegAddr	register address	int	>0	Register start address
nRegCount	number of registers	int	>0	number of registers

* Note: The end is EtherCAT Bus version IO valid.

✓ output variable

input variable	name	type of data	Effective range	content
vecData	register data	vector<int>	-	register data

✓ return value

return value	name	type of data	Effective range	content
nRet	return value	int	Integer value >=0	nRet=0: return the function call successfully nRet>0: return the error code of the call failure

3.15.4.2. example

```
// Set slave ID=1
int nSlaveID = 1;
// set function code
int nFunction = 3;
// Set the register start address
int nRegAddr = 5000;
```

```
// Set the number of registers
int nRegCount = 5;
// read register data
List<int> vecData = new List<int>();
int nData1 = 0;
int nData2 = 0;
int nData3 = 0;
int nData4 = 0;
int nData5 = 0;
// read register data
int nRet = hr_api.HRIF_ReadEndHoldingRegisters(0, 0, nSlaveID, nFunction, nRegAddr, nRegCount, ref vecData);
```

Chapter4 appendix

4.1. reference document

1. " HuayanRobot_ErrorCode.docx " for error code and state machine description .
2. For the use of Huayan robot communication protocol, please refer to " HuayanRobotV6Control Communication Protocol Interface" .
3. Huayan Robot C++ SDK Please refer to " HuayanRobot Library C++ " for usage .
4. Huayan Robot Java SDK Please refer to " HuayanRobot Library Java " for usage .
5. Huayan Robot Python SDK Please refer to " HuayanRobot Library Python " for usage .